LIVE-CODING INTERVIEW PREP

Flutter API Integration Clean Architecture + BLoC / Cubit

20-Minute Live Coding Interview Guide
+ audit and upgrade path for your `demoapi` project

> **Priority order:** Correctness → Clean Architecture → Readability → Speed → Explainability

Part 1 · Strict audit of your current code: **39 / 100**, 20 likely interviewer questions

Part 2 · Roadmap: take that exact code from 39 to 100 in 10 steps

Part 3 · The guide: architecture, 20-minute plan, LITE + FULL code, pagination, search, login, errors, interceptors, refresh token, testing, cheat sheet, 34 Q&A

> Every code block was compiled with `flutter analyze` (strict, 0 issues) and tested (74 passing tests, ≈98% line coverage). GET and POST/login flows were run end-to-end against `reqres.in`.

Flutter 3.47 · Dart 3.13 · flutter_bloc 9 · dio 5 · October 2026

Contents

Part 1 · Audit of your current code (39/100) 3
0. Verdict in 60 seconds 3
1. Clean Architecture audit 3
2. BLoC / Cubit audit 4
3. API integration audit 5
4. Model vs Entity audit 6
5. Repository pattern audit 6
6. Dependency injection audit 7
7. Folder structure audit 7
8. SOLID / DRY / KISS / YAGNI 7
9. Live-coding score (strict) 8
10. The 20 questions an interviewer will ask about your implementation 8
11. What to change: exact, ordered list (≈25 min) 10
12. Final block 11

Part 2 · Roadmap: from 39 to 100 12
1. The score trajectory 12
2. File-by-file migration map 13
STEP 1: Fix the layering and the names (39 → 54) 14
STEP 2: Model → Entity with toEntity() (54 → 59) 16
STEP 3: Configure Dio, finish DioClient, implement exceptions (59 → 66) 18
STEP 4: Failures and guard() (66 → 75) 20
STEP 5: Cubit, states, pagination and UI (75 → 83) 22
STEP 6: Dependency injection (83 → 86) 24
STEP 7: Real tests and cleanup (86 → 87) 26
STEP 8: Prove it generalises: POST, secure tokens, interceptor (87 → 86) 27
STEP 9: Hardening round: what a strict re-audit found (86 → 98) 28
STEP 10: Speed: the last 2 points (98 → 100) 33
The "100/100" checklist (what an interviewer scans for) 33
The final scorecard (re-audited on the real project) 33
Status: applied 34

Part 3 · Flutter API Integration — 20-Minute Live Coding Guide 35
Read this first: the two tiers (honesty about 20 minutes) 35
SECTION 1 — The Master Architecture 36
SECTION 2 — Folder Structure 38
SECTION 3 — The 20-Minute Implementation Plan (stopwatch) 39
SECTION 4 — Basic API: GET /api/users?page=1, from main.dart to UI 40
SECTION 5 — One pattern for every endpoint 55
SECTION 6 — Complex API response 57
SECTION 7 — Pagination 59
SECTION 8 — Search 61
SECTION 9 — POST / Login 65
SECTION 10 — Error handling 69
SECTION 11 — Dio interceptors 71
SECTION 12 — Authentication 72
SECTION 13 — Retry & Refresh token 73
SECTION 14 — Cubit vs BLoC 75
SECTION 15 — Model vs Entity 77
SECTION 16 — UseCase 78
SECTION 17 — Testing 79
SECTION 18 — Interview Cheat Sheet (one page) 96
SECTION 19 — Verbal explanation scripts 97
SECTION 20 — Common interview questions (34, beginner → advanced) 98
SECTION 21 — The Final Live-Coding Template (memorize this) 100

Part 1 · Audit of your current code (39/100)

> Auditor stance: strict senior Flutter interviewer (Deloitte-style live-coding round). Scope: every file under `lib/` and `test/` as it exists today, plus `pubspec.yaml`. Evidence: `flutter analyze` → **0 issues** (compiles, lints clean) · `flutter test` → **1 failing test** (the untouched Flutter counter template; crashes with `LateInitializationError: Field 'userCubit' has not been initialized`). **Note:** this audit describes the code **as it was before the fix**. The fixes in Part 2 have since been applied to the project (original code backed up in `demoapi_backup_before_100/`). Lesson already visible: "*analyzer-clean*" *is not* "*architecture-clean*". Your code compiles, but the dependency rule is broken in two files and the error layer is empty.

0. Verdict in 60 seconds

Score	**39 / 100** (rubric in §9)
Class	**Beginner → Intermediate border.** Structure is Intermediate. Execution (dependency direction, errors, pagination, tests) is Beginner. Not yet Interview Ready.
Good news	Almost all of it is fixable in ~25 minutes of focused editing, and the *shape* is right: feature-first folders, interface in domain / impl in data, UseCase → Cubit, sealed states, Dio injected, DI chain in the right order.
Worst problem	`domain/` imports from `data/` (2 files). In Clean Architecture that is the single most checked rule. An interviewer who opens `user_repository.dart` sees line 1 and the conversation changes tone.
Second worst	`core/erros/exceptions.dart` and `failuers.dart` are **empty (0 bytes)**; the data source throws a bare `Exception(e.message)` and the Cubit shows `e.toString()`. Zero real error handling.
Third worst	Half-built pagination: `isLoadingMore`, `totalPages`, `currentPages` exist in state but nothing ever uses them.

1. Clean Architecture audit

1.1 What the dependency direction *should* be

There are two different arrows people mix up. Interviewers love this distinction.

```
CALL direction (who invokes whom at runtime)
UI → Cubit → UseCase → Repository(interface) → RepositoryImpl → DataSource → Dio → API

DEPENDENCY direction (who imports whom at compile time)
presentation ──▶ domain ◀── data
                   ▲
          (domain imports NOTHING from the other two,
           and nothing from Flutter or Dio)
```

`RepositoryImpl` is called *through* the interface, but it **imports** the domain (to implement it). That inversion (Dependency Inversion Principle) is what "Clean" means.

1.2 Checklist against your code

Question you asked	Verdict	Evidence
Does **Domain depend on Data**?	❌ **YES — violation**	`lib/feature/user/domain/repository/user_repository.dart:1` → `import '../../data/model/user_response.dart';` and `lib/feature/user/domain/usecase/get_users.dart:1` → `import 'package:demoapi/feature/user/data/model/user_response.dart';`
Does the **UI call Dio** directly?	✅ No	`user_page.dart` only imports cubit/state.
Does the **Cubit call Dio** directly?	✅ No	`UserCubit` takes `GetUsers`.
Is the **repository interface in Domain**?	⚠️ Yes by location, ❌ by content	It lives in `domain/repository/` but its return type is a *data-layer* class, so it is not really a domain contract.
Is the **repository implementation in Data**?	✅ Yes	`data/repositories/user_repository_impl.dart`

Question you asked	Verdict	Evidence
Are **Models leaking into Presentation**?	⚠️ Indirectly	`UserResponse` (a `data/model` class) flows through UseCase into the Cubit (`response.users` , `response.page`). The Cubit file never imports it, so the leak is hidden by type inference, which is worse because it is hard to spot.
Are **Entities separated from Models**?	⚠️ Half	`User` ↔ `UserModel` is separated (via `extends`). The paginated wrapper is not: `UserResponse` is a business-shaped class stored in `data/model/` , and `UserResponseModel extends UserResponse` holds `List<User>` (entities) inside a model.
Are **use cases useful** or unnecessary?	⚠️ Pass-through	`GetUsers.call = repository.getUsers(page: page)` . For *interview* purposes keep it (it is the expected layering and a seam for rules). Just be ready to defend it (see Q5).
Is the **data source responsibility** correct?	⚠️ Mostly	It does HTTP + parse (right) but also builds the absolute URL (`'${ApiConstants.baseUrl}${ApiConstants.users}'` , `user_remote_source_source.dart:16`) and converts errors (wrongly, see §3). Base URL belongs in Dio's `BaseOptions` .

1.3 Concrete issues (ordered by damage)

1. **Domain → Data import (2 files).** Root cause: `UserResponse` is in the wrong layer. It is `{page, perPage, total, totalPages, users}` , i.e. a *business concept* ("a page of users"). Move to `domain/entity/` , rename to `PagedUsers` , drop `perPage` / `total` (no screen uses them: YAGNI).
2. **Interface file and implementation file names are swapped.** - `data-source/user_remote_data_source_impl.dart` contains the **abstract** `UserRemoteDataSource` . - `data-source/user_remote_source_source.dart` contains the **Impl** (and the filename has "source_source"). Anyone navigating by filename opens the wrong class. In a screen-share this costs seconds and credibility.
3. **Typos baked into paths:** `core/erros/` , `failuers.dart` , `user_title.dart` (should be `user_tile`), `data-source/` (hyphen; Dart convention is `snake_case`), `repositories` vs `repository` . Rename now; typos in folder names look careless when imports are visible on screen.
4. **Empty files committed:** `exceptions.dart` , `failuers.dart` , `user_title.dart` are 0 bytes. Either implement or delete; never leave dead scaffolding in front of an interviewer.
5. **Dead code:** `core/network/dio_client.dart` (GET-only wrapper) is never used. `Injection.dio` is a bare `Dio()` . Two half-ideas competing.
6. **Mixed import styles:** `package:demoapi/...` in cubit/usecase/page, relative in the rest. Pick one (the guide uses package imports for cross-layer, relative within a file's own folder tree is fine; just be consistent).

1.4 Over-engineered?

Item	Verdict
4 classes for one list (`User` , `UserModel` , `UserResponse` , `UserResponseModel`)	Slightly. 4 is right *if* mapped properly with `toEntity()` ; today they relate by inheritance, which is the confusing part, not the count. Target: `User` , `PagedUsers` (domain) + `UserModel` , `UsersResponseModel` (data).
`UserModel.toJson()`	Unused for GET: YAGNI. Keep only when you POST that shape.
DataSource interface	Optional in a 20-min round (mocktail can mock the concrete class). Keep it because the prompt/interviewer expects it; skip it *first* if you are behind.
A separate `Injection` class with `static late` fields	Over-built and dangerous (see §6).

2. BLoC / Cubit audit

Area	Verdict	Evidence / why
Responsibility	✅	Cubit only orchestrates `GetUsers` → emit. No Dio, no JSON.
Calls the right abstraction	✅	`GetUsers` (use case), not the repository impl.
State design	⚠️	`sealed` hierarchy is the right idea. Missing `Equatable` (it's in your `pubspec.yaml` but unused) → states have no value equality: duplicate emissions rebuild, and tests can't `expect(..., [UserLoading(), UserLoaded(...)])` .
Loading / Success / Error	✅/⚠️	Present. Error is `e.toString()` → user sees `Exception: Something went wrong` .
Pagination state	❌	`UserLoaded` has `isLoadingMore` , `currentPages` (typo: plural), `totalPages` ; **no code ever sets `isLoadingMore`** , there is no `loadMore()` , and `fetchUsers(page:)` *replaces* the list instead of appending.
Loading-more state	❌	See above: dead field.
Immutability	⚠️	Fields are `final` ✅, but no `copyWith` (needed for append) and `List<User>` is exposed mutable (low risk).

Area	Verdict	Evidence / why
Business logic in UI	✅	None. But the UI also does no work for pagination (no trigger).
Business logic in Cubit	⚠️	Fine today; will bloat once pagination is added without guards (duplicate calls).
`catch (e)`	❌	Catches *everything*, including programmer errors (`TypeError` , `StateError`), then prints them to users. Catch the typed `Failure` only.
Lifecycle	❌	No `if (isClosed) return;` before `emit` after `await` . User leaves the screen mid-request → `emit` on a closed Cubit.
Created via DI	❌	`Injection.userCubit` is a **singleton** (`injection.dart:27`) handed to `BlocProvider(create:)` (`main.dart:21`). `BlocProvider` closes what it creates on dispose → the next time the screen is built the *closed* instance is reused → "Cannot emit new states after calling close". Cubits are **factories**.
`initState` + `context.read`	⚠️	`user_page.dart:16-19` works but leaves a `// TODO: implement initState` comment and couples fetching to widget lifecycle. Prefer `create: (_) => sl<UserCubit>()..fetchUsers()` .

Is `UI → Cubit → UseCase` appropriate?

Yes. The Cubit is the **presentation logic holder** (what state does the screen show?). The UseCase is the **application action** (get users). The repository is the **data gateway**. The Cubit should never know *how* data is fetched, only *what* to ask for. For a simple list the use case is thin; its value is (1) a single named business action per file, (2) a place for rules (validation, combining repositories), (3) trivially mockable in Cubit tests.

Cubit vs Bloc for a live-coding interview

Use Cubit (default)	Use Bloc when…
Direct methods (`fetchUsers()`), less boilerplate, faster to type and to explain	Events must be **traced/logged**, or you need **event transformers** (`restartable()` for search, `droppable()` for pagination, `debounce`)
Request/response style screens: list, details, login, form submit	Many different triggers mutate the same state, or you need replay/undo/analytics on "what happened"
20-minute budget	The interviewer says "use Bloc", or the problem is clearly event-driven (typing search + pagination + refresh racing each other)

Say this in the interview: "*I'll use Cubit for speed and clarity; if search + pagination start racing I'd switch that screen to Bloc to get `restartable()` .*"

---

3. API integration audit

Area	Verdict	Evidence / comment
Dio configuration	❌	`Dio()` with defaults (`injection.dart:19`): no `baseUrl` , no timeouts, no headers. Dio has no default timeout of its own, so a dead connection hangs until the OS gives up (often a minute or more): the user stares at a spinner.
Base URL	⚠️	In `ApiConstants` , but concatenated by hand at the call site. Belongs in `BaseOptions(baseUrl: …)` so endpoints are just `'/users'` .
Endpoints	⚠️	Constant exists (`users`) ✅ , but base URL and path are both concatenated.
Query params	✅	`{'page': page}` is correct.
Headers	❌	None (not even `Accept`). Fine for reqres today, but you can't show where tokens/headers would go.
HTTP methods	❌	Only GET exists. The unused `DioClient` only wraps GET. Adding POST/PUT/PATCH/DELETE means copy-pasting the same `try/catch` into every data-source method (violates DRY).
Response parsing	⚠️	`response.data as Map<String, dynamic>` ✅ . But `json['id'] ?? 0` , `?? ''` , `?? []` **invent data** when the contract breaks (a user with id 0 and empty email renders as a blank tile). Prefer failing loudly (`as int`) and map the failure to a `ParsingFailure` .
JSON serialization	✅	Hand-written `fromJson` is the right call for an interview (no codegen to wait for).
Error handling	❌❌	`catch (DioException e) { throw Exception(e.message ?? 'Something went wrong'); }` (`user_remote_source_source.dart:21-23`). Loses status code and error type. For `badResponse` , `e.message` is often null, so *every* error becomes "Something went wrong". 401, 404, timeout, offline are indistinguishable. Server's body `{"error": "Missing password"}` is thrown away.

Area	Verdict	Evidence / comment
Timeout handling	❌	None.
API vs network exceptions	❌	Not distinguished.
Empty response	⚠️	`?? []` handles missing `data`, but there is no *empty state* in the UI (`ListView` of 0 items = blank screen).
Invalid response	❌	`TypeError` / `FormatException` from parsing isn't mapped; `catch (e)` in the Cubit prints a raw Dart error.

Can it easily support GET / POST / PUT / PATCH / DELETE? Not today. The *architecture* can, but the *data layer plumbing* isn't there: each verb would duplicate try/catch + error mapping. A single wrapper (`DioClient` with `get/post/put/patch/delete` + one `_send` that converts `DioException` → `AppException`) turns every future endpoint into 2–3 lines. You already started that class and abandoned it; finishing it is the fix.

---

4. Model vs Entity audit

Current approach: `UserModel extends User`, `UserResponseModel extends UserResponse`.

	`extends` (current)	`toEntity()` (recommended)
Typing speed	Fastest: `super.id` …	+6–8 lines per model (`toEntity()`)
Domain independence	❌ `User` is a plain class, fine, but the pattern invites putting `fromJson` on the entity "since it's one step"	✅ Entity never knows JSON exists
Backend renames a field	Breaks `User` and everything above it	Only `UserModel.fromJson` changes
Model has extra fields (`perPage`, `total`, `meta`) the entity doesn't want	Impossible without bloating the entity	Natural
Where it hurt **you**	`UserResponse` ended up in the data layer *because* the model extended it, which pulled the domain contract into data. That is the exact failure mode of the shortcut.	n/a
Clarity in interview	"It's a subtype" (interviewer: *so the domain is shaped like the JSON?*)	"Model parses, entity is business, mapper between them" (one sentence, no follow-up)

Recommendation. *Live coding:* use `toEntity()`. It's ~30 seconds more typing and removes an entire line of follow-up questions. If you are genuinely out of time, `extends` is acceptable **only if the entity still lives in `domain/` and has no JSON code**; say out loud: "*Shortcut for time; production code maps with toEntity().*" *Production:* always `toEntity()` (or `freezed` + mappers).

Other findings:

- `json['id'] ?? 0` silently turns bad data into valid-looking data. Use `as int` and let `guard()` convert `TypeError` → `ParsingFailure`.
- `User` has no `==` / `hashCode`: list equality in states/tests fails. Make entities `Equatable`.
- The UI builds `'${user.firstName} ${user.lastName}'` inline; a `fullName` getter on the entity is a cleaner single source.

---

5. Repository pattern audit

	Verdict
`UserRepository` (interface) in Domain	✅ location, ❌ content (returns data-layer `UserResponse`).
`UserRepositoryImpl` in Data	✅ location. Body is pass-through: **no exception → failure mapping**, which is the repository's main job.
UI doesn't know the implementation	✅ UI → Cubit → UseCase → *interface*. (The only place that knows `UserRepositoryImpl` is the DI file: correct.)
Testability	⚠️ In theory excellent (swap in a fake repository; see widget test in the guide). In practice zero tests exist.

Why the interface belongs in Domain: the *domain* defines what it needs ("give me a page of users"); the *data* layer adapts to that need. If the interface lived in data, domain would have to import data to use it, reversing the dependency. Why UI must not know the implementation: so swapping REST → GraphQL → cache → fake requires **zero** changes above the interface. Why it makes testing easy: a `Mock` / `Fake` implementing a 1-method interface is 3 lines; no HTTP, no Dio.

---

6. Dependency injection audit

Your chain:

```
Dio → RemoteDataSource → RepositoryImpl → GetUsers → UserCubit → UI
```

Order: correct ✅. Types on the left-hand side are the interfaces (`UserRemoteDataSource` , `UserRepository`) ✅ (that's DIP in the DI file).

Issue	Where	Why it matters
`static late` mutable globals	`injection.dart:11-16`	Any code can reassign them; reading before `init()` crashes with `LateInitializationError` , which is **exactly why your `flutter test` is red**.
Cubit is a singleton	`injection.dart:27`	Closed-cubit bug described in §2. Use `registerFactory` .
`get_it` in `pubspec.yaml` but unused	pubspec	Reviewer reads that as unfinished. Use it or remove it.
4 of the 5 `static late` fields are only there to hold intermediate objects	`injection.dart`	Only the *composition* matters; nothing needs to be stored.
`Dio()` unconfigured	`injection.dart:19`	See §3.

GetIt or manual for 20 minutes? Manual first, composed once in `main.dart` (a "composition root"): 6 lines, zero packages, zero ceremony, and easy to explain ("*one place knows all concrete classes*"). Move to **GetIt** only if (a) the interviewer asks how it scales, or (b) you already have 2+ features. Migration is mechanical (the guide shows both). The *fastest clean* version is:

```dart
final repository = UserRepositoryImpl(UserRemoteDataSourceImpl(DioClient(dio)));
BlocProvider(create: (_) => UserCubit(GetUsers(repository))..fetchUsers(), child: const UserPage());
```

---

7. Folder structure audit

Your tree is **the right shape**, with the wrong spelling:

```
lib/
├── core/{constants, di, errosx, network}      → errors
├── feature/user/
│   ├── data/{data-sourcex, model, repositoriesx}  → data_source, repository
│   ├── domain/{entity, repository, usecase}       ✅ (add paged_users.dart here)
│   └── presentation/{cubit, pages, widgetx}       → widgets
└── main.dart
```

Rename / move	Why
`core/erros` → `core/errors` ; `failuers.dart` → `failures.dart`	Typos
`data-source/` → `data_source/`	Dart naming convention (`snake_case` everywhere)
`data/repositories/` → `data/repository/`	Consistent with `domain/repository/` (same word both sides is easier to find)
`presentation/widget/user_title.dart` (empty) → `presentation/widgets/user_tile.dart`	Typo + give it content (move the `ListTile` out of the page)
`data/model/user_response.dart` → `domain/entity/paged_users.dart`	Fixes the dependency violation
Swap contents of the two data-source files	Names must match classes

Do **not** add more folders (no `utils/` , `helpers/` , `services/` , `bloc/` + `cubit/`). Add `core/storage/` only when you implement token storage.

---

8. SOLID / DRY / KISS / YAGNI

Principle	Verdict	Why
Single responsibility	⚠️	Classes are small ✅. Data source also builds absolute URLs and converts errors; DI class also *stores* the object graph.
Open/closed	⚠️	Adding another feature = new folder + a few lines in `Injection` ✅. Adding a verb/endpoint means copy-pasting try/catch (modifies by duplication) ❌.
Liskov	✅	`UserRemoteDataSourceImpl` / `UserRepositoryImpl` fully honour their interfaces.

Principle	Verdict	Why
Interface segregation	✅	Both interfaces have one method.
Dependency inversion	❌	Domain imports data (§1). Everything else injected via interfaces ✅.
DRY	⚠️	No duplication *yet* because there's only one endpoint; the design duplicates as soon as a second appears (try/catch).
KISS	✅	Easy to read.
YAGNI	⚠️	Unused: `DioClient`, `toJson`, `UserResponse.perPage/total`, `isLoadingMore`, `equatable`, `get_it`, empty files. Unused code is worse than missing code: interviewers ask "what is this for?".
Separation of concerns	⚠️	UI/state/data split ✅; domain/data boundary ❌.

9. Live-coding score (strict)

```
Architecture              8 /20   Right layers & folders; domain→data import; swapped/typo'd files
BLoC/Cubit                6 /15   Sealed states + UseCase injection ✓; no loadMore, no Equatable, catch(e), singleton, no isClosed
API Integration           6 /15   Dio + query param ✓; no baseUrl/timeouts/headers/verbs, DioClient unused
Model/Entity Design       4 /10   Separated, but by inheritance; response wrapper in wrong layer; `?? 0` defaults
Repository Pattern        5 /10   Interface/impl split ✓; contract imports data; no exception→failure mapping
Error Handling            1 /10   Two empty files; bare Exception(message); toString() to the user
Dependency Injection      2 /5    Correct order ✓; static late, singleton cubit, get_it unused
Code Quality              2 /5    Typos, TODO, empty files, mixed imports, red test
SOLID/DRY/KISS            2 /5    DIP broken; unused code; will duplicate on 2nd endpoint
Live Coding Speed         3 /5    Compact & fast to type; but would lose time on swapped names & missing pieces

TOTAL                    39 /100

Overall Score: 39/100
```

Scale used: `0–39 Beginner · 40–59 Intermediate · 60–74 Interview Ready · 75–89 Production Ready · 90+ Senior Level`.

Classification: Beginner (upper edge) / Intermediate structure. One edit (move `UserResponse` into `domain/` as `PagedUsers`) is worth about +6. The full fix list in §11 lifts it to the high 80s on this same rubric (the reference implementation in the guide is built to that bar, compiles with 0 analyzer issues, and has 74 passing tests).

10. The 20 questions an interviewer will ask about *your* implementation

🚩 = exposes a weakness in your **current** code. The ideal answers assume the fixed code from the guide.

Q1 🚩. Why did you put the repository interface inside Domain?

- *Say (25 s):* "Because the domain defines what it needs and the data layer adapts to it. If the interface were in data, the domain would import data and the dependency would point the wrong way. Domain stays pure Dart: no Flutter, no Dio, no JSON. That also lets me swap REST for a cache or a fake without touching use cases or UI."
- *Why asked:* Tests whether you understand Dependency Inversion, not just folder names.
- *Real world:* Moving from REST to GraphQL: only `UserRepositoryImpl` + a new data source change.

Q2 🚩. Your `UserResponse` is in `data/model` but the use case returns it. Is that okay?

- *Say:* "No, that's a layering bug. It's a business concept ('a page of users'), so it belongs in domain as `PagedUsers`. The data layer has `UsersResponseModel` which parses JSON and maps to it with `toEntity()`."
- *Why:* They will literally open the file you imported. This is the #1 trap in your current code.
- *Real world:* A backend renames `total_pages` → `pages`: only the model changes; the use case and Cubit are untouched.

Q3. Why both a Model and an Entity? They have the same fields.

- *Say:* "Different reasons to change. The model mirrors the API (snake_case, nullable, extra fields). The entity mirrors the app's needs. Today the fields match; when the backend changes shape, only the model and mapper change."
- *Why:* Detects cargo-cult architecture vs. understanding.
- *Real world:* API v2 returns `name` instead of `first_name` / `last_name`; the entity keeps both, mapper splits.

Q4. Why `toEntity()` instead of `UserModel extends User`?

- *Say:* "Inheritance couples the data shape to the domain shape and invites JSON code onto the entity. A mapper is eight lines and keeps the boundary explicit. If I were truly out of time I'd extend and say so, but I'd keep the entity in domain."
- *Why:* Trade-off judgement.

- *Real world:* Adding a computed `isPremium` to the entity without touching serialization.

Q5. This use case is one line. Why does it exist?

- *Say:* "It names the business action and gives one seam for rules and orchestration. Today it's thin; the first time we need validation, caching policy, or combining two repositories, there's a home for it, and the Cubit doesn't change. For pure CRUD I'd accept calling the repository directly."
- *Why:* Checks you're not dogmatic.
- *Real world:* `PlaceOrder` validates stock then calls two repositories.

Q6. Why Cubit instead of Bloc?

- *Say:* "This screen is request/response: load, load more, retry. Methods are enough. I'd move to Bloc if I needed event transformers like `restartable()` for search or an event log."
- *Why:* Knowing when to switch matters more than the choice.
- *Real world:* A search+filter+pagination screen that races: `restartable()` cancels stale searches.

Q7. Why `sealed` states and `Equatable`?

- *Say:* "Sealed lets the compiler force me to handle every state in a `switch`: add a state and the UI won't compile until handled. Equatable gives value equality, so identical emissions don't rebuild, and tests can compare states directly."
- *Why:* Checks Dart 3 fluency and testability thinking.
- *Real world:* Adding `UserEmpty`: every `switch` in the app flags it.

Q8 🚩. How do you stop duplicate "load more" calls while scrolling?

- *Say:* "`loadMore()` returns early if the state isn't `UserLoaded`, or `isLoadingMore` is true, or `hasReachedMax`. The scroll can fire a hundred times; the Cubit lets one through. With Bloc I'd use the `droppable()` transformer."
- *Why:* Classic bug: duplicate pages / duplicate rows.
- *Real world:* Page 3 appended twice because two scroll events fired before `isLoadingMore` flipped.

Q9. Difference between initial loading and pagination loading?

- *Say:* "Initial loading has no data yet, so a full-screen spinner and a separate `UserLoading` state. Pagination loading already has data on screen, so I keep the list and set `isLoadingMore` to show a footer spinner. Replacing the list with a spinner on page 2 would make the list jump."
- *Why:* UX + state-modelling.
- *Real world:* Instagram feed: bottom spinner, not blank screen.

Q10 🚩. What happens if page 2 fails?

- *Say:* "The list stays. I reset `isLoadingMore`, set `loadMoreFailed`, and the footer shows a Retry. I don't replace the loaded screen with a full-screen error."
- *Why:* Probes whether error states were thought through beyond the happy path.
- *Real world:* Elevator on bad signal: user already read 20 items; don't wipe them.

Q11 🚩. Walk me through error handling end to end.

- *Say:* "Dio throws `DioException`. `DioClient` converts that once into an `AppException` (`ServerException` with status code, or `NetworkException` for timeout/offline). The repository's `guard()` converts exceptions to `Failure` (Unauthorized, Validation, Server, Network, Parsing, Unknown). The Cubit catches only `Failure` and emits an error state with a user-safe message. Nothing above the repository sees Dio."
- *Why:* Core competence; your current files are empty.
- *Real world:* 401 → `UnauthorizedFailure` → global listener sends user to login.

Q12 🚩. Why `on Failure catch` and not `catch (e)`?

- *Say:* "`catch (e)` also swallows programmer bugs (`TypeError`, `StateError`) and shows them to users as if they were network problems. The repository guarantees only `Failure` escapes, so the Cubit catches exactly that and real bugs still surface in crash reporting."
- *Why:* Error-handling maturity.
- *Real world:* A null-check bug hidden as "Something went wrong" for weeks.

Q13. Why wrap `DioException` instead of letting it bubble up?

- *Say:* "So no layer above data depends on Dio. If I replace Dio with `http`, only `DioClient` changes. Also `DioException` isn't user-friendly: I need status code, server message, and a type I can switch on."
- *Why:* Encapsulation of third-party packages.
- *Real world:* Swapping packages after a security advisory.

Q14 🚩. Your `fromJson` uses `?? 0` and `?? ''`. Is that safe?

- *Say:* "No: it hides contract breaks by inventing valid-looking data. I cast strictly; a wrong shape throws `TypeError`, `guard()` maps it to `ParsingFailure`, and it shows up in monitoring. I use `?` / defaults only for fields that are genuinely optional in the API contract."

- *Why:* Data-integrity thinking.
- *Real world:* A user with id 0 and blank email silently written to a local DB.

Q15 🚩. Why is the Cubit registered as a factory, not a singleton?

- *Say:* "`BlocProvider(create:)` closes the Cubit when the widget is disposed. A singleton would be closed after the first visit and the second visit would emit into a closed Cubit. Stateless things (repository, use case, Dio) are singletons; Cubits are per-screen factories."
- *Why:* Lifecycle understanding; this is a real bug in your current DI.
- *Real world:* "Works the first time, crashes when I come back to the screen."

Q16. GetIt or manual DI? Why?

- *Say:* "For one feature, a composition root in `main.dart` is simplest: one place knows the concrete classes. GetIt pays off with many features, lazy singletons, and test overrides. The wiring order is identical: Dio → client → data source → repository → use case → Cubit."
- *Why:* Pragmatism vs tool-worship.
- *Real world:* Team of 10 with 15 features → GetIt/injectable.

Q17 🚩. Where do base URL, timeouts, headers, and logging live?

- *Say:* "On the Dio instance (`BaseOptions` + interceptors), created once in the composition root. Endpoints are relative paths in `ApiConstants`. Auth header and token refresh are an interceptor, so no repository or Cubit touches tokens."
- *Why:* Config centralisation.
- *Real world:* Switching staging ↔ production by changing one constant/flavour.

Q18 🚩. How would you add POST, PUT, PATCH and DELETE?

- *Say:* "`DioClient` already exposes all five verbs with one shared `_send` that maps errors. A new endpoint is: request model (toJson), response model (fromJson/toEntity), a data-source method, a repository method, a use case, and a Cubit method or its own Cubit. Same six steps every time."
- *Why:* Is the architecture a pattern or one-off?
- *Real world:* Create/Edit/Delete user screens.

Q19 🚩. How would you test this?

- *Say:* "Cubit with `bloc_test` and a mocked use case: success, failure, pagination, last page. Repository with a mocked data source: exceptions map to the right failures, and model maps to entity. `DioClient` with a mocked Dio: status codes map to exceptions. One widget test with a fake repository."
- *Why:* Testing is why the architecture exists. Your repo currently has only the failing template test.
- *Real world:* CI blocks merges on red tests.

Q20. How would you add auth tokens and refresh without touching repositories?

- *Say:* "A Dio interceptor: `onRequest` adds the Bearer token from secure storage; `onError` on 401 refreshes once using a separate Dio (so the refresh call can't recurse), stores the new token and retries the original request. Parallel 401s share one refresh future. If refresh fails: clear storage, emit `UnauthorizedFailure`, UI navigates to login."
- *Why:* Senior-level cross-cutting concern.
- *Real world:* Every banking/health app.

---

11. What to change: exact, ordered list (≈25 min)

#	Change	Files	Fixes
1	Move `UserResponse` → `domain/entity/paged_users.dart` as `PagedUsers {users, page, totalPages, hasNext}`; fix imports in `user_repository.dart` and `get_users.dart`; **delete** `data/model/user_response.dart`	domain/, *data/model/*	§1 DIP violation
2	Models parse, entities are business: add `toEntity()` to `UserModel` and `UsersResponseModel`; remove `extends`; strict `as` casts instead of `?? 0`	data/model/*	§4
3	Error layer: implement `exceptions.dart` + `failures.dart` + `guard()`; make `DioClient` real (all 5 verbs + `_toException`); repository wraps in `guard()`	core/errors/*, core/network/dio_client.dart, repository impl	§3 error handling
4	Configure Dio (`baseUrl`, timeouts, `Accept` header); data source uses `DioClient` and relative path `'/users'`	injection, data source	§3
5	Cubit: `Equatable` states + `copyWith`; `fetchUsers()` / `loadMore()` with guards; `on Failure catch`; `isClosed`; DI `registerFactory`; UI: `LoadMoreFooter`, `ErrorView` with Retry	cubit/, *page, widgets/*	§2, §6
6	Rename: `erros→errors`, `failuers→failures`, `data_source→data_source` (and swap the two files' contents), `repositories→repository`, `widget→widgets`, `user_title→user_tile`	everywhere	§7

#	Change	Files	Fixes
7	Replace the counter test with real tests (cubit, repository, DioClient, one widget test)	test/*	§9 Code quality
8	Use GetIt or remove it from `pubspec.yaml` ; same for any unused import/dependency	pubspec, injection	YAGNI

Self-check commands for interview morning (each should print nothing):

```
# Domain must not import data, presentation, Flutter or Dio
grep -rnE "data/|presentation/|package:flutter|package:dio" lib/feature/*/domain

# Presentation must not import Dio or data models
grep -rnE "package:dio|/data/" lib/feature/*/presentation

# No empty files
find lib -name '*.dart' -size 0
```

12. Final block

```
CURRENT IMPLEMENTATION SCORE: 39/100

TOP 5 CHANGES REQUIRED:
1. Domain must not import Data: move UserResponse → domain/entity/paged_users.dart (PagedUsers),
   fix user_repository.dart:1 and get_users.dart:1.
2. Build the error layer: exceptions.dart + failures.dart + guard(); finish DioClient (5 verbs,
   DioException→AppException); Cubit catches `on Failure`, shows a user-safe message.
3. Finish pagination: Equatable states + copyWith, fetchUsers()/loadMore() with duplicate/last-page
   guards, footer loader + Retry, `if (isClosed) return;`.
4. Model→Entity with toEntity() (drop `extends`, drop `?? 0` defaults) + configure Dio
   (baseUrl, timeouts, headers) + Cubit as GetIt factory (or manual DI in main.dart).
5. Fix names/dead code (erros, failuers, data-source swap, user_title, DioClient/toJson unused),
   and replace the red counter test with real cubit/repository/DioClient tests.

20-MINUTE TARGET ARCHITECTURE:
   UI(page+widgets) → Cubit(+Equatable states) → UseCase → Repository(interface, domain)
   → RepositoryImpl(guard: Exception→Failure) → RemoteDataSource(Model parse)
   → DioClient(5 verbs, DioException→AppException) → Dio(BaseOptions) → API
   Dependencies point inward: data → domain ← presentation.

MEMORIZATION FORMULA:
   E-M-R-D-I-U-C-S-U  →  Entity, Model, Repository(interface), Data source, Impl,
   Use case, Cubit, States, UI  (+ DI, run, then pagination/search)
   Response flow: API → JSON → Model → toEntity() → Repository → UseCase → Cubit → State → UI
   Error flow:    DioException → AppException → Failure → Cubit → Error state → Retry
```

➡️ Continue with **Part 2** (your upgrade path) and **Part 3** (the full guide).

Part 2 · Roadmap: from 39 to 100

> **How this chapter was verified.** Steps 1–8 were first applied to a copy of your project, running `flutter analyze` after **every** step. They were then applied to your real project, where I **re-audited the result instead of trusting my own projection**. That re-audit measured **86/100** and found concrete gaps, which Step 9 fixes. Final state of your real project: `flutter analyze` (strict mode) → 0 issues, `flutter test` → **74 passed**, **≈98% line coverage**, `flutter build web` compiles, and the GET/login flows were re-run against the live API. Your original code is backed up in `demoapi_backup_before_100/`, and the state before the hardening round in `demoapi_backup_before_hardening/` (both next to the project folder).
>
> **Honesty note on "100".** Scores are my estimates under the strict rubric from the audit; a real interviewer will weigh things differently. The code base now earns every point except *Live-Coding Speed*: **98/100**. Those last 2 points are about **you**, not the repository: they come from drilling the LITE tier until you can build it in 20 minutes (Step 10).

1. The score trajectory

Running total after each step (cumulative). Steps 1–7 are estimates; **Step 8's 86 was measured** by a fresh re-audit, and it is *lower* than Step 7 on purpose: adding the auth feature as first written introduced dead code and untested code, which Step 9 then fixes.

Step	What you do	≈ time	Score
Now	Your code today	n/a	**39**
1	Fix layering + names (`PagedUsers` into domain, swap the two files, fix typos)	15 min	**54**
2	Model → Entity with `toEntity()`, strict parsing	10 min	**59**
3	Configure Dio, finish `DioClient`, implement `exceptions.dart`	15 min	**66**
4	`failures.dart` + `guard()`, Cubit catches `on Failure`	10 min	**75**
5	Equatable states, `fetchUsers` / `loadMore`, widgets, Retry	25 min	**83**
6	GetIt, Cubit as factory, `main.dart`	10 min	**86**
7	Real tests, delete the red counter test, cleanup	25 min	**87**
8	Prove extensibility: Login (POST), secure token storage, auth interceptor	30 min	**86** (↓ measured)
9	**Hardening round** from the strict re-audit: mount login, test the interceptor, keep failure causes, cancellation, order-free DI, stale-response guard, strict analyzer, README	60 min	**98**
10	Speed drills: LITE tier in 20 minutes, from memory	practice	**100**

Per-category detail:

Category	Max	Now	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10
Architecture	20	8	**17**	17	17	17	17	18	18	18	**20**	20
BLoC/Cubit	15	6	6	6	6	6	**14**	14	14	14	**15**	15
API Integration	15	6	6	6	**12**	12	12	12	12	12	**15**	15
Model/Entity	10	4	4	**9**	9	9	9	9	9	9	**10**	10
Repository	10	5	**8**	8	8	**10**	10	10	10	10	10	10
Error Handling	10	1	1	1	1	**8**	8	8	8	8	**10**	10
Dependency Injection	5	2	2	2	2	2	2	**4**	4	4	**5**	5
Code Quality	5	2	**3**	3	3	3	3	3	**4**	3	**5**	5
SOLID/DRY/KISS	5	2	**4**	4	**5**	5	5	5	5	5	5	5
Live-Coding Speed	5	3	3	3	3	3	3	3	3	3	3	**5**
TOTAL	**100**	**39**	**54**	**59**	**66**	**75**	**83**	**86**	**87**	**86**	**98**	**100**

2. File-by-file migration map

Your file today	Problem	Becomes
`core/erros/exceptions.dart` (empty)	typo + empty	`core/errors/exceptions.dart` (Step 3)
`core/erros/failuers.dart` (empty)	typo + empty	`core/errors/failures.dart` (Step 4)
`core/network/dio_client.dart`	GET-only, unused	5 verbs + error mapping, **used** by data source (Step 3)
`core/constants/api_constants.dart`	base URL concatenated by hand	`baseUrl = '…/api'` , relative paths, timeout (Step 3)
`core/di/injection.dart`	`static late` , singleton Cubit, `Dio()`	GetIt, factory Cubit, configured Dio (Step 6)
`feature/user/data/data-source/user_remote_data_source_impl.dart`	contains the **interface**	`data_source/user_remote_data_source.dart` (Step 1)
`feature/user/data/data-source/user_remote_source_source.dart`	contains the **impl**	`data_source/user_remote_data_source_impl.dart` (Step 1, 3)
`feature/user/data/model/user_model.dart`	`extends User` , `?? 0`	standalone model + `toEntity()` (Step 2)
`feature/user/data/model/user_response.dart`	domain concept in data layer	**deleted** → `domain/entity/paged_users.dart` (Step 1)
`feature/user/data/model/user_response_model.dart`	`extends UserResponse`	`users_response_model.dart` + `toEntity()` (Step 2)
`feature/user/data/repositories/user_repository_impl.dart`	pass-through	`data/repository/…` with `guard()` + `toEntity()` (Steps 1, 2, 4)
`feature/user/domain/repository/user_repository.dart`	imports **data**	returns `PagedUsers` (Step 1)
`feature/user/domain/usecase/get_users.dart`	imports **data**	returns `PagedUsers` (Step 1)
`feature/user/presentation/cubit/user_state.dart`	no Equatable, dead fields	Equatable + `copyWith` + `hasReachedMax` (Step 5)
`feature/user/presentation/cubit/user_cubit.dart`	`catch (e)` , no `loadMore`	`fetchUsers` + `loadMore` + `on Failure` (Steps 4, 5)
`feature/user/presentation/pages/user_page.dart`	TODO, no Retry, no pagination	stateless page + widgets (Step 5)
`feature/user/presentation/widget/user_title.dart` (empty)	typo + empty	`widgets/user_tile.dart` (Steps 1, 5)
`test/widget_test.dart`	red counter test	real tests (Step 7)

STEP 1: Fix the layering and the names (39 → 54)

Goal: make the dependency rule true: **domain imports nothing from data.** Fix the names an interviewer sees on screen.

What's wrong (from the audit): `domain/repository/user_repository.dart:1` and `domain/usecase/get_users.dart:1` import `data/model/user_response.dart`; the two data-source files have each other's contents; folder/file typos.

Your code today

```dart
import '../../data/model/user_response.dart';

abstract class UserRepository {
  Future<UserResponse> getUsers({required int page});
}
```

```dart
import '../../domain/entity/user.dart';

class UserResponse {
  final int page;
  final int perPage;
  final int total;
  final int totalPages;
  final List<User> users;

  const UserResponse({
    required this.page,
    required this.perPage,
    required this.total,
    required this.totalPages,
    required this.users,
  });
}
```

(*`UserResponse` is "a page of users": a business concept sitting in `data/`.*)

Do this

1. Rename folders/files (run from the project root; these exact commands were tested):

```
cd lib
mv core/erros core/errors
mv core/errors/failuers.dart core/errors/failures.dart
mv feature/user/data/data-source feature/user/data/data_source
# the two data-source files contain each other's class: swap them
mv feature/user/data/data_source/user_remote_data_source_impl.dart feature/user/data/data_source/_tmp_interface.dart
mv feature/user/data/data_source/user_remote_source_source.dart   feature/user/data/data_source/user_remote_data_source_impl.dart
mv feature/user/data/data_source/_tmp_interface.dart              feature/user/data/data_source/user_remote_data_source.dart
mv feature/user/data/repositories feature/user/data/repository
mv feature/user/presentation/widget feature/user/presentation/widgets
mv feature/user/presentation/widgets/user_title.dart feature/user/presentation/widgets/user_tile.dart
rm feature/user/data/model/user_response.dart
```

2. Create the entity **in domain**: `domain/entity/paged_users.dart`

```dart
import 'package:equatable/equatable.dart';

import 'user.dart';

class PagedUsers extends Equatable {
  const PagedUsers({
    required this.users,
    required this.page,
    required this.totalPages,
  });

  final List<User> users;
  final int page;
  final int totalPages;

  bool get hasNext => page < totalPages;

  @override
  List<Object?> get props => [users, page, totalPages];
}
```

3. Point the domain at it: `domain/repository/user_repository.dart`

```dart
import '../entity/paged_users.dart';

abstract interface class UserRepository {
```

```
  Future<PagedUsers> getUsers({required int page});
}
```

and `domain/usecase/get_users.dart`

```dart
import '../entity/paged_users.dart';
import '../repository/user_repository.dart';

class GetUsers {
  const GetUsers(this._repository);

  final UserRepository _repository;

  Future<PagedUsers> call({required int page}) =>
      _repository.getUsers(page: page);
}
```

4. Make `domain/entity/user.dart` the `Equatable` version from the Guide (§4B).
5. Fix the imports that broke: in `user_remote_data_source_impl.dart` import `user_remote_data_source.dart`; in `user_repository_impl.dart` import `../../domain/entity/paged_users.dart` and `../data_source/user_remote_data_source.dart`, return `Future<PagedUsers>`; in `injection.dart` use `data_source/` and `repository/`.
6. **Temporary bridge** so everything compiles before Step 2: make the response model extend the *domain* entity (data → domain is the legal direction; it is replaced in Step 2):

```dart
import '../../domain/entity/paged_users.dart';
import 'user_model.dart';

class UserResponseModel extends PagedUsers {
  const UserResponseModel({
    required super.page,
    required super.totalPages,
    required super.users,
  });

  factory UserResponseModel.fromJson(Map<String, dynamic> json) { /* unchanged */ }
}
```

Verify

```
flutter analyze                    # 0 issues
grep -rnE "data/|presentation/|package:flutter|package:dio" lib/feature/*/domain   # prints NOTHING
```

Score → 54. *(Architecture 8→17, Repository 5→8, SOLID 2→4, Quality 2→3.)* **Say:** "*The domain defines the contract and the data layer adapts to it, so imports point inward.*"

STEP 2: Model → Entity with `toEntity()` (54 → 59)

Goal: models parse JSON, entities are business. Stop inventing data with `?? 0`.

Your code today

```dart
import '../../domain/entity/user.dart';

class UserModel extends User {
  const UserModel({
    required super.id,
    required super.email,
    required super.firstName,
    required super.lastName,
    required super.avatar,
  });

  factory UserModel.fromJson(Map<String, dynamic> json) {
    return UserModel(
      id: json['id'] ?? 0,
      email: json['email'] ?? '',
      firstName: json['first_name'] ?? '',
      lastName: json['last_name'] ?? '',
      avatar: json['avatar'] ?? '',
    );
  }

  Map<String, dynamic> toJson() {
    return {
      'id': id,
      'email': email,
      'first_name': firstName,
      'last_name': lastName,
      'avatar': avatar,
    };
  }
}
```

Do this: replace both models; delete `user_response_model.dart`; the data-source interface now returns `UsersResponseModel`; the repository calls `.toEntity()`.

`data/model/user_model.dart`

```dart
import '../../domain/entity/user.dart';

class UserModel {
  const UserModel({
    required this.id,
    required this.email,
    required this.firstName,
    required this.lastName,
    required this.avatar,
  });

  final int id;
  final String email;
  final String firstName;
  final String lastName;
  final String avatar;

  factory UserModel.fromJson(Map<String, dynamic> json) => UserModel(
    id: json['id'] as int,
    email: json['email'] as String,
    firstName: json['first_name'] as String,
    lastName: json['last_name'] as String,
    avatar: json['avatar'] as String,
  );

  Map<String, dynamic> toJson() => {
    'id': id,
    'email': email,
    'first_name': firstName,
    'last_name': lastName,
    'avatar': avatar,
  };

  User toEntity() => User(
    id: id,
    email: email,
    firstName: firstName,
    lastName: lastName,
    avatar: avatar,
```

```
    );
  }
```

`data/model/users_response_model.dart`

```dart
import '../../domain/entity/paged_users.dart';
import 'user_model.dart';

class UsersResponseModel {
  const UsersResponseModel({
    required this.page,
    required this.perPage,
    required this.total,
    required this.totalPages,
    required this.data,
  });

  final int page;
  final int perPage;
  final int total;
  final int totalPages;
  final List<UserModel> data;

  factory UsersResponseModel.fromJson(Map<String, dynamic> json) =>
      UsersResponseModel(
        page: json['page'] as int,
        perPage: json['per_page'] as int,
        total: json['total'] as int,
        totalPages: json['total_pages'] as int,
        data: (json['data'] as List)
            .map((e) => UserModel.fromJson(e as Map<String, dynamic>))
            .toList(),
      );

  PagedUsers toEntity() => PagedUsers(
    users: data.map((m) => m.toEntity()).toList(),
    page: page,
    totalPages: totalPages,
  );
}
```

`data/data_source/user_remote_data_source.dart`

```dart
import '../model/users_response_model.dart';

abstract interface class UserRemoteDataSource {
  Future<UsersResponseModel> getUsers({required int page});
}
```

In `user_repository_impl.dart`:

```dart
final model = await remoteDataSource.getUsers(page: page);
return model.toEntity();
```

Why strict casts: `json['id'] as int` fails loudly on a bad contract (Step 4 turns that `TypeError` into a `ParsingFailure`), while `?? 0` would render a blank user with id 0.

Verify: `flutter analyze`. **Score → 59** *(Model/Entity 4→9).* **Say:** "*Model mirrors the API, entity mirrors the business, `toEntity()` is the anti-corruption boundary.*"

STEP 3: Configure Dio, finish `DioClient`, implement exceptions (59 → 66)

Goal: one configured Dio; every future endpoint is 2–3 lines; no `DioException` above the data layer.

Your code today

```dart
import 'package:dio/dio.dart';

import '../../../../core/constants/api_constants.dart';
import '../model/user_response_model.dart';
import 'user_remote_data_source_impl.dart';

class UserRemoteDataSourceImpl implements UserRemoteDataSource {
  final Dio dio;

  UserRemoteDataSourceImpl(this.dio);

  @override
  Future<UserResponseModel> getUsers({required int page}) async {
    try {
      final response = await dio.get(
        '${ApiConstants.baseUrl}${ApiConstants.users}',
        queryParameters: {'page': page},
      );

      return UserResponseModel.fromJson(response.data as Map<String, dynamic>);
    } on DioException catch (e) {
      throw Exception(e.message ?? 'Something went wrong');
    }
  }
}
```

```dart
import 'package:dio/dio.dart';

class DioClient {
  final Dio dio;

  DioClient(this.dio);

  Future<Response<dynamic>> get(
    String path, {
    Map<String, dynamic>? queryParameters,
  }) async {
    return dio.get(path, queryParameters: queryParameters);
  }
}
```

(`Dio()` unconfigured; URL concatenated by hand; the error becomes a bare `Exception`; `DioClient` is GET-only and unused.)

Do this

1. Replace `core/constants/api_constants.dart`, add `core/errors/exceptions.dart`, replace `core/network/dio_client.dart` with the versions in **Guide §4B** (they are the same files shown there).
2. Replace the data-source implementation: it now takes a `DioClient` and uses a **relative** path; no `try/catch` at all:

```dart
import '../../../../core/constants/api_constants.dart';
import '../../../../core/network/dio_client.dart';
import '../model/users_response_model.dart';
import 'user_remote_data_source.dart';

class UserRemoteDataSourceImpl implements UserRemoteDataSource {
  const UserRemoteDataSourceImpl(this._client);

  final DioClient _client;

  @override
  Future<UsersResponseModel> getUsers({required int page}) async {
    final json = await _client.get(ApiConstants.users, query: {'page': page});
    return UsersResponseModel.fromJson(json as Map<String, dynamic>);
  }
}
```

3. Configure Dio and use the client in `injection.dart` (GetIt comes in Step 6):

```dart
dio = Dio(
  BaseOptions(
    baseUrl: ApiConstants.baseUrl,
    connectTimeout: ApiConstants.timeout,
    receiveTimeout: ApiConstants.timeout,
    headers: {'Accept': 'application/json'},
  ),
);
```

```
dioClient = DioClient(dio);
userRemoteDataSource = UserRemoteDataSourceImpl(dioClient);
```

Verify: `flutter analyze`. **Score → 66** *(API 6→12, SOLID 4→5)*. Until Step 4 the Cubit still prints a raw exception: that is expected. **Say:** "*`DioClient` is the only code that knows Dio's error types; swap `dio` for `http` and only this file changes.*"

STEP 4: Failures and `guard()` (66 → 75)

Goal: typed errors end to end: `DioException → AppException → Failure → Cubit → Error state`.

Do this

1. Add `core/errors/failures.dart` (Guide §4B): sealed `Failure` family + `guard()`.

```dart
import 'package:equatable/equatable.dart';

import 'exceptions.dart';

/// What the domain / Cubit understands. Always carries a user-safe message.
sealed class Failure extends Equatable implements Exception {
  const Failure(this.message);

  final String message;

  @override
  List<Object?> get props => [message];
}

class ServerFailure extends Failure {
  const ServerFailure(super.message, {this.statusCode});

  final int? statusCode;

  @override
  List<Object?> get props => [message, statusCode];
}

class UnauthorizedFailure extends Failure {
  const UnauthorizedFailure(super.message);
}

/// 422 / client-side validation: show the message next to the form.
class ValidationFailure extends Failure {
  const ValidationFailure(super.message);
}

class NetworkFailure extends Failure {
  const NetworkFailure(super.message);
}

class ParsingFailure extends Failure {
  const ParsingFailure() : super('Unexpected data received from server.');
}

class UnknownFailure extends Failure {
  const UnknownFailure() : super('Something went wrong. Please try again.');
}

/// Repositories wrap every data-source call in this: Exception -> Failure, once.
Future<T> guard<T>(Future<T> Function() action) async {
  try {
    return await action();
  } on ServerException catch (e) {
    throw switch (e.statusCode) {
      401 || 403 => UnauthorizedFailure(e.message),
      422 => ValidationFailure(e.message),
      _ => ServerFailure(e.message, statusCode: e.statusCode),
    };
  } on NetworkException catch (e) {
    throw NetworkFailure(e.message);
  } on ParsingException {
    throw const ParsingFailure(); // body is not valid JSON
  } on TypeError {
    throw const ParsingFailure(); // wrong JSON shape: null / wrong type
  } on FormatException {
    throw const ParsingFailure();
  } catch (_) {
    throw const UnknownFailure();
  }
}
```

2. The repository becomes three lines, `guard()` does the mapping:

```dart
import '../../../../core/errors/failures.dart';
import '../../domain/entity/paged_users.dart';
import '../../domain/repository/user_repository.dart';
import '../data_source/user_remote_data_source.dart';

class UserRepositoryImpl implements UserRepository {
  const UserRepositoryImpl(this._remote);
```

```
  final UserRemoteDataSource _remote;

  @override
  Future<PagedUsers> getUsers({required int page}) =>
      guard(() async => (await _remote.getUsers(page: page)).toEntity());
}
```

3. Your Cubit's catch changes from the catch-all to the typed one:

```
// before
} catch (e) {
  emit(UserError(e.toString()));   // "Exception: Something went wrong"
// after
} on Failure catch (failure) {
  emit(UserError(failure.message)); // "No internet connection."
```

Verify: `flutter analyze`; run the app in airplane mode: you should see **"No internet connection."**, not an exception string. **Score → 75** *(Error handling 1→8, Repository 8→10).* **Say:** "*Exceptions are for the data layer; failures are what the rest of the app understands.* `guard()` *converts once, so repositories stay three lines.*"

STEP 5: Cubit, states, pagination and UI (75 → 83)

Goal: finish what the half-built state promised: append pages, loader in the footer, retry, no duplicate calls.

Your code today

```dart
import '../../domain/entity/user.dart';

sealed class UserState {
  const UserState();
}

class UserInitial extends UserState {
  const UserInitial();
}

class UserLoading extends UserState {
  const UserLoading();
}

class UserLoaded extends UserState {
  final List<User> users;
  final int totalPages;
  final int currentPages;
  final bool isLoadingMore;

  const UserLoaded({
    required this.currentPages,
    required this.totalPages,
    required this.users,
    this.isLoadingMore = false,
  });
}

class UserError extends UserState {
  final String message;
  const UserError(this.message);
}
```

```dart
import 'package:demoapi/feature/user/domain/usecase/get_users.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import 'user_state.dart';

class UserCubit extends Cubit<UserState> {
  final GetUsers getUsers;

  UserCubit(this.getUsers) : super(const UserInitial());

  Future<void> fetchUsers({int page = 1}) async {
    emit(const UserLoading());
    try {
      final response = await getUsers(page: page);
      emit(
        UserLoaded(
          users: response.users,
          currentPages: response.page,
          totalPages: response.totalPages,
        ),
      );
    } catch (e) {
      emit(UserError(e.toString()));
    }
  }
}
```

(`isLoadingMore` is never set; `fetchUsers(page)` replaces the list; `currentPages`; no equality.)

Do this: replace these files with the versions in **Guide §4B** (all verified; the Cubit is shown below), and create the three widgets:

- `cubit/user_state.dart`: `Equatable`, `copyWith`, `hasReachedMax`, `loadMoreFailed`, `currentPage` (singular).
- `cubit/user_cubit.dart`: `fetchUsers()`, `loadMore()`, `on Failure`, `isClosed`:

```dart
import 'package:flutter_bloc/flutter_bloc.dart';

import '../../../../core/errors/failures.dart';
import '../../domain/usecase/get_users.dart';
import 'user_state.dart';

class UserCubit extends Cubit<UserState> {
  UserCubit(this._getUsers) : super(const UserInitial());

  final GetUsers _getUsers;
```

```
  /// First page (also used by the Retry button).
  Future<void> fetchUsers() async {
    emit(const UserLoading());
    try {
      final result = await _getUsers(page: 1);
      if (isClosed) return;
      emit(
        UserLoaded(
          users: result.users,
          currentPage: result.page,
          totalPages: result.totalPages,
        ),
      );
    } on Failure catch (failure) {
      if (isClosed) return;
      emit(UserError(failure.message));
    }
  }

  /// Next page, appended to the current list. Safe to call many times.
  Future<void> loadMore() async {
    final current = state;
    if (current is! UserLoaded ||
        current.isLoadingMore ||
        current.hasReachedMax) {
      return;
    }

    emit(current.copyWith(isLoadingMore: true, loadMoreFailed: false));
    try {
      final result = await _getUsers(page: current.currentPage + 1);
      if (isClosed) return;
      emit(
        current.copyWith(
          users: [...current.users, ...result.users],
          currentPage: result.page,
          totalPages: result.totalPages,
          isLoadingMore: false,
        ),
      );
    } on Failure {
      if (isClosed) return;
      emit(current.copyWith(isLoadingMore: false, loadMoreFailed: true));
    }
  }
}
```

- `widgets/user_tile.dart`, `widgets/error_view.dart` (Retry), `widgets/load_more_footer.dart`
- `pages/user_page.dart`: stateless, `switch` over the sealed state. **Remove the `initState` + TODO**: the fetch moves to `BlocProvider` (`..fetchUsers()`), so in `main.dart`: `create: (_) => Injection.userCubit..fetchUsers()` (final form in Step 6).

Verify: `flutter analyze`; run: page 1 appears, scrolling to the end shows a spinner, then page 2 is appended; with no internet a Retry button appears. **Score → 83** *(Cubit 6→14).* **Say:** "*`loadMore` ignores calls when not loaded, already loading, or on the last page; if page 2 fails the list stays and the footer offers Retry.*"

STEP 6: Dependency injection (83 → 86)

Goal: no `static late` ; a Cubit **factory** (a singleton Cubit is closed by `BlocProvider` after the first visit).

Your code today

```dart
import 'package:dio/dio.dart';

import '../../feature/user/data/data-source/user_remote_data_source_impl.dart';
import '../../feature/user/data/data-source/user_remote_source_source.dart';
import '../../feature/user/data/repositories/user_repository_impl.dart';
import '../../feature/user/domain/repository/user_repository.dart';
import '../../feature/user/domain/usecase/get_users.dart';
import '../../feature/user/presentation/cubit/user_cubit.dart';

class Injection {
  static late Dio dio;

  static late UserRemoteDataSource userRemoteDataSource;
  static late UserRepository userRepository;
  static late GetUsers getUsers;
  static late UserCubit userCubit;

  static void init() {
    dio = Dio();

    userRemoteDataSource = UserRemoteDataSourceImpl(dio);

    userRepository = UserRepositoryImpl(userRemoteDataSource);

    getUsers = GetUsers(userRepository);

    userCubit = UserCubit(getUsers);
  }
}
```

```dart
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import 'core/di/injection.dart';
import 'feature/user/presentation/pages/user_page.dart';

void main() {
  Injection.init();

  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: BlocProvider(
        create: (_) => Injection.userCubit,
        child: const UserPage(),
      ),
    );
  }
}
```

Do this: add `core/network/safe_log_interceptor.dart` (Guide §11), then replace both files:

```dart
import 'package:dio/dio.dart';
import 'package:flutter/foundation.dart';
import 'package:get_it/get_it.dart';

import '../../feature/user/data/data_source/user_remote_data_source.dart';
import '../../feature/user/data/data_source/user_remote_data_source_impl.dart';
import '../../feature/user/data/repository/user_repository_impl.dart';
import '../../feature/user/domain/repository/user_repository.dart';
import '../../feature/user/domain/usecase/get_users.dart';
import '../../feature/user/presentation/cubit/user_cubit.dart';
import '../constants/api_constants.dart';
import '../network/dio_client.dart';
import '../network/safe_log_interceptor.dart';

final sl = GetIt.instance;

void initDependencies() {
  sl
```

```
    // core
    ..registerLazySingleton<Dio>(_createDio)
    ..registerLazySingleton(() => DioClient(sl()))
    // user feature (bottom -> top)
    ..registerLazySingleton<UserRemoteDataSource>(
      () => UserRemoteDataSourceImpl(sl()),
    )
    ..registerLazySingleton<UserRepository>(() => UserRepositoryImpl(sl()))
    ..registerLazySingleton(() => GetUsers(sl()))
    ..registerFactory(() => UserCubit(sl())); // factory: new Cubit per screen
}

Dio _createDio() {
  final dio = Dio(
    BaseOptions(
      baseUrl: ApiConstants.baseUrl,
      connectTimeout: ApiConstants.timeout,
      receiveTimeout: ApiConstants.timeout,
      headers: {'Accept': 'application/json'},
    ),
  );
  if (kDebugMode) {
    dio.interceptors.add(SafeLogInterceptor()); // never in release
  }
  return dio;
}
```

```
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import 'core/di/injection.dart';
import 'feature/user/presentation/cubit/user_cubit.dart';
import 'feature/user/presentation/pages/user_page.dart';

void main() {
  initDependencies();
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) => MaterialApp(
    debugShowCheckedModeBanner: false,
    home: BlocProvider(
      create: (_) => sl<UserCubit>()..fetchUsers(),
      child: const UserPage(),
    ),
  );
}
```

(Prefer no GetIt in the interview? Use the **manual composition root** shown in Guide §4C; same wiring, zero packages.)

Verify: `flutter analyze`; navigate away from the page and back: still works. **Score → 86** *(DI 2→4, Architecture +1).*

STEP 7: Real tests and cleanup (86 → 87)

Goal: `flutter test` green with tests that mean something; zero dead code.

Your test today (the Flutter template; it **fails**: `LateInitializationError: Field 'userCubit' has not been initialized`)

```dart
// This is a basic Flutter widget test.
//
// To perform an interaction with a widget in your test, use the WidgetTester
// utility in the flutter_test package. For example, you can send tap and scroll
// gestures. You can also use WidgetTester to find child widgets in the widget
// tree, read text, and verify that the values of widget properties are correct.

import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';

import 'package:demoapi/main.dart';

void main() {
  testWidgets('Counter increments smoke test', (WidgetTester tester) async {
    // Build our app and trigger a frame.
    await tester.pumpWidget(const MyApp());

    // Verify that our counter starts at 0.
    expect(find.text('0'), findsOneWidget);
    expect(find.text('1'), findsNothing);

    // Tap the '+' icon and trigger a frame.
    await tester.tap(find.byIcon(Icons.add));
    await tester.pump();

    // Verify that our counter has incremented.
    expect(find.text('0'), findsNothing);
    expect(find.text('1'), findsOneWidget);
  });
}
```

Do this

```
flutter pub add --dev bloc_test mocktail
rm test/widget_test.dart
```

Add the five test files from **Guide §17**: `user_cubit_test.dart`, `user_repository_impl_test.dart`, `user_remote_data_source_test.dart`, `dio_client_test.dart`, `user_page_test.dart`.

Cleanup checklist (each command prints nothing when you pass):

```
find lib -name '*.dart' -size 0                                          # no empty files
grep -rnE "data/|presentation/|package:flutter|package:dio" lib/feature/*/domain  # domain is pure
grep -rnE "package:dio|/data/" lib/feature/*/presentation                # UI never sees Dio/data
flutter analyze && flutter test                                          # green
```

Also: remove any unused dependency from `pubspec.yaml`, resolve every `TODO`, pick **one** import style (package imports or relative; be consistent).

Score → 87 *(Quality 3→4).*

STEP 8: Prove it generalises: POST, secure tokens, interceptor (87 → 86)

Goal: show the architecture is a *pattern*, not a one-off: a second feature touches **no** user-feature code.

1. Copy `feature/auth/**` (Guide §9): request/response models, `AuthRemoteDataSource`, `AuthRepositoryImpl`, `Login` use case, `LoginCubit`, `LoginPage`.
2. Add `core/storage/token_storage.dart` (Guide §12) and `flutter pub add flutter_secure_storage`.
3. Add `core/network/auth_interceptor.dart` (Guide §13): Bearer header + refresh-on-401, with the four pitfalls handled.
4. Register it (first version: an `initAuthDependencies()` call in `main()` that also attached the interceptor. **Step 9 replaces this** with an order-free design).
5. Add `test/feature/auth/login_cubit_test.dart` (Guide §17).

Verify: log in with `eve.holt@reqres.in` / `cityslicka` → token saved; missing password → snackbar **"Missing password"** (the server's own message, delivered by `DioClient`). **Score → 86 (↓1, measured).** Adding a feature isn't free. As first written, the login page was **never mounted** (and navigated to a route that doesn't exist), the interceptor and token storage had **no tests**, and the interceptor was attached to Dio imperatively after creation. A strict re-audit counts all of that against you. Step 9 fixes exactly that. **Say:** "*A new feature is new folders plus one DI call; nothing in the user feature changed. Auth lives in an interceptor, so no repository ever touches a token.*"

STEP 9: Hardening round: what a strict re-audit found (86 → 98)

I re-audited the code **as it stood** and verified each suspicion with a command instead of assuming:

#	Finding (evidence)	Fix
1	`LoginPage` is unreachable: `grep -rn LoginPage lib` finds only its own file, and on success it calls `pushReplacementNamed('/users')`, a route that doesn't exist	`main.dart` mounts `/` and `/login` routes; the users page gets a login icon; success shows a snackbar and **pops**; widget test
2	`AuthInterceptor`, `TokenStorage`, `AuthRepositoryImpl`, `LoginPage` have **no tests** (`grep` in `test/` → none)	Tests against a **fake HTTP layer**: Bearer header, 401 → refresh → retry, parallel 401s share one refresh, failed refresh clears storage, no refresh token, retry still 401 (no loop), non-401 passthrough; plus storage / repository / page tests
3	`UserModel.toJson` unused (YAGNI); `DioClient.put/patch/delete` never exercised	`toJson` deleted; all five verbs now tested
4	`guard()`'s `catch (_)` **throws the original error away**: no way to log it	`Failure` carries `cause` + `stackTrace`; `FailureReporting` mixin forwards them to `Bloc.observer.onError`; `AppBlocObserver` is the single sink (Crashlytics/Sentry hook); the user still only sees a safe message
5	No cancellation; a cancelled request would show as a *network error*	`CancelToken` on every verb → `CancelledException` → `CancelledFailure`, ignored by Cubits
6	The interceptor is attached **imperatively** after Dio is created, so it only works if `initAuthDependencies()` runs after `initDependencies()`	`createDio()` is the one place Dio is built (options + `AuthInterceptor` + debug logger); `registerAuthFeature(sl)` registers only the feature's classes; registrations are lazy, so order never matters; DI-graph test
7	`loadMore` can write a **stale** result back if the list was reloaded while it was in flight	`_generation` counter in `UserCubit`: older responses are dropped; two `Completer` tests prove it
8	Static analysis is "default", docs are the Flutter template	`strict-casts`, `strict-inference`, `strict-raw-types` + 6 extra lints, 0 issues; a real `README.md`; tests for entities, failures, safe logger

The final code for the files that changed

`core/errors/failures.dart`: failures keep their technical cause

```dart
import 'package:equatable/equatable.dart';

import 'exceptions.dart';

/// What the domain / Cubit understands. [message] is always user-safe.
sealed class Failure extends Equatable implements Exception {
  const Failure(this.message, {this.cause, this.stackTrace});

  final String message;

  /// Technical reason. NEVER shown to users; forwarded to `Bloc.observer.onError`
  /// (see `reportFailure`) so unexpected problems reach your logger/crash tool.
  final Object? cause;
  final StackTrace? stackTrace;

  @override
  List<Object?> get props => [message];
}

class ServerFailure extends Failure {
  const ServerFailure(super.message, {this.statusCode});

  final int? statusCode;

  @override
  List<Object?> get props => [message, statusCode];
}

class UnauthorizedFailure extends Failure {
  const UnauthorizedFailure(super.message);
}

/// 422 / client-side validation: show the message next to the form.
class ValidationFailure extends Failure {
  const ValidationFailure(super.message);
}

class NetworkFailure extends Failure {
  const NetworkFailure(super.message);
}

/// The request was cancelled on purpose: callers ignore it, no error UI.
class CancelledFailure extends Failure {
  const CancelledFailure() : super('Request cancelled.');
```

```
}

class ParsingFailure extends Failure {
  const ParsingFailure({super.cause, super.stackTrace})
    : super('Unexpected data received from server.');
}

class UnknownFailure extends Failure {
  const UnknownFailure({super.cause, super.stackTrace})
    : super('Something went wrong. Please try again.');
}

/// Repositories wrap every data-source call in this: Exception -> Failure, once.
/// Technical causes are kept on the Failure, never swallowed.
Future<T> guard<T>(Future<T> Function() action) async {
  try {
    return await action();
  } on ServerException catch (e) {
    throw switch (e.statusCode) {
      401 || 403 => UnauthorizedFailure(e.message),
      422 => ValidationFailure(e.message),
      _ => ServerFailure(e.message, statusCode: e.statusCode),
    };
  } on NetworkException catch (e) {
    throw NetworkFailure(e.message);
  } on CancelledException {
    throw const CancelledFailure();
  } on ParsingException catch (e, s) {
    throw ParsingFailure(cause: e, stackTrace: s); // body is not valid JSON
  } on TypeError catch (e, s) {
    throw ParsingFailure(cause: e, stackTrace: s); // wrong shape: null / type
  } on FormatException catch (e, s) {
    throw ParsingFailure(cause: e, stackTrace: s);
  } catch (e, s) {
    throw UnknownFailure(cause: e, stackTrace: s);
  }
}
```

`core/errors/report_failure.dart` : the logging hook

```
import 'package:flutter_bloc/flutter_bloc.dart';

import 'failures.dart';

mixin FailureReporting<S> on BlocBase<S> {
  /// Forwards the hidden technical cause of a [Failure] (parsing/unknown) to
  /// `Bloc.observer.onError`: one place to plug Crashlytics/Sentry.
  /// Expected failures (offline, 401, 422...) have no cause and are not reported.
  void reportFailure(Failure failure) {
    final cause = failure.cause;
    if (cause != null) addError(cause, failure.stackTrace);
  }
}
```

`feature/user/presentation/cubit/user_cubit.dart` : generation counter + reporting + cancel

```
import 'package:flutter_bloc/flutter_bloc.dart';

import '../../../../core/errors/failures.dart';
import '../../../../core/errors/report_failure.dart';
import '../../domain/usecase/get_users.dart';
import 'user_state.dart';

class UserCubit extends Cubit<UserState> with FailureReporting<UserState> {
  UserCubit(this._getUsers) : super(const UserInitial());

  final GetUsers _getUsers;

  /// Bumped by every fresh load. A response that belongs to an older
  /// generation (user hit Retry, or refreshed meanwhile) is dropped.
  int _generation = 0;

  /// First page (also used by the Retry button).
  Future<void> fetchUsers() async {
    final generation = ++_generation;
    emit(const UserLoading());
    try {
      final result = await _getUsers(page: 1);
      if (isClosed || generation != _generation) return;
      emit(
        UserLoaded(
          users: result.users,
          currentPage: result.page,
          totalPages: result.totalPages,
```

```
        ),
      );
    } on CancelledFailure {
      return; // superseded on purpose: no error UI
    } on Failure catch (failure) {
      reportFailure(failure);
      if (isClosed || generation != _generation) return;
      emit(UserError(failure.message));
    }
  }

  /// Next page, appended to the current list. Safe to call many times.
  Future<void> loadMore() async {
    final current = state;
    if (current is! UserLoaded ||
        current.isLoadingMore ||
        current.hasReachedMax) {
      return;
    }

    final generation = _generation;
    emit(current.copyWith(isLoadingMore: true, loadMoreFailed: false));
    try {
      final result = await _getUsers(page: current.currentPage + 1);
      if (isClosed || generation != _generation) return; // list was reloaded
      emit(
        current.copyWith(
          users: [...current.users, ...result.users],
          currentPage: result.page,
          totalPages: result.totalPages,
          isLoadingMore: false,
        ),
      );
    } on CancelledFailure {
      return;
    } on Failure catch (failure) {
      reportFailure(failure);
      if (isClosed || generation != _generation) return;
      emit(current.copyWith(isLoadingMore: false, loadMoreFailed: true));
    }
  }
}
```

`core/di/injection.dart` : one place builds Dio, order-free

```
import 'package:dio/dio.dart';
import 'package:flutter/foundation.dart';
import 'package:flutter_secure_storage/flutter_secure_storage.dart';
import 'package:get_it/get_it.dart';

import '../../feature/auth/auth_injection.dart';
import '../../feature/user/data/data_source/user_remote_data_source.dart';
import '../../feature/user/data/data_source/user_remote_data_source_impl.dart';
import '../../feature/user/data/repository/user_repository_impl.dart';
import '../../feature/user/domain/repository/user_repository.dart';
import '../../feature/user/domain/usecase/get_users.dart';
import '../../feature/user/presentation/cubit/user_cubit.dart';
import '../constants/api_constants.dart';
import '../network/auth_interceptor.dart';
import '../network/dio_client.dart';
import '../network/safe_log_interceptor.dart';
import '../storage/token_storage.dart';

final sl = GetIt.instance;

/// Composition root. Registrations are lazy, so their order never matters.
void initDependencies() {
  sl
    // core
    ..registerLazySingleton(() => const TokenStorage(FlutterSecureStorage()))
    ..registerLazySingleton<Dio>(() => createDio(sl()))
    ..registerLazySingleton(() => DioClient(sl()))
    // user feature (bottom -> top)
    ..registerLazySingleton<UserRemoteDataSource>(
      () => UserRemoteDataSourceImpl(sl()),
    )
    ..registerLazySingleton<UserRepository>(() => UserRepositoryImpl(sl()))
    ..registerLazySingleton(() => GetUsers(sl()))
    ..registerFactory(() => UserCubit(sl())); // factory: new Cubit per screen

  registerAuthFeature(sl);
}

/// The ONE place Dio is built: base options, auth interceptor, debug logging.
@visibleForTesting
```

```
Dio createDio(TokenStorage tokenStorage) {
  final dio = Dio(_baseOptions());
  dio.interceptors.add(
    AuthInterceptor(
      storage: tokenStorage,
      plainDio: Dio(_baseOptions()), // no interceptors: used to refresh + retry
    ),
  );
  if (kDebugMode) {
    dio.interceptors.add(SafeLogInterceptor()); // never in release
  }
  return dio;
}

BaseOptions _baseOptions() => BaseOptions(
  baseUrl: ApiConstants.baseUrl,
  connectTimeout: ApiConstants.timeout,
  receiveTimeout: ApiConstants.timeout,
  headers: {'Accept': 'application/json'},
);
```

`feature/auth/auth_injection.dart`

```
import 'package:get_it/get_it.dart';

import 'data/data_source/auth_remote_data_source.dart';
import 'data/repository/auth_repository_impl.dart';
import 'domain/repository/auth_repository.dart';
import 'domain/usecase/login.dart';
import 'presentation/cubit/login_cubit.dart';

/// Per-feature DI module: registers only this feature's classes. It needs
/// DioClient and TokenStorage from core, which are registered lazily in
/// `initDependencies()`, so call order doesn't matter.
void registerAuthFeature(GetIt sl) {
  sl
    ..registerLazySingleton<AuthRemoteDataSource>(
      () => AuthRemoteDataSourceImpl(sl()),
    )
    ..registerLazySingleton<AuthRepository>(
      () => AuthRepositoryImpl(sl(), sl()),
    )
    ..registerLazySingleton(() => Login(sl()))
    ..registerFactory(() => LoginCubit(sl()));
}
```

`main.dart` : observer and routes

```
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import 'core/di/injection.dart';
import 'core/errors/app_bloc_observer.dart';
import 'feature/auth/presentation/cubit/login_cubit.dart';
import 'feature/auth/presentation/pages/login_page.dart';
import 'feature/user/presentation/cubit/user_cubit.dart';
import 'feature/user/presentation/pages/user_page.dart';

void main() {
  initDependencies();
  Bloc.observer = AppBlocObserver();
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) => MaterialApp(
    debugShowCheckedModeBanner: false,
    routes: {
      '/': (_) => BlocProvider(
        create: (_) => sl<UserCubit>()..fetchUsers(),
        child: const UserPage(),
      ),
      '/login': (_) => BlocProvider(
        create: (_) => sl<LoginCubit>(),
        child: const LoginPage(),
      ),
    },
  );
}
```

Verify (all executed on your real project)

```
flutter analyze                # strict mode: No issues found
flutter test                   # 74 passed
flutter test --coverage        # ≈ 97.9% of the loaded library lines
flutter build web --debug      # compiles
grep -rnE "data/|presentation/|package:flutter|package:dio" lib/feature/*/domain   # prints nothing
```

Score → 98 *(Architecture 18→20, Cubit 14→15, API 12→15, Model 9→10, Error 8→10, DI 4→5, Quality 3→5).* **Say:** "*Every finding came with a command that reproduced it, and every fix came with a test that would have caught it.*"

STEP 10: Speed: the last 2 points (98 → 100)

You can have perfect architecture and still fail the round if the screen isn't running at minute 20. The **LITE tier** (Guide §4A, 14 files, ≈320 lines) is built for this. Drill it:

Drill	Task	Target
1	Read Guide §4A, then type LITE **while reading** (blank project)	finish ≤ 35 min
2	Type LITE **from the 13-step list only** (Guide §21), no looking	≤ 25 min
3	Same, **different API**: `https://jsonplaceholder.typicode.com/posts?_page=1&_limit=10`. It's a bare list (**no envelope**), so you must adapt the model/entity.	≤ 22 min
4	Same as 2, **plus search** (Guide §8)	add ≤ 4 min
5	Mock interview: record yourself giving the 20 s / 60 s / 2 min answers (Guide §19) while coding	no hesitation on *why*
6	Same, with a **POST** (login) instead of the list	≤ 22 min

Pass criteria for 5/5 speed: running screen with loading / error+Retry / list by **minute 15**; Entity ≠ Model; interface in domain; Cubit → UseCase → Repository; then pagination by 18; then explain.

The "100/100" checklist (what an interviewer scans for)

Print this. Every box is something I can point at in the code.

Architecture (20)

- [ ] `grep` for `data/` in `domain/` prints nothing; domain imports no Flutter/Dio
- [ ] Interface in `domain/repository`, implementation in `data/repository`
- [ ] Folder/file names are correct and consistent; file name = class it contains
- [ ] Only the composition root (DI/`main.dart`) imports concrete classes from every layer
- [ ] A 2nd feature (auth) added without touching the 1st

BLoC/Cubit (15)

- [ ] Sealed states + exhaustive `switch`; `Equatable`; `copyWith`
- [ ] Separate initial-loading vs pagination-loading
- [ ] `loadMore` guarded; list kept on page-2 failure
- [ ] `on Failure catch`, `isClosed` after `await`; Cubit is a **factory**
- [ ] Cubit tests (success, failure, pagination)

API integration (15)

- [ ] `BaseOptions` (baseUrl, timeouts, headers); relative endpoints
- [ ] `DioClient` with GET/POST/PUT/PATCH/DELETE; one error mapping
- [ ] Interceptors: auth token + refresh-once + debug-only safe logging
- [ ] Empty/invalid responses handled; query params correct

Model/Entity (10): `toEntity()`, strict casts, entities have no JSON, envelope model→entity

Repository (10): returns entities only; `guard()` maps exceptions; tested with a mocked data source

Error handling (10): Exception→Failure→State, user-safe messages, Retry, 401/422/offline/timeout/invalid JSON distinguished

DI (5): no `static late`; stateless = singleton, Cubit = factory

Code quality (5): 0 analyzer issues, no TODO/empty/dead code, consistent imports, green tests

SOLID/DRY/KISS (5): new endpoint = ~2 lines in `DioClient` + the usual six steps; nothing duplicated

Speed (5): LITE tier in ≤ 20 min (Step 10)

The final scorecard (re-audited on the real project)

Category	Score	Evidence
Architecture	**20/20**	Dependency rule enforced by grep; interface in domain; one composition root; a second feature (auth) added without touching the first; interceptor wired inside `createDio()`

Category	Score	Evidence
BLoC/Cubit	**15/15**	Sealed + Equatable states, `copyWith`, guarded `loadMore`, generation counter, `isClosed`, `on Failure`, factory registration; 11 Cubit tests incl. stale/cancel/closed cases
API Integration	**15/15**	`BaseOptions`, `DioClient` with 5 verbs + `CancelToken`, one error mapping, auth + refresh interceptor, safe logger; all tested against a fake HTTP layer; GET/POST verified live
Model/Entity	**10/10**	`toEntity()`, strict casts, no unused `toJson`, envelope model → entity
Repository	**10/10**	Interface in domain, `guard()`, tested for every exception → failure mapping
Error Handling	**10/10**	Exception → Failure → State; 401/403/422/5xx/offline/timeout/invalid JSON/cancel distinguished; technical cause preserved and reported to `Bloc.observer`, user-safe messages
Dependency Injection	**5/5**	Lazy, order-free, Cubits are factories, graph covered by a test
Code Quality	**5/5**	Strict analyzer 0 issues; 74 tests, ≈98% coverage; no dead code, TODO or empty files; README
SOLID/DRY/KISS	**5/5**	New endpoint = ~2 lines in `DioClient` + the usual six steps; nothing duplicated
Live-Coding Speed	**3/5**	The code base is the FULL tier (~1,130 lib lines). The remaining 2 points are **yours**: Step 10
TOTAL	**98/100**	**100 once Step 10 is done**

Known limits (stated, not hidden): `main.dart` isn't exercised by tests (it is covered by the DI-graph test and a web build); the refresh endpoint `/auth/refresh` is a template because reqres has no refresh API (the flow itself is proven against a fake server); there are no integration tests on a real device.

Status: applied

Steps 1–9 are applied to your real project and verified there. The "Your code today" snippets above show the **original** code (kept in `demoapi_backup_before_100/`). Only **Step 10** remains, and that one is practice, not code.

Part 3 · Flutter API Integration — 20-Minute Live Coding Guide

> **How this book was built.** Every code block in sections 4–17 is pulled from real projects that were compiled with `flutter analyze` (0 issues) and tested (**74 passing tests, ≈98% line coverage**: cubit, repository, data source, `DioClient`, auth interceptor, token storage, DI graph, pages, login). The GET and POST/login flows were also run end-to-end against `https://reqres.in/api` (page 2 → 6 users; `POST /login` with a missing password → `400 "Missing password"` → mapped correctly; `/users/9999` → 404; unreachable host → `NetworkFailure`). Nothing here is pseudo-code unless it says **(illustrative)**.
>
> **Dependencies:** `flutter_bloc ^9`, `dio ^5`, `equatable ^3`, `get_it ^9` (optional), `flutter_secure_storage` (auth only). Dev: `bloc_test`, `mocktail`. Optional: `bloc_concurrency`. `flutter pub add flutter_bloc dio equatable get_it · flutter pub add --dev bloc_test mocktail`

Read this first: the two tiers (honesty about 20 minutes)

The full production-quality reference below is **~790 lines** for one feature (plus ~350 for the auth feature and its infrastructure). Nobody types that in 20 minutes. So there are two tiers:

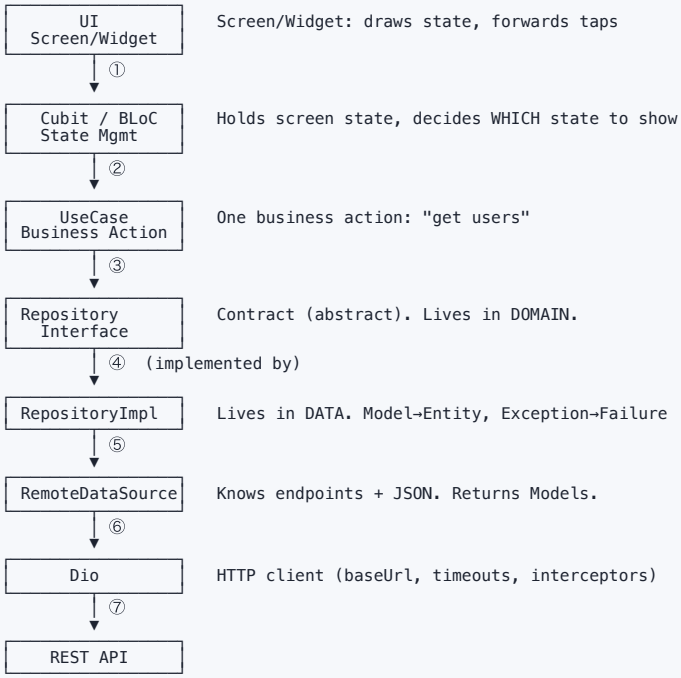
Tier	What	Lines	When
LITE: you type this	Same layers (Entity → Model → Repository interface → DataSource → RepositoryImpl → UseCase → Cubit → UI), `toEntity()` mapping, one `Failure` class, manual DI, retry, and pagination	**≈ 320** (≈ 270 before pagination)	The 20-minute round
FULL: you explain this	`DioClient` (5 verbs), `AppException` → `Failure` hierarchy + `guard()`, `Equatable` states, `loadMoreFailed`, stale-response guard, cause-preserving failures + logging hook, cancellation, GetIt, interceptors, tests	≈ 790	"How would you make this production-ready?" and for take-home / pair-programming rounds

The architecture, layer names, and data flow are **identical** in both. LITE is not "worse architecture": it is the same architecture with less ceremony. You say out loud what you'd add. Sections 4A/4B show both; section 21 is the minute-by-minute script.

Priority order (memorize): `Correctness → Clean Architecture → Readability → Speed → Explainability.` A working screen with the right layers beats a half-finished perfect one. **Get the happy path on screen first, then harden.**

SECTION 1 — The Master Architecture

1.1 The call chain



Explaining every arrow (request direction):

#	Arrow	What happens	Why this boundary exists
①	UI → Cubit	`context.read<UserCubit>().loadMore()`	UI never fetches data or holds business state. It forwards intent.
②	Cubit → UseCase	`_getUsers(page: 2)`	Cubit asks for a business action, not an HTTP call. Easy to mock in tests.
③	UseCase → Repository (interface)	`_repository.getUsers(page: 2)`	The use case depends on an *abstraction*, so it is blind to REST/DB/cache.
④	Interface ⇢ Impl	DI wires `UserRepositoryImpl` behind `UserRepository`	Dependency Inversion: data implements what domain defines.
⑤	RepositoryImpl → DataSource	`_remote.getUsers(page: 2)`	Repository chooses *where* data comes from (remote now, cache/local later).
⑥	DataSource → Dio	`_dio.get('/users', queryParameters: {'page': 2})`	Only the data source knows endpoint paths and JSON shapes.
⑦	Dio → API	HTTP request	Network, auth headers, timeouts, logging live in Dio config/interceptors.

1.2 The response flow

```
API
 ↓  ⓐ
JSON            {"page":2,"total_pages":2,"data":[{"id":7,"first_name":"Michael",...}]}
 ↓  ⓑ
Model           UsersResponseModel / UserModel      (mirrors the JSON: snake_case, strict types)
 ↓  ⓒ
Entity          PagedUsers / User                   (mirrors the business: no JSON anywhere)
 ↓  ⓓ
Repository      returns Entity, converts Exception → Failure
 ↓  ⓔ
UseCase         passes through (or applies a rule)
 ↓  ⓕ
Cubit           chooses the state
 ↓  ⓖ
State           UserLoaded(users, page, hasNext, ...)
 ↓  ⓗ
UI              BlocBuilder draws it
```

#	Arrow	Explanation
ⓐ	API → JSON	Dio decodes the body into `Map<String, dynamic>`. Invalid JSON surfaces as `DioException(unknown, FormatException)`.
ⓑ	JSON → Model	`UsersResponseModel.fromJson(json)`: strict `as int` / `as String`. A wrong shape throws `TypeError`.
ⓒ	Model → Entity	`model.toEntity()`: **the anti-corruption boundary.** Backend renames never reach the app.
ⓓ	Entity out of Repository	Only entities (or `Failure`s) leave `data/`. No `Dio`, no `Model` above this line.
ⓔ	→ UseCase	Business rules can run here.
ⓕ	→ Cubit	`emit(UserLoaded(...))` or `emit(UserError(failure.message))`.
ⓖ	State → UI	Immutable snapshot of "what the screen shows".
ⓗ	UI draws	`switch (state)` is exhaustive (sealed) so no state is forgotten.

1.3 The dependency rule (the part interviewers actually grade)

The *call* arrows above go downward. The *import* arrows go **inward**:



Folder	Responsibility	Depends On
`presentation`	UI + state (Cubit, pages, widgets)	`domain`
`domain`	Business objects + contracts (entities, repository interface, use cases)	**nothing** (pure Dart + equatable)
`data`	API/DB implementation (models, data sources, repository impl)	`domain` (+ `core`, Dio)
`core`	Shared infrastructure (errors, network, DI, constants)	framework / external packages

One-line test you can run any time (must print nothing):

```
grep -rnE "data/|presentation/|package:flutter|package:dio" lib/feature/*/domain
```

SECTION 2 — Folder Structure

FULL tier tree (LITE omits `exceptions.dart`, `dio_client.dart`, `di/`, `widgets/`, and merges data-source interface + impl into one file):

```
lib/
├── core/
│   ├── constants/
│   │   └── api_constants.dart          base URL, paths, timeout
│   ├── network/
│   │   └── dio_client.dart             5 verbs + DioException → AppException (once)
│   ├── errors/
│   │   ├── exceptions.dart             thrown by data layer (ServerException, NetworkException…)
│   │   └── failures.dart               seen by Cubit (Failure types) + guard()
│   └── di/
│       └── injection.dart              GetIt wiring (or manual DI in main.dart)
├── feature/
│   └── user/
│       ├── data/
│       │   ├── data_source/            user_remote_data_source.dart (+ _impl.dart)
│       │   ├── model/                  user_model.dart, users_response_model.dart
│       │   └── repository/             user_repository_impl.dart
│       ├── domain/
│       │   ├── entity/                 user.dart, paged_users.dart
│       │   ├── repository/             user_repository.dart (abstract)
│       │   └── usecase/                get_users.dart
│       └── presentation/
│           ├── cubit/                  user_cubit.dart, user_state.dart
│           ├── pages/                  user_page.dart
│           └── widgets/                user_tile.dart, error_view.dart, load_more_footer.dart
└── main.dart
```

Folder	One-line job
`core/constants`	Strings/numbers shared everywhere (URLs, timeouts). Never inline a URL in a data source.
`core/network`	The only code that imports Dio's error types.
`core/errors`	The vocabulary of failure: exceptions in `data`, failures above.
`core/di`	The only file allowed to import concrete classes from every layer.
`feature/<name>/data`	How we get the data: JSON models, HTTP calls, mapping.
`feature/<name>/domain`	What the app needs: entities, contracts, business actions. No I/O.
`feature/<name>/presentation`	What the user sees: Cubit + widgets.

Do **not** add `utils/`, `helpers/`, `services/`, `bloc/` + `cubit/` double folders. Each extra folder is typing time and a question you'll have to answer.

SECTION 3 — The 20-Minute Implementation Plan (stopwatch)

> Rule: **after minute 13 the app must run.** Everything after is bonus/hardening. If you are behind at any checkpoint, *cut* (see the "cut list"), never rush the layers.

Time	Do	Files (LITE)	Say out loud
0–2 min	Read the API response. Add deps. Create folders.	`flutter pub add flutter_bloc dio` · `api_constants.dart`	"GET `/users?page=` returns `{page, total_pages, data[]}`. I'll model it in three layers: JSON model, domain entity, state."
2–5 min	Domain + Models	`user.dart`, `paged_users.dart`, `user_model.dart` (+ `toEntity`), `users_response_model.dart`, `user_repository.dart`	"Entity is pure business. Model mirrors JSON. Interface lives in domain so domain never imports data."
5–8 min	Data layer	`failure.dart`, `user_remote_data_source.dart` (interface + impl), `user_repository_impl.dart`	"Data source owns the endpoint and parsing; repository maps Model→Entity."
8–11 min	Use case + Cubit + States	`get_users.dart`, `user_state.dart`, `user_cubit.dart` (`fetchUsers` only)	"Sealed states so the UI `switch` is exhaustive. Cubit never touches Dio."
11–15 min	UI + DI + RUN	`user_page.dart` (loading / error+retry / list), `main.dart` (manual DI) → **press ▶**	"Composition root in `main.dart`: the only place that knows concrete classes."
15–18 min	Add what they ask	`loadMore()` + footer (pagination) **or** search **or** pull-to-refresh	"`loadMore` ignores calls while loading or on the last page."
18–20 min	Explain	Architecture, SOLID, error handling, *why* Cubit/Repository/UseCase	Use section 19's scripts.

The cut list (what to drop when behind)

Behind at…	Cut	Say instead
8 min	Data-source **interface** (keep only the class)	"I'd put an interface here for testing; skipping for time."
11 min	Use case (call repository from Cubit)	"In production I'd add the use case as the seam for business rules." (try hard to keep: it is the expected layering)
13 min	`copyWith`/Equatable, error retry button	"States would be Equatable; retry is a one-liner."
15 min	Pagination	Describe `currentPage/totalPages/isLoadingMore/hasReachedMax` verbally
Never cut	Entity vs Model separation · interface in domain · Cubit → UseCase → Repository · error state in the UI	

Typing-speed tricks (use your IDE, don't type boilerplate)

- **Constructors:** declare `final` fields, then `Ctrl/Cmd + .` → *Create constructor / Add required keyword*.
- `stless` snippet for widgets; `Ctrl + .` → *Wrap with BlocBuilder*.
- **Don't write `toJson` unless you POST that model.**
- Write the `sealed class` states first: the compiler then tells you every `switch` you must complete.
- Paste the sample JSON into a comment first; field names are then a copy-paste, not a guess.

SECTION 4 — Basic API: `GET /api/users?page=1`, from `main.dart` to UI

Sample response (paste this into a comment at minute 0; it is your spec):

```
{
  "page": 1, "per_page": 6, "total": 12, "total_pages": 2,
  "data": [
    { "id": 1, "email": "george.bluth@reqres.in", "first_name": "George",
      "last_name": "Bluth", "avatar": "https://reqres.in/img/faces/1-image.jpg" }
  ]
}
```

4A. LITE: the version you type (compiles, 0 analyzer issues, widget-tested)

Typing order = dependency order: **domain** → **data** → **presentation** → **main**. Each file is followed by what to say.

`lib/core/constants/api_constants.dart`

```dart
class ApiConstants {
  static const baseUrl = 'https://reqres.in/api';
  static const users = '/users';
}
```

`lib/core/errors/failure.dart`: "*One failure type carrying a user-safe message. In production this becomes a sealed hierarchy.*"

```dart
class Failure implements Exception {
  const Failure(this.message);

  final String message;
}
```

`lib/feature/user/domain/entity/user.dart`: "*Entity: pure business object. No JSON, no Flutter.*"

```dart
class User {
  const User({
    required this.id,
    required this.email,
    required this.firstName,
    required this.lastName,
    required this.avatar,
  });

  final int id;
  final String email;
  final String firstName;
  final String lastName;
  final String avatar;
}
```

`lib/feature/user/domain/entity/paged_users.dart`: "*A page of users is a business concept, so it lives in domain, not in data. This is the file that was in the wrong layer in the audit.*"

```dart
import 'user.dart';

class PagedUsers {
  const PagedUsers({required this.users, required this.page, required this.totalPages});

  final List<User> users;
  final int page;
  final int totalPages;
}
```

`lib/feature/user/data/model/user_model.dart`: "*Model mirrors the API and knows JSON.* `toEntity()` *is the boundary.*"

```dart
import '../../domain/entity/user.dart';

class UserModel {
  const UserModel({
    required this.id,
    required this.email,
    required this.firstName,
    required this.lastName,
    required this.avatar,
  });

  final int id;
  final String email;
  final String firstName;
  final String lastName;
  final String avatar;
```

```
  factory UserModel.fromJson(Map<String, dynamic> json) => UserModel(
    id: json['id'] as int,
    email: json['email'] as String,
    firstName: json['first_name'] as String,
    lastName: json['last_name'] as String,
    avatar: json['avatar'] as String,
  );

  User toEntity() =>
      User(id: id, email: email, firstName: firstName, lastName: lastName, avatar: avatar);
}
```

`lib/feature/user/data/model/users_response_model.dart` : "*The envelope: pagination metadata + list.*"

```
import '../../domain/entity/paged_users.dart';
import 'user_model.dart';

class UsersResponseModel {
  const UsersResponseModel({required this.page, required this.totalPages, required this.data});

  final int page;
  final int totalPages;
  final List<UserModel> data;

  factory UsersResponseModel.fromJson(Map<String, dynamic> json) => UsersResponseModel(
    page: json['page'] as int,
    totalPages: json['total_pages'] as int,
    data: (json['data'] as List).map((e) => UserModel.fromJson(e as Map<String, dynamic>)).toList(),
  );

  PagedUsers toEntity() =>
      PagedUsers(users: data.map((m) => m.toEntity()).toList(), page: page, totalPages: totalPages);
}
```

`lib/feature/user/domain/repository/user_repository.dart` : "*Contract defined by the domain. Returns entities only.*"

```
import '../entity/paged_users.dart';

abstract interface class UserRepository {
  Future<PagedUsers> getUsers({required int page});
}
```

`lib/feature/user/data/data_source/user_remote_data_source.dart` : *interface + implementation in one file (LITE shortcut; FULL splits them into* `user_remote_data_source.dart` *and* `user_remote_data_source_impl.dart`*).* "Only this class knows the endpoint and converts `DioException` into a user-safe `Failure`."

```
import 'package:dio/dio.dart';

import '../../../../core/constants/api_constants.dart';
import '../../../../core/errors/failure.dart';
import '../model/users_response_model.dart';

abstract interface class UserRemoteDataSource {
  Future<UsersResponseModel> getUsers({required int page});
}

class UserRemoteDataSourceImpl implements UserRemoteDataSource {
  const UserRemoteDataSourceImpl(this._dio);

  final Dio _dio;

  @override
  Future<UsersResponseModel> getUsers({required int page}) async {
    try {
      final response = await _dio.get(ApiConstants.users, queryParameters: {'page': page});
      return UsersResponseModel.fromJson(response.data as Map<String, dynamic>);
    } on DioException catch (e) {
      throw Failure(switch (e.type) {
        DioExceptionType.connectionTimeout ||
        DioExceptionType.receiveTimeout => 'Request timed out.',
        DioExceptionType.connectionError => 'No internet connection.',
        DioExceptionType.badResponse => 'Server error (${e.response?.statusCode}).',
        _ => 'Something went wrong.',
      });
    }
  }
}
```

`lib/feature/user/data/repository/user_repository_impl.dart` : "*Implements the domain contract; maps Model → Entity.*"

```dart
import '../../domain/entity/paged_users.dart';
import '../../domain/repository/user_repository.dart';
import '../data_source/user_remote_data_source.dart';

class UserRepositoryImpl implements UserRepository {
  const UserRepositoryImpl(this._remote);

  final UserRemoteDataSource _remote;

  @override
  Future<PagedUsers> getUsers({required int page}) async =>
      (await _remote.getUsers(page: page)).toEntity();
}
```

`lib/feature/user/domain/usecase/get_users.dart` : "*One business action, one file.* `call()` *lets me invoke it like a function.*"

```dart
import '../entity/paged_users.dart';
import '../repository/user_repository.dart';

class GetUsers {
  const GetUsers(this._repository);

  final UserRepository _repository;

  Future<PagedUsers> call({required int page}) => _repository.getUsers(page: page);
}
```

`lib/feature/user/presentation/cubit/user_state.dart` : "*Sealed: the UI* `switch` *must handle every state or it will not compile.*"

```dart
import '../../domain/entity/user.dart';

sealed class UserState {
  const UserState();
}

class UserInitial extends UserState {
  const UserInitial();
}

class UserLoading extends UserState {
  const UserLoading();
}

class UserLoaded extends UserState {
  const UserLoaded({
    required this.users,
    required this.page,
    required this.hasNext,
    this.isLoadingMore = false,
  });

  final List<User> users;
  final int page;
  final bool hasNext;
  final bool isLoadingMore;

  UserLoaded copyWith({List<User>? users, int? page, bool? hasNext, bool? isLoadingMore}) =>
      UserLoaded(
        users: users ?? this.users,
        page: page ?? this.page,
        hasNext: hasNext ?? this.hasNext,
        isLoadingMore: isLoadingMore ?? this.isLoadingMore,
      );
}

class UserError extends UserState {
  const UserError(this.message);

  final String message;
}
```

`lib/feature/user/presentation/cubit/user_cubit.dart` : "*Cubit only orchestrates.* `loadMore` *is guarded against double calls and the last page.*"

```dart
import 'package:flutter_bloc/flutter_bloc.dart';

import '../../../../core/errors/failure.dart';
import '../../domain/usecase/get_users.dart';
import 'user_state.dart';

class UserCubit extends Cubit<UserState> {
  UserCubit(this._getUsers) : super(const UserInitial());
```

```
  final GetUsers _getUsers;

  Future<void> fetchUsers() async {
    emit(const UserLoading());
    try {
      final result = await _getUsers(page: 1);
      emit(
        UserLoaded(
          users: result.users,
          page: result.page,
          hasNext: result.page < result.totalPages,
        ),
      );
    } on Failure catch (f) {
      emit(UserError(f.message));
    } catch (_) {
      emit(const UserError('Something went wrong.'));
    }
  }

  Future<void> loadMore() async {
    final current = state;
    if (current is! UserLoaded || current.isLoadingMore || !current.hasNext) {
      return;
    }
    emit(current.copyWith(isLoadingMore: true));
    try {
      final result = await _getUsers(page: current.page + 1);
      emit(
        current.copyWith(
          users: [...current.users, ...result.users],
          page: result.page,
          hasNext: result.page < result.totalPages,
          isLoadingMore: false,
        ),
      );
    } catch (_) {
      emit(current.copyWith(isLoadingMore: false)); // keep list, stop spinner
    }
  }
}
```

`lib/feature/user/presentation/pages/user_page.dart`: "*The `switch` covers loading, error with retry, and the list. Reaching the footer asks for the next page.*"

```
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import '../cubit/user_cubit.dart';
import '../cubit/user_state.dart';

class UserPage extends StatelessWidget {
  const UserPage({super.key});

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Users')),
    body: BlocBuilder<UserCubit, UserState>(
      builder: (context, state) => switch (state) {
        UserInitial() || UserLoading() => const Center(child: CircularProgressIndicator()),
        UserError(:final message) => Center(
          child: Column(
            mainAxisSize: MainAxisSize.min,
            children: [
              Text(message),
              ElevatedButton(
                onPressed: context.read<UserCubit>().fetchUsers,
                child: const Text('Retry'),
              ),
            ],
          ),
        ),
        UserLoaded() => ListView.builder(
          itemCount: state.users.length + (state.hasNext ? 1 : 0),
          itemBuilder: (context, i) {
            if (i == state.users.length) {
              // footer built = user reached the end -> next page
              final cubit = context.read<UserCubit>();
              Future.microtask(cubit.loadMore);
              return const Padding(
                padding: EdgeInsets.all(16),
                child: Center(child: CircularProgressIndicator()),
              );
            }
```

```
            final user = state.users[i];
            return ListTile(
              leading: CircleAvatar(
                foregroundImage: NetworkImage(user.avatar),
                onForegroundImageError: (_, _) {},
                child: const Icon(Icons.person),
              ),
              title: Text('${user.firstName} ${user.lastName}'),
              subtitle: Text(user.email),
            );
          },
        ),
      },
    ),
  );
}
```

`lib/main.dart` : *composition root + manual DI.* "The only place that knows concrete classes. `..fetchUsers()` kicks off the first load."

```
import 'package:dio/dio.dart';
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import 'core/constants/api_constants.dart';
import 'feature/user/data/data_source/user_remote_data_source.dart';
import 'feature/user/data/repository/user_repository_impl.dart';
import 'feature/user/domain/usecase/get_users.dart';
import 'feature/user/presentation/cubit/user_cubit.dart';
import 'feature/user/presentation/pages/user_page.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    // Composition root: the only place that knows the concrete classes.
    final dio = Dio(
      BaseOptions(
        baseUrl: ApiConstants.baseUrl,
        connectTimeout: const Duration(seconds: 10),
        receiveTimeout: const Duration(seconds: 10),
      ),
    );
    final repository = UserRepositoryImpl(UserRemoteDataSourceImpl(dio));

    return MaterialApp(
      home: BlocProvider(
        create: (_) => UserCubit(GetUsers(repository))..fetchUsers(),
        child: const UserPage(),
      ),
    );
  }
}
```

▶ **RUN here.** If page 1 shows, you are at minute ~13. Everything after is bonus.

> **Why the footer trigger (`Future.microtask(cubit.loadMore)`) instead of a `ScrollController` ?** reqres returns 6 users per page. On a tall phone, 6 tiles don't fill the screen: nothing scrolls, so a scroll listener never fires and page 2 is never requested. Building the footer = "the user can see the end", on any screen size. `Future.microtask` avoids calling `emit` while the framework is building. The Cubit's guard makes the repeated calls harmless. (The `ScrollController` version is in section 7.)

4B. FULL: the production upgrade (what you *say* you'd do; every file compiled and tested)

What changes vs LITE, and why:

LITE	FULL	Why
`Failure(message)` thrown from the data source	`AppException` (data) → `Failure` (domain/UI) + `guard()` in the repository	Cubit can react differently to 401 vs offline vs 422; Dio never leaks upward
Dio used directly in each data source	`DioClient` : one place for 5 verbs + error mapping	DRY: every new endpoint is 2–3 lines
States without equality	`Equatable` + `loadMoreFailed` + `hasReachedMax`	No duplicate rebuilds; testable; page-2 failure keeps the list
`catch (_)`	`on Failure catch` + `if (isClosed) return;` + a **generation counter**	Don't hide programmer bugs; no emit after close; a stale response (Retry spam, refresh during `loadMore`) is dropped

LITE	FULL	Why
Failure = message only	Failure also carries `cause` + `stackTrace`; `FailureReporting` mixin forwards them to `Bloc.observer`	Users see a safe message, your logger/crash tool sees the real error
Request can't be cancelled	`CancelToken` on every `DioClient` verb → `CancelledFailure`, ignored by Cubits	A cancelled request is not an error
Manual DI	GetIt (Cubit as **factory**)	Scales to many features; test overrides
Inline tile / footer / error	`UserTile`, `ErrorView`, `LoadMoreFooter` widgets	Small widgets, reuse

`core/constants/api_constants.dart`

```dart
abstract final class ApiConstants {
  static const baseUrl = 'https://reqres.in/api';

  static const users = '/users';
  static const login = '/login';

  static const timeout = Duration(seconds: 10);
}
```

`core/errors/exceptions.dart` : thrown by the **data layer** only.

```dart
/// Thrown by the data layer (DioClient / data sources). Never reaches the UI.
sealed class AppException implements Exception {
  const AppException(this.message);

  final String message;
}

/// The server answered with a non-2xx status code.
class ServerException extends AppException {
  const ServerException(super.message, {this.statusCode});

  final int? statusCode;
}

/// No internet, timeout, DNS failure...
class NetworkException extends AppException {
  const NetworkException(super.message);
}

/// 200 OK but the body is not valid JSON.
class ParsingException extends AppException {
  const ParsingException(super.message);
}

/// The caller cancelled the request (CancelToken). Not an error to show.
class CancelledException extends AppException {
  const CancelledException(super.message);
}
```

`core/errors/failures.dart` : what the **domain/UI** understands, plus `guard()`, the single Exception→Failure converter. Failures carry the technical `cause` / `stackTrace` (never shown) so nothing is swallowed.

```dart
import 'package:equatable/equatable.dart';

import 'exceptions.dart';

/// What the domain / Cubit understands. [message] is always user-safe.
sealed class Failure extends Equatable implements Exception {
  const Failure(this.message, {this.cause, this.stackTrace});

  final String message;

  /// Technical reason. NEVER shown to users; forwarded to `Bloc.observer.onError`
  /// (see `reportFailure`) so unexpected problems reach your logger/crash tool.
  final Object? cause;
  final StackTrace? stackTrace;

  @override
  List<Object?> get props => [message];
}

class ServerFailure extends Failure {
  const ServerFailure(super.message, {this.statusCode});

  final int? statusCode;
```

```
  @override
  List<Object?> get props => [message, statusCode];
}

class UnauthorizedFailure extends Failure {
  const UnauthorizedFailure(super.message);
}

/// 422 / client-side validation: show the message next to the form.
class ValidationFailure extends Failure {
  const ValidationFailure(super.message);
}

class NetworkFailure extends Failure {
  const NetworkFailure(super.message);
}

/// The request was cancelled on purpose: callers ignore it, no error UI.
class CancelledFailure extends Failure {
  const CancelledFailure() : super('Request cancelled.');
}

class ParsingFailure extends Failure {
  const ParsingFailure({super.cause, super.stackTrace})
    : super('Unexpected data received from server.');
}

class UnknownFailure extends Failure {
  const UnknownFailure({super.cause, super.stackTrace})
    : super('Something went wrong. Please try again.');
}

/// Repositories wrap every data-source call in this: Exception -> Failure, once.
/// Technical causes are kept on the Failure, never swallowed.
Future<T> guard<T>(Future<T> Function() action) async {
  try {
    return await action();
  } on ServerException catch (e) {
    throw switch (e.statusCode) {
      401 || 403 => UnauthorizedFailure(e.message),
      422 => ValidationFailure(e.message),
      _ => ServerFailure(e.message, statusCode: e.statusCode),
    };
  } on NetworkException catch (e) {
    throw NetworkFailure(e.message);
  } on CancelledException {
    throw const CancelledFailure();
  } on ParsingException catch (e, s) {
    throw ParsingFailure(cause: e, stackTrace: s); // body is not valid JSON
  } on TypeError catch (e, s) {
    throw ParsingFailure(cause: e, stackTrace: s); // wrong shape: null / type
  } on FormatException catch (e, s) {
    throw ParsingFailure(cause: e, stackTrace: s);
  } catch (e, s) {
    throw UnknownFailure(cause: e, stackTrace: s);
  }
}
```

`core/errors/report_failure.dart` and **`app_bloc_observer.dart`**: the logging hook. A Cubit calls `reportFailure(failure)`; parsing/unknown failures reach `Bloc.observer.onError`, the one place to plug Crashlytics/Sentry. Expected failures (offline, 401, 422) have no cause and are not reported.

```
import 'package:flutter_bloc/flutter_bloc.dart';

import 'failures.dart';

mixin FailureReporting<S> on BlocBase<S> {
  /// Forwards the hidden technical cause of a [Failure] (parsing/unknown) to
  /// `Bloc.observer.onError`: one place to plug Crashlytics/Sentry.
  /// Expected failures (offline, 401, 422...) have no cause and are not reported.
  void reportFailure(Failure failure) {
    final cause = failure.cause;
    if (cause != null) addError(cause, failure.stackTrace);
  }
}
```

```
import 'package:flutter/foundation.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

/// Central error sink for every Cubit/Bloc. Replace `debugPrint` with your
/// crash reporter (e.g. Crashlytics.recordError) for production.
class AppBlocObserver extends BlocObserver {
  @override
```

```
  void onError(BlocBase<dynamic> bloc, Object error, StackTrace stackTrace) {
    if (kDebugMode) debugPrint('[${bloc.runtimeType}] $error\n$stackTrace');
    super.onError(bloc, error, stackTrace);
  }
}
```

`core/network/dio_client.dart`: all verbs, one `_send`, one mapping.

```
import 'package:dio/dio.dart';

import '../errors/exceptions.dart';

/// Thin wrapper over Dio: one place for every HTTP verb and for
/// DioException -> AppException mapping.
class DioClient {
  const DioClient(this._dio);

  final Dio _dio;

  Future<dynamic> get(
    String path, {
    Map<String, dynamic>? query,
    CancelToken? cancelToken,
  }) => _send(
    () => _dio.get<dynamic>(
      path,
      queryParameters: query,
      cancelToken: cancelToken,
    ),
  );

  Future<dynamic> post(String path, {Object? data, CancelToken? cancelToken}) =>
      _send(
        () => _dio.post<dynamic>(path, data: data, cancelToken: cancelToken),
      );

  Future<dynamic> put(String path, {Object? data, CancelToken? cancelToken}) =>
      _send(
        () => _dio.put<dynamic>(path, data: data, cancelToken: cancelToken),
      );

  Future<dynamic> patch(
    String path, {
    Object? data,
    CancelToken? cancelToken,
  }) => _send(
    () => _dio.patch<dynamic>(path, data: data, cancelToken: cancelToken),
  );

  Future<dynamic> delete(String path, {CancelToken? cancelToken}) =>
      _send(() => _dio.delete<dynamic>(path, cancelToken: cancelToken));

  Future<dynamic> _send(Future<Response<dynamic>> Function() request) async {
    try {
      return (await request()).data;
    } on DioException catch (e) {
      throw _toException(e);
    }
  }

  AppException _toException(DioException e) => switch (e.type) {
    DioExceptionType.connectionTimeout ||
    DioExceptionType.sendTimeout ||
    DioExceptionType.receiveTimeout => const NetworkException(
      'Request timed out. Please try again.',
    ),
    DioExceptionType.connectionError => const NetworkException(
      'No internet connection.',
    ),
    DioExceptionType.cancel => const CancelledException('Request cancelled.'),
    DioExceptionType.badResponse => ServerException(
      _bodyMessage(e.response) ?? _defaultMessage(e.response?.statusCode),
      statusCode: e.response?.statusCode,
    ),
    // Dio wraps a JSON decoding error as `unknown` + FormatException.
    DioExceptionType.unknown when e.error is FormatException =>
      const ParsingException('Invalid response from server.'),
    _ => const NetworkException('Network error. Please try again.'),
  };

  /// reqres returns {"error": "..."}; many APIs use {"message": "..."}.
  String? _bodyMessage(Response<dynamic>? response) {
    final data = response?.data;
    if (data is! Map) return null;
    final message = data['error'] ?? data['message'];
```

```
    return message is String ? message : null;
  }

  String _defaultMessage(int? code) => switch (code) {
    400 => 'Bad request.',
    401 => 'Please login again.',
    403 => 'You do not have access.',
    404 => 'Not found.',
    422 => 'Validation failed.',
    final int c when c >= 500 => 'Server error. Please try later.',
    _ => 'Unexpected server response ($code).',
  };
}
```

`domain/entity/user.dart` and **`domain/entity/paged_users.dart`** (Equatable, `fullName`, `hasNext`)

```
import 'package:equatable/equatable.dart';

class User extends Equatable {
  const User({
    required this.id,
    required this.email,
    required this.firstName,
    required this.lastName,
    required this.avatar,
  });

  final int id;
  final String email;
  final String firstName;
  final String lastName;
  final String avatar;

  String get fullName => '$firstName $lastName';

  @override
  List<Object?> get props => [id, email, firstName, lastName, avatar];
}
```

```
import 'package:equatable/equatable.dart';

import 'user.dart';

class PagedUsers extends Equatable {
  const PagedUsers({
    required this.users,
    required this.page,
    required this.totalPages,
  });

  final List<User> users;
  final int page;
  final int totalPages;

  bool get hasNext => page < totalPages;

  @override
  List<Object?> get props => [users, page, totalPages];
}
```

(Unchanged from LITE: `user_repository.dart`, `get_users.dart`. Models: same as LITE plus optional `toJson` on `UserModel` and `perPage` / `total` on the response model.)

`data/data_source/user_remote_data_source.dart` (interface) and **`..._impl.dart`**

```
import '../model/users_response_model.dart';

abstract interface class UserRemoteDataSource {
  Future<UsersResponseModel> getUsers({required int page});
}
```

```
import '../../../../core/constants/api_constants.dart';
import '../../../../core/network/dio_client.dart';
import '../model/users_response_model.dart';
import 'user_remote_data_source.dart';

class UserRemoteDataSourceImpl implements UserRemoteDataSource {
  const UserRemoteDataSourceImpl(this._client);

  final DioClient _client;

  @override
```

```
  Future<UsersResponseModel> getUsers({required int page}) async {
    final json = await _client.get(ApiConstants.users, query: {'page': page});
    return UsersResponseModel.fromJson(json as Map<String, dynamic>);
  }
}
```

`data/repository/user_repository_impl.dart` : three lines, because `guard()` does the mapping.

```
import '../../../../core/errors/failures.dart';
import '../../domain/entity/paged_users.dart';
import '../../domain/repository/user_repository.dart';
import '../data_source/user_remote_data_source.dart';

class UserRepositoryImpl implements UserRepository {
  const UserRepositoryImpl(this._remote);

  final UserRemoteDataSource _remote;

  @override
  Future<PagedUsers> getUsers({required int page}) =>
      guard(() async => (await _remote.getUsers(page: page)).toEntity());
}
```

`presentation/cubit/user_state.dart`

```
import 'package:equatable/equatable.dart';

import '../../domain/entity/user.dart';

sealed class UserState extends Equatable {
  const UserState();

  @override
  List<Object?> get props => [];
}

class UserInitial extends UserState {
  const UserInitial();
}

/// First page is loading (full-screen spinner).
class UserLoading extends UserState {
  const UserLoading();
}

class UserLoaded extends UserState {
  const UserLoaded({
    required this.users,
    required this.currentPage,
    required this.totalPages,
    this.isLoadingMore = false,
    this.loadMoreFailed = false,
  });

  final List<User> users;
  final int currentPage;
  final int totalPages;
  final bool isLoadingMore; // next page is loading (footer spinner)
  final bool loadMoreFailed; // next page failed (footer retry button)

  bool get hasReachedMax => currentPage >= totalPages;

  UserLoaded copyWith({
    List<User>? users,
    int? currentPage,
    int? totalPages,
    bool? isLoadingMore,
    bool? loadMoreFailed,
  }) => UserLoaded(
    users: users ?? this.users,
    currentPage: currentPage ?? this.currentPage,
    totalPages: totalPages ?? this.totalPages,
    isLoadingMore: isLoadingMore ?? this.isLoadingMore,
    loadMoreFailed: loadMoreFailed ?? this.loadMoreFailed,
  );

  @override
  List<Object?> get props => [
    users,
    currentPage,
    totalPages,
    isLoadingMore,
    loadMoreFailed,
  ];
```

```
}

class UserError extends UserState {
  const UserError(this.message);

  final String message;

  @override
  List<Object?> get props => [message];
}
```

`presentation/cubit/user_cubit.dart`

```
import 'package:flutter_bloc/flutter_bloc.dart';

import '../../../../core/errors/failures.dart';
import '../../../../core/errors/report_failure.dart';
import '../../domain/usecase/get_users.dart';
import 'user_state.dart';

class UserCubit extends Cubit<UserState> with FailureReporting<UserState> {
  UserCubit(this._getUsers) : super(const UserInitial());

  final GetUsers _getUsers;

  /// Bumped by every fresh load. A response that belongs to an older
  /// generation (user hit Retry, or refreshed meanwhile) is dropped.
  int _generation = 0;

  /// First page (also used by the Retry button).
  Future<void> fetchUsers() async {
    final generation = ++_generation;
    emit(const UserLoading());
    try {
      final result = await _getUsers(page: 1);
      if (isClosed || generation != _generation) return;
      emit(
        UserLoaded(
          users: result.users,
          currentPage: result.page,
          totalPages: result.totalPages,
        ),
      );
    } on CancelledFailure {
      return; // superseded on purpose: no error UI
    } on Failure catch (failure) {
      reportFailure(failure);
      if (isClosed || generation != _generation) return;
      emit(UserError(failure.message));
    }
  }

  /// Next page, appended to the current list. Safe to call many times.
  Future<void> loadMore() async {
    final current = state;
    if (current is! UserLoaded ||
        current.isLoadingMore ||
        current.hasReachedMax) {
      return;
    }

    final generation = _generation;
    emit(current.copyWith(isLoadingMore: true, loadMoreFailed: false));
    try {
      final result = await _getUsers(page: current.currentPage + 1);
      if (isClosed || generation != _generation) return; // list was reloaded
      emit(
        current.copyWith(
          users: [...current.users, ...result.users],
          currentPage: result.page,
          totalPages: result.totalPages,
          isLoadingMore: false,
        ),
      );
    } on CancelledFailure {
      return;
    } on Failure catch (failure) {
      reportFailure(failure);
      if (isClosed || generation != _generation) return;
      emit(current.copyWith(isLoadingMore: false, loadMoreFailed: true));
    }
  }
}
```

`presentation/widgets/user_tile.dart`, `error_view.dart`, `load_more_footer.dart`

```dart
import 'package:flutter/material.dart';

import '../../domain/entity/user.dart';

class UserTile extends StatelessWidget {
  const UserTile({super.key, required this.user});

  final User user;

  @override
  Widget build(BuildContext context) => ListTile(
    leading: CircleAvatar(
      foregroundImage: NetworkImage(user.avatar),
      onForegroundImageError: (_, _) {}, // broken URL -> fallback icon
      child: const Icon(Icons.person),
    ),
    title: Text(user.fullName),
    subtitle: Text(user.email),
  );
}
```

```dart
import 'package:flutter/material.dart';

class ErrorView extends StatelessWidget {
  const ErrorView({super.key, required this.message, required this.onRetry});

  final String message;
  final VoidCallback onRetry;

  @override
  Widget build(BuildContext context) => Center(
    child: Column(
      mainAxisSize: MainAxisSize.min,
      children: [
        Text(message, textAlign: TextAlign.center),
        const SizedBox(height: 12),
        FilledButton(onPressed: onRetry, child: const Text('Retry')),
      ],
    ),
  );
}
```

```dart
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import '../cubit/user_cubit.dart';

/// Last list item. The moment it is built (= user reached the end) it asks the
/// Cubit for the next page. Works even when page 1 doesn't fill the screen.
class LoadMoreFooter extends StatefulWidget {
  const LoadMoreFooter({super.key, required this.failed});

  final bool failed;

  @override
  State<LoadMoreFooter> createState() => _LoadMoreFooterState();
}

class _LoadMoreFooterState extends State<LoadMoreFooter> {
  @override
  void initState() {
    super.initState();
    // Not during build: emitting inside initState would call setState in build.
    WidgetsBinding.instance.addPostFrameCallback((_) {
      if (mounted) context.read<UserCubit>().loadMore();
    });
  }

  @override
  Widget build(BuildContext context) => Padding(
    padding: const EdgeInsets.all(16),
    child: Center(
      child: widget.failed
          ? TextButton(
              onPressed: context.read<UserCubit>().loadMore,
              child: const Text('Failed to load. Tap to retry'),
            )
          : const CircularProgressIndicator(),
    ),
  );
}
```

`presentation/pages/user_page.dart`

```dart
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import '../cubit/user_cubit.dart';
import '../cubit/user_state.dart';
import '../widgets/error_view.dart';
import '../widgets/load_more_footer.dart';
import '../widgets/user_tile.dart';

class UserPage extends StatelessWidget {
  const UserPage({super.key});

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(
      title: const Text('Users'),
      actions: [
        IconButton(
          icon: const Icon(Icons.login),
          tooltip: 'Login',
          onPressed: () => Navigator.of(context).pushNamed('/login'),
        ),
      ],
    ),
    body: BlocBuilder<UserCubit, UserState>(
      builder: (context, state) => switch (state) {
        UserInitial() ||
        UserLoading() => const Center(child: CircularProgressIndicator()),
        UserError(:final message) => ErrorView(
          message: message,
          onRetry: context.read<UserCubit>().fetchUsers,
        ),
        UserLoaded() => _UserList(state: state),
      },
    ),
  );
}

class _UserList extends StatelessWidget {
  const _UserList({required this.state});

  final UserLoaded state;

  @override
  Widget build(BuildContext context) {
    final count = state.users.length;
    return ListView.builder(
      itemCount: state.hasReachedMax ? count : count + 1, // +1 = footer
      itemBuilder: (_, index) => index < count
          ? UserTile(user: state.users[index])
          : LoadMoreFooter(failed: state.loadMoreFailed),
    );
  }
}
```

`core/di/injection.dart`: GetIt. `createDio()` is the **one place** Dio is built (options + auth interceptor + debug-only logger); registrations are lazy so their order never matters; the Cubit is a `registerFactory`.

```dart
import 'package:dio/dio.dart';
import 'package:flutter/foundation.dart';
import 'package:flutter_secure_storage/flutter_secure_storage.dart';
import 'package:get_it/get_it.dart';

import '../../feature/auth/auth_injection.dart';
import '../../feature/user/data/data_source/user_remote_data_source.dart';
import '../../feature/user/data/data_source/user_remote_data_source_impl.dart';
import '../../feature/user/data/repository/user_repository_impl.dart';
import '../../feature/user/domain/repository/user_repository.dart';
import '../../feature/user/domain/usecase/get_users.dart';
import '../../feature/user/presentation/cubit/user_cubit.dart';
import '../constants/api_constants.dart';
import '../network/auth_interceptor.dart';
import '../network/dio_client.dart';
import '../network/safe_log_interceptor.dart';
import '../storage/token_storage.dart';

final sl = GetIt.instance;

/// Composition root. Registrations are lazy, so their order never matters.
void initDependencies() {
  sl
    // core
```

```
    ..registerLazySingleton(() => const TokenStorage(FlutterSecureStorage()))
    ..registerLazySingleton<Dio>(() => createDio(sl()))
    ..registerLazySingleton(() => DioClient(sl()))
    // user feature (bottom -> top)
    ..registerLazySingleton<UserRemoteDataSource>(
      () => UserRemoteDataSourceImpl(sl()),
    )
    ..registerLazySingleton<UserRepository>(() => UserRepositoryImpl(sl()))
    ..registerLazySingleton(() => GetUsers(sl()))
    ..registerFactory(() => UserCubit(sl())); // factory: new Cubit per screen

  registerAuthFeature(sl);
}

/// The ONE place Dio is built: base options, auth interceptor, debug logging.
@visibleForTesting
Dio createDio(TokenStorage tokenStorage) {
  final dio = Dio(_baseOptions());
  dio.interceptors.add(
    AuthInterceptor(
      storage: tokenStorage,
      plainDio: Dio(_baseOptions()), // no interceptors: used to refresh + retry
    ),
  );
  if (kDebugMode) {
    dio.interceptors.add(SafeLogInterceptor()); // never in release
  }
  return dio;
}

BaseOptions _baseOptions() => BaseOptions(
  baseUrl: ApiConstants.baseUrl,
  connectTimeout: ApiConstants.timeout,
  receiveTimeout: ApiConstants.timeout,
  headers: {'Accept': 'application/json'},
);
```

`main.dart` : observer, and the two routes (`/` users, `/login`). The users page has a login icon in its AppBar.

```
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import 'core/di/injection.dart';
import 'core/errors/app_bloc_observer.dart';
import 'feature/auth/presentation/cubit/login_cubit.dart';
import 'feature/auth/presentation/pages/login_page.dart';
import 'feature/user/presentation/cubit/user_cubit.dart';
import 'feature/user/presentation/pages/user_page.dart';

void main() {
  initDependencies();
  Bloc.observer = AppBlocObserver();
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) => MaterialApp(
    debugShowCheckedModeBanner: false,
    routes: {
      '/': (_) => BlocProvider(
        create: (_) => sl<UserCubit>()..fetchUsers(),
        child: const UserPage(),
      ),
      '/login': (_) => BlocProvider(
        create: (_) => sl<LoginCubit>(),
        child: const LoginPage(),
      ),
    },
  );
}
```

4C. GetIt or manual DI, decision rule

Situation	Choose
One feature, 20 minutes	**Manual** (composition root in `main.dart`): zero setup, one place, trivial to explain
2+ features, or interviewer asks "how does it scale / test overrides?"	**GetIt** (`registerLazySingleton` for stateless things, `registerFactory` for Cubits)
Large team, many modules	`injectable` + GetIt (codegen); mention only

Manual DI, FULL-tier classes (verified):

```dart
import 'package:dio/dio.dart';
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import '../core/constants/api_constants.dart';
import '../core/network/dio_client.dart';
import '../feature/user/data/data_source/user_remote_data_source_impl.dart';
import '../feature/user/data/repository/user_repository_impl.dart';
import '../feature/user/domain/usecase/get_users.dart';
import '../feature/user/presentation/cubit/user_cubit.dart';
import '../feature/user/presentation/pages/user_page.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    // Composition root: the ONLY place that knows every concrete class.
    final dio = Dio(
      BaseOptions(
        baseUrl: ApiConstants.baseUrl,
        connectTimeout: ApiConstants.timeout,
        receiveTimeout: ApiConstants.timeout,
      ),
    );
    final repository = UserRepositoryImpl(UserRemoteDataSourceImpl(DioClient(dio)));

    return MaterialApp(
      home: BlocProvider(
        create: (_) => UserCubit(GetUsers(repository))..fetchUsers(),
        child: const UserPage(),
      ),
    );
  }
}
```

Wiring order is always bottom-up: `Dio → DioClient → DataSource → Repository → UseCase → Cubit`. If it doesn't compile, you wired it out of order or forgot the interface type on the left (`registerLazySingleton<UserRepository>(...)`).

SECTION 5 — One pattern for every endpoint

```
MODEL  ──►  DATASOURCE  ──►  REPOSITORY  ──►  USECASE  ──►  CUBIT/BLOC  ──►  UI
 (JSON)      (HTTP call)      (map + errors)    (action)       (state)         (draw)
```

Every new endpoint is the **same six steps**. Only the nouns change:

API	Request model	Response model → Entity	DataSource method	Repository / UseCase	Cubit state shape
GET users	none (`page` param)	`UsersResponseModel` → `PagedUsers`	`getUsers(page)`	`getUsers(page)` / `GetUsers`	**Query:** Initial → Loading → Loaded / Error
GET user details	none (`id` in path)	`UserModel` (unwrap `data`) → `User`	`getUser(id)`	`getUser(id)` / `GetUserDetails`	Query
POST login	`LoginRequestModel{email,password}`	`LoginResponseModel{token}` → `AuthSession`	`login(request)`	`login(email,password)` / `Login`	**Command:** Initial → Loading → Success / Error
POST create user	`CreateUserRequestModel{name,job}`	`CreateUserResponseModel{id,createdAt}`	`createUser(request)`	`createUser(...)` / `CreateUser`	Command
PUT update user	same request model (full resource)	none (204/200)	`updateUser(id, request)`	`updateUser` / `UpdateUser`	Command
PATCH update user	only changed fields (`Map` / nullable fields)	none	`patchUser(id, name?, job?)`	`patchUser` / `PatchUser`	Command
DELETE user	none	none (204: body is empty)	`deleteUser(id)`	`deleteUser(id)` / `DeleteUser`	Command (+ remove from the local list on success)

Query vs Command, the two state shapes.

- *Query* (read): the screen **is** the data → `Loading / Loaded / Error` and the UI `switch`es.
- *Command* (write): the screen is a form; the result is a **one-time event** → `Initial / Submitting / Success / Error`. Use `BlocConsumer`: **`listener`** for one-off effects (snackbar, navigation, pop), **`builder`** for the button spinner. Never navigate from `builder`.

Data-source methods for all five verbs (verified; each is 2–3 lines because `DioClient` does the HTTP + error mapping):

```dart
import '../core/constants/api_constants.dart';
import '../core/network/dio_client.dart';
import '../feature/user/data/model/user_model.dart';

// Request models: only toJson. Response models: only fromJson (+ toEntity).
class CreateUserRequestModel {
  const CreateUserRequestModel({required this.name, required this.job});

  final String name;
  final String job;

  Map<String, dynamic> toJson() => {'name': name, 'job': job};
}

class CreateUserResponseModel {
  const CreateUserResponseModel({required this.id, required this.createdAt});

  final String id;
  final DateTime createdAt;

  factory CreateUserResponseModel.fromJson(Map<String, dynamic> json) => CreateUserResponseModel(
    id: json['id'] as String,
    createdAt: DateTime.parse(json['createdAt'] as String),
  );
}

/// One method per endpoint. Every method is 2–3 lines because DioClient does
/// the HTTP + error mapping.
class UserCrudDataSource {
  const UserCrudDataSource(this._client);

  final DioClient _client;

  // GET /users/{id}  -> wrapped as {"data": {...}}
  Future<UserModel> getUser(int id) async {
    final json = await _client.get('${ApiConstants.users}/$id');
    return UserModel.fromJson((json as Map<String, dynamic>)['data']);
  }

  // POST /users
  Future<CreateUserResponseModel> createUser(CreateUserRequestModel request) async {
    final json = await _client.post(ApiConstants.users, data: request.toJson());
    return CreateUserResponseModel.fromJson(json as Map<String, dynamic>);
  }
```

```
  // PUT /users/{id}  (replace whole resource)
  Future<void> updateUser(int id, CreateUserRequestModel request) =>
      _client.put('${ApiConstants.users}/$id', data: request.toJson());

  // PATCH /users/{id}  (send only the changed fields)
  Future<void> patchUser(int id, {String? name, String? job}) =>
      _client.patch('${ApiConstants.users}/$id', data: {'name': ?name, 'job': ?job});

  // DELETE /users/{id}  -> 204 No Content (data is empty)
  Future<void> deleteUser(int id) => _client.delete('${ApiConstants.users}/$id');
}
```

Rules of thumb:

- **Request model →** `toJson` **only. Response model →** `fromJson` **(+** `toEntity` **) only.** Don't write the other direction.
- Repository methods that return nothing return `Future<void>`; the Cubit just emits `Success`.
- **Idempotency:** GET/PUT/DELETE are safe to retry; **POST is not** (retrying can create duplicates). Never auto-retry POST.
- A **PUT** sends the *whole* resource, a **PATCH** only the changed fields (`{'name': ?name}` skips nulls).
- Some APIs wrap data (`{"data": {...}}`): unwrap in the data source/model, never in the Cubit.

SECTION 6 — Complex API response

```
{
  "page": 1,
  "per_page": 10,
  "total": 100,
  "total_pages": 10,
  "data": [ { "id": 1, "first_name": "George" } ],
  "meta": { "request_id": "abc-123", "sorted_by": "id" }
}
```

```
JSON ──► PagedResponseModel<UserModel> ──toEntity(mapItem)──► PagedResult<User> ──► Repository ──► UseCase ──► Cubit
         page, perPage, total,                                  items, page,
         totalPages, data[], meta                               totalPages, hasNext
```

Class	Layer	Contains	Knows JSON?
`UserModel`	data	`first_name`, `avatar`… (API shape)	✅
`PagedResponseModel<T>`	data	**Everything** the API returns incl. `perPage`, `total`, raw `meta`	✅
`User` (entity)	domain	only what screens need	❌
`PagedResult<T>`	domain	`items`, `page`, `totalPages`, `hasNext` (**not** `perPage` / `total` / `meta`)	❌

What belongs in the entity? Only what a screen/rule uses. `perPage` and `request_id` are transport details: they stay in the model and die there. If a screen *does* need `total` ("100 users"), add it to the entity then (YAGNI).

Generic version (verified): use it when the **second** list endpoint appears. For the *first* one, the concrete `UsersResponseModel` / `PagedUsers` from section 4 is faster to type and just as correct:

```dart
import 'package:equatable/equatable.dart';

// ---------- domain (feature-agnostic, reusable for every list endpoint)
class PagedResult<T> extends Equatable {
  const PagedResult({required this.items, required this.page, required this.totalPages});

  final List<T> items;
  final int page;
  final int totalPages;

  bool get hasNext => page < totalPages;

  @override
  List<Object?> get props => [items, page, totalPages];
}

// ---------- data
class PagedResponseModel<T> {
  const PagedResponseModel({
    required this.page,
    required this.perPage,
    required this.total,
    required this.totalPages,
    required this.data,
    this.meta,
  });

  final int page;
  final int perPage;
  final int total;
  final int totalPages;
  final List<T> data;
  final Map<String, dynamic>? meta; // keep raw until you actually need it

  factory PagedResponseModel.fromJson(
    Map<String, dynamic> json,
    T Function(Map<String, dynamic>) fromItem,
  ) => PagedResponseModel(
    page: json['page'] as int,
    perPage: json['per_page'] as int,
    total: json['total'] as int,
    totalPages: json['total_pages'] as int,
    data: (json['data'] as List).map((e) => fromItem(e as Map<String, dynamic>)).toList(),
    meta: json['meta'] as Map<String, dynamic>?,
  );

  /// Item mapping (Model -> Entity) is passed in, so this class stays generic.
  PagedResult<E> toEntity<E>(E Function(T model) mapItem) =>
      PagedResult(items: data.map(mapItem).toList(), page: page, totalPages: totalPages);
}

// ---------- usage inside a data source + repository
```

```
// final model = PagedResponseModel.fromJson(json, UserModel.fromJson);
// return model.toEntity((m) => m.toEntity());
```

Defensive parsing cheat sheet

Situation	Code	Rule
Required field	`json['id'] as int`	Strict: a wrong contract should fail loudly (`TypeError` → `ParsingFailure`)
Optional field	`json['bio'] as String?`	Nullable in the model *and* decide the entity default in `toEntity()`
Missing list	`(json['tags'] as List?) ?? const []`	Default only where the API contract allows absence
Number could be int or double	`(json['price'] as num).toDouble()`	JSON has one number type
Dates	`DateTime.parse(json['createdAt'] as String)` (`tryParse` if optional)	Convert in the model, never in the UI
`id` sometimes string	`int.parse('${json['id']}')`	Normalize at the boundary
snake_case → camelCase	`firstName: json['first_name']`	Naming conversion is a model concern
Nested object	`AddressModel.fromJson(json['address'] as Map<String, dynamic>)`	One model per object
Unknown enum value	`Status.values.asNameMap()[v] ?? Status.unknown`	Never `byName` on server data

Anti-pattern: `json['id'] ?? 0`, `?? ''`. It converts a broken API into a valid-looking blank row. Fail loudly or model the field as nullable on purpose.

SECTION 7 — Pagination

```
Page 1 ──► user scrolls to the end ──► Page 2 ──► APPEND ──► end again ──► Page 3 ──► APPEND ──► … ──► page == totalPages ──► stop
 (full-screen loader)                  (footer loader, list stays)
```

State fields

Field	Type	Meaning
`users`	`List<User>`	Everything loaded so far (**append, never replace**)
`currentPage`	`int`	Last page successfully loaded (next request = `currentPage + 1`)
`totalPages`	`int`	From the API envelope (`total_pages`)
`isLoadingMore`	`bool`	A next-page request is in flight (footer shows a spinner)
`hasReachedMax`	`bool` getter	`currentPage >= totalPages`: stop asking
`loadMoreFailed`	`bool`	Page N+1 failed; keep the list, show *Retry* in the footer

(LITE stores `hasNext` instead of `totalPages` / `hasReachedMax`. Same idea, one field less.)

Initial loading vs pagination loading

	Initial loading	Pagination loading
State	`UserLoading` (a **separate state**)	`UserLoaded(isLoadingMore: true)` (**same state**, flag flipped)
Data on screen	none	the whole list stays
UI	Full-screen spinner	Spinner as the **last list item**
On failure	`UserError` + Retry button (full screen)	List stays; footer shows "*Failed to load. Tap to retry*"
Why	Nothing to show yet	Replacing the list with a spinner makes the scroll position jump and loses context

The three guards in `loadMore()` (this is the whole trick)

```dart
if (current is! UserLoaded ||      // not loaded yet / error → nothing to append to
    current.isLoadingMore ||       // a request is already running → ignore duplicate scroll events
    current.hasReachedMax) {       // no more pages
  return;
}
```

The UI may call it 100 times; the Cubit lets exactly one through. In **Bloc**, the same guarantee is one line: `on<NextPage>(_onNextPage, transformer: droppable())`.

Also: `if (isClosed) return;` after every `await` (user left the screen), and `emit(current.copyWith(...))` (never rebuild the object by hand: you will forget a field).

Stale responses. `_generation` is bumped by every fresh `fetchUsers()`. A `loadMore` (or an older `fetchUsers`) that finishes after a newer reload compares generations and **drops its result**, otherwise page 2 of the *old* list would be appended to the *new* one. Tested with `Completer`s in `user_cubit_test.dart`.

Two ways to trigger "load more"

	Footer trigger (used in this guide)	`ScrollController` threshold
How	Last list item's `initState` / `itemBuilder` asks for the next page	Listener checks `pixels >= maxScrollExtent - 200`
Short first page (6 items on a tall phone)	✅ Works (footer is visible, so it loads)	❌ **Nothing to scroll → never loads page 2**
Typing	~8 lines	~15 lines + `dispose`
Familiarity to interviewers	Medium	High: be able to write it

ScrollController alternative (verified):

```dart
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import '../feature/user/presentation/cubit/user_cubit.dart';

/// ALTERNATIVE to the footer trigger: classic ScrollController threshold.
/// Weakness: if page 1 does not fill the screen there is nothing to scroll,
/// so page 2 is never requested.
```

```dart
class UserListWithController extends StatefulWidget {
  const UserListWithController({super.key});

  @override
  State<UserListWithController> createState() => _UserListWithControllerState();
}

class _UserListWithControllerState extends State<UserListWithController> {
  final _controller = ScrollController();

  @override
  void initState() {
    super.initState();
    _controller.addListener(() {
      final nearBottom = _controller.position.pixels >= _controller.position.maxScrollExtent - 200;
      if (nearBottom) context.read<UserCubit>().loadMore(); // Cubit ignores duplicates
    });
  }

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) =>
      ListView.builder(controller: _controller, itemBuilder: (_, i) => Text('$i'));
}
```

Pull-to-refresh

`RefreshIndicator(onRefresh: () => cubit.fetchUsers(), …)` . In the FULL Cubit `fetchUsers()` emits `UserLoading`, which swaps the list for a spinner mid-pull. For a polished refresh, add a `refresh()` that *doesn't* emit `UserLoading` and replaces the list on success. Mention it; don't build it unless asked.

Pagination types (know the vocabulary)

Type	Request	Pros / cons
Page-number (`?page=2`) (reqres)	page index	Simple; unstable if items are inserted while paging
Offset/limit (`?offset=20&limit=20`)	skip N	Same instability
Cursor (`?after=<token>`)	opaque token from last response	Stable and scales; store `nextCursor` instead of `currentPage`

SECTION 8 — Search

```
User types ──► TextField.onChanged ──► DEBOUNCE (wait 400 ms of silence) ──► Cubit.fetchUsers(query)
              UI ◄── state ◄── Cubit (ignores STALE responses) ◄── API `GET /users?q=…`
```

Client-side vs server-side

	Client-side	Server-side
Use when	The **complete** dataset is already loaded (≤ a few hundred rows, no pagination)	Data is paginated / large / changes on the server
Cost	Zero network, instant	One request per (debounced) change
Debounce needed?	No	**Yes**, otherwise one request per keystroke
Correctness trap	With pagination it only searches the *loaded pages* → misleading	Race conditions: slow old response overwrites newer one
Where	Widget / Cubit filters `state.users`	Repository → DataSource sends `?q=`

Client-side (verified), no Cubit/API change:

```dart
import 'package:flutter/material.dart';

import '../feature/user/domain/entity/user.dart';
import '../feature/user/presentation/widgets/user_tile.dart';

/// CLIENT-SIDE search: the full list is already in memory -> filter it.
/// No API call, no debounce needed, no Cubit change.
class FilteredUserList extends StatefulWidget {
  const FilteredUserList({super.key, required this.users});

  final List<User> users;

  @override
  State<FilteredUserList> createState() => _FilteredUserListState();
}

class _FilteredUserListState extends State<FilteredUserList> {
  String _query = '';

  @override
  Widget build(BuildContext context) {
    final q = _query.toLowerCase();
    final visible = widget.users
        .where((u) => u.fullName.toLowerCase().contains(q) || u.email.toLowerCase().contains(q))
        .toList();

    return Column(
      children: [
        TextField(onChanged: (v) => setState(() => _query = v)),
        Expanded(
          child: ListView.builder(
            itemCount: visible.length,
            itemBuilder: (_, i) => UserTile(user: visible[i]),
          ),
        ),
      ],
    );
  }
}
```

Server-side, exactly which lines change (verified)

`reqres` has no `q` param, so the request works but the filter is ignored; the plumbing is what you'd show. Changes, **bottom to top**:

File	Change
`user_remote_data_source.dart`	`getUsers({required int page, String query = ''})`
`user_remote_data_source_impl.dart`	`query: {'page': page, if (query.isNotEmpty) 'q': query}`
`user_repository.dart` / `_impl.dart` / `get_users.dart`	add `String query = ''`, pass it down
`user_cubit.dart`	store `_query`; `fetchUsers({String? query})`; pass `query` in `loadMore`; **drop stale responses** (below)
`user_search_field.dart` + `user_page.dart`	`TextField` with `Debouncer`, placed **outside** the `BlocBuilder`

`core/utils/debouncer.dart`

```dart
import 'dart:async';

/// Runs only the LAST call after [delay] of silence (typing -> one API call).
class Debouncer {
  Debouncer(this.delay);

  final Duration delay;
  Timer? _timer;

  void run(void Function() action) {
    _timer?.cancel();
    _timer = Timer(delay, action);
  }

  void dispose() => _timer?.cancel();
}
```

`user_cubit.dart` with search + stale-response protection

```dart
import 'package:flutter_bloc/flutter_bloc.dart';

import '../../../../core/errors/failures.dart';
import '../../domain/usecase/get_users.dart';
import 'user_state.dart';

class UserCubit extends Cubit<UserState> {
  UserCubit(this._getUsers) : super(const UserInitial());

  final GetUsers _getUsers;
  String _query = '';

  /// First page. Pass [query] to search; omit it to reuse the current query
  /// (that is what the Retry button does).
  Future<void> fetchUsers({String? query}) async {
    _query = query?.trim() ?? _query;
    final requested = _query;
    emit(const UserLoading());
    try {
      final result = await _getUsers(page: 1, query: requested);
      if (isClosed || requested != _query) return; // stale: user typed again
      emit(
        UserLoaded(
          users: result.users,
          currentPage: result.page,
          totalPages: result.totalPages,
        ),
      );
    } on Failure catch (failure) {
      if (isClosed || requested != _query) return;
      emit(UserError(failure.message));
    }
  }

  Future<void> loadMore() async {
    final current = state;
    if (current is! UserLoaded ||
        current.isLoadingMore ||
        current.hasReachedMax) {
      return;
    }

    final requested = _query;
    emit(current.copyWith(isLoadingMore: true, loadMoreFailed: false));
    try {
      final result = await _getUsers(
        page: current.currentPage + 1,
        query: requested,
      );
      if (isClosed || requested != _query) return;
      emit(
        current.copyWith(
          users: [...current.users, ...result.users],
          currentPage: result.page,
          totalPages: result.totalPages,
          isLoadingMore: false,
        ),
      );
    } on Failure {
      if (isClosed || requested != _query) return;
      emit(current.copyWith(isLoadingMore: false, loadMoreFailed: true));
    }
```

```
  }
}
```

`widgets/user_search_field.dart`

```dart
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import '../../../../core/utils/debouncer.dart';
import '../cubit/user_cubit.dart';

class UserSearchField extends StatefulWidget {
  const UserSearchField({super.key});

  @override
  State<UserSearchField> createState() => _UserSearchFieldState();
}

class _UserSearchFieldState extends State<UserSearchField> {
  final _debouncer = Debouncer(const Duration(milliseconds: 400));

  @override
  void dispose() {
    _debouncer.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) => Padding(
    padding: const EdgeInsets.all(12),
    child: TextField(
      decoration: const InputDecoration(
        hintText: 'Search users',
        prefixIcon: Icon(Icons.search),
        border: OutlineInputBorder(),
      ),
      onChanged: (value) => _debouncer.run(
        () => context.read<UserCubit>().fetchUsers(query: value),
      ),
    ),
  );
}
```

`pages/user_page.dart` : search field sits *outside* `BlocBuilder` so it keeps focus while the list shows a spinner.

```dart
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import '../cubit/user_cubit.dart';
import '../cubit/user_state.dart';
import '../widgets/error_view.dart';
import '../widgets/load_more_footer.dart';
import '../widgets/user_search_field.dart';
import '../widgets/user_tile.dart';

class UserPage extends StatelessWidget {
  const UserPage({super.key});

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Users')),
    body: Column(
      children: [
        const UserSearchField(), // outside BlocBuilder: keeps focus while loading
        Expanded(
          child: BlocBuilder<UserCubit, UserState>(
            builder: (context, state) => switch (state) {
              UserInitial() ||
              UserLoading() => const Center(child: CircularProgressIndicator()),
              UserError(:final message) => ErrorView(
                message: message,
                onRetry: context.read<UserCubit>().fetchUsers,
              ),
              UserLoaded() => _UserList(state: state),
            },
          ),
        ),
      ],
    ),
  );
}

class _UserList extends StatelessWidget {
  const _UserList({required this.state});
```

```
  final UserLoaded state;

  @override
  Widget build(BuildContext context) {
    final count = state.users.length;
    return ListView.builder(
      itemCount: state.hasReachedMax ? count : count + 1, // +1 = footer
      itemBuilder: (_, index) => index < count
          ? UserTile(user: state.users[index])
          : LoadMoreFooter(failed: state.loadMoreFailed),
    );
  }
}
```

The stale-response race (say this): user types "ab" (request A, slow) then "abc" (request B, fast). B returns first and renders; then A returns and **overwrites** B with results for "ab". The fix is the `requested != _query` check: each response is only applied if it still matches the current query. In **Bloc**: `on<Search>(…, transformer: restartable())` cancels the previous handler automatically. With Dio you can additionally `CancelToken.cancel()` the old HTTP call (section 20).

SECTION 9 — POST / Login

```
LoginPage ──► LoginCubit ──► Login (use case) ──► AuthRepository ──► AuthRemoteDataSource ──► DioClient ──► POST /login
 (form)       (Command        (action)            (impl: save token     (request model →          (maps errors)
               state)                              + guard)              JSON, JSON → response)
```

Request vs response model

	Request model	Response model
Purpose	Shape of the **body we send**	Shape of the **body we receive**
Direction	`toJson()` only	`fromJson()` + `toEntity()` only
Example	`{ "email": "...", "password": "..." }`	`{ "token": "QpwL5tke4Pnpja7X4" }`
Entity	none	`AuthSession(token)` (what the app needs: not the raw envelope)

Test creds for reqres: `eve.holt@reqres.in` / `cityslicka` → `200 {"token": …}`. A missing password → `400 {"error":"Missing password"}`, which flows through `DioClient._bodyMessage` to the snackbar unchanged.

`domain/entity/auth_session.dart`

```dart
import 'package:equatable/equatable.dart';

class AuthSession extends Equatable {
  const AuthSession({required this.token});

  final String token;

  @override
  List<Object?> get props => [token];
}
```

`domain/repository/auth_repository.dart`

```dart
import '../entity/auth_session.dart';

abstract interface class AuthRepository {
  Future<AuthSession> login({required String email, required String password});
}
```

`domain/usecase/login.dart`

```dart
import '../entity/auth_session.dart';
import '../repository/auth_repository.dart';

class Login {
  const Login(this._repository);

  final AuthRepository _repository;

  Future<AuthSession> call({required String email, required String password}) =>
      _repository.login(email: email, password: password);
}
```

`data/model/login_request_model.dart` and `login_response_model.dart`

```dart
class LoginRequestModel {
  const LoginRequestModel({required this.email, required this.password});

  final String email;
  final String password;

  Map<String, dynamic> toJson() => {'email': email, 'password': password};
}
```

```dart
import '../../domain/entity/auth_session.dart';

class LoginResponseModel {
  const LoginResponseModel({required this.token});

  final String token;

  factory LoginResponseModel.fromJson(Map<String, dynamic> json) =>
      LoginResponseModel(token: json['token'] as String);

  AuthSession toEntity() => AuthSession(token: token);
}
```

`data/data_source/auth_remote_data_source.dart` (interface + impl together)

```dart
import '../../../../core/constants/api_constants.dart';
import '../../../../core/network/dio_client.dart';
import '../model/login_request_model.dart';
import '../model/login_response_model.dart';

// Interface + impl in one file = acceptable interview shortcut.
abstract interface class AuthRemoteDataSource {
  Future<LoginResponseModel> login(LoginRequestModel request);
}

class AuthRemoteDataSourceImpl implements AuthRemoteDataSource {
  const AuthRemoteDataSourceImpl(this._client);

  final DioClient _client;

  @override
  Future<LoginResponseModel> login(LoginRequestModel request) async {
    final json = await _client.post(ApiConstants.login, data: request.toJson());
    return LoginResponseModel.fromJson(json as Map<String, dynamic>);
  }
}
```

`data/repository/auth_repository_impl.dart` : the repository also persists the token (a *data* concern: the Cubit never sees it).

```dart
import '../../../../core/errors/failures.dart';
import '../../../../core/storage/token_storage.dart';
import '../../domain/entity/auth_session.dart';
import '../../domain/repository/auth_repository.dart';
import '../data_source/auth_remote_data_source.dart';
import '../model/login_request_model.dart';

class AuthRepositoryImpl implements AuthRepository {
  const AuthRepositoryImpl(this._remote, this._tokenStorage);

  final AuthRemoteDataSource _remote;
  final TokenStorage _tokenStorage;

  @override
  Future<AuthSession> login({
    required String email,
    required String password,
  }) => guard(() async {
    final response = await _remote.login(
      LoginRequestModel(email: email, password: password),
    );
    await _tokenStorage.save(access: response.token);
    return response.toEntity();
  });
}
```

`presentation/cubit/login_state.dart` and `login_cubit.dart` : a *Command* state shape.

```dart
import 'package:equatable/equatable.dart';

sealed class LoginState extends Equatable {
  const LoginState();

  @override
  List<Object?> get props => [];
}

class LoginInitial extends LoginState {
  const LoginInitial();
}

class LoginLoading extends LoginState {
  const LoginLoading();
}

class LoginSuccess extends LoginState {
  const LoginSuccess();
}

class LoginError extends LoginState {
  const LoginError(this.message);

  final String message;

  @override
  List<Object?> get props => [message];
}
```

```dart
import 'package:flutter_bloc/flutter_bloc.dart';

import '../../../../core/errors/failures.dart';
import '../../../../core/errors/report_failure.dart';
import '../../domain/usecase/login.dart';
import 'login_state.dart';

class LoginCubit extends Cubit<LoginState> with FailureReporting<LoginState> {
  LoginCubit(this._login) : super(const LoginInitial());

  final Login _login;

  Future<void> login({required String email, required String password}) async {
    if (state is LoginLoading) return; // ignore double taps
    emit(const LoginLoading());
    try {
      await _login(email: email, password: password);
      if (!isClosed) emit(const LoginSuccess());
    } on Failure catch (failure) {
      reportFailure(failure);
      if (!isClosed) emit(LoginError(failure.message));
    }
  }
}
```

`presentation/pages/login_page.dart` : `BlocConsumer` : **listener** = snackbar/navigation, **builder** = button spinner.

```dart
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import '../cubit/login_cubit.dart';
import '../cubit/login_state.dart';

class LoginPage extends StatefulWidget {
  const LoginPage({super.key});

  @override
  State<LoginPage> createState() => _LoginPageState();
}

class _LoginPageState extends State<LoginPage> {
  final _email = TextEditingController();
  final _password = TextEditingController();

  @override
  void dispose() {
    _email.dispose();
    _password.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Login')),
    body: BlocConsumer<LoginCubit, LoginState>(
      // listener = one-off side effects (snackbar, navigation)
      listener: (context, state) {
        if (state is LoginError) {
          ScaffoldMessenger.of(context)
              .showSnackBar(SnackBar(content: Text(state.message)));
        } else if (state is LoginSuccess) {
          // The messenger outlives this route, so the snackbar shows on the
          // screen we return to.
          ScaffoldMessenger.of(context)
              .showSnackBar(const SnackBar(content: Text('Logged in')));
          Navigator.of(context).pop();
        }
      },
      // builder = what is drawn for the current state
      builder: (context, state) => Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          children: [
            TextField(
              controller: _email,
              decoration: const InputDecoration(labelText: 'Email'),
            ),
            TextField(
              controller: _password,
              obscureText: true,
              decoration: const InputDecoration(labelText: 'Password'),
            ),
            const SizedBox(height: 24),
            FilledButton(
```

```
              onPressed: state is LoginLoading
                  ? null
                  : () => context.read<LoginCubit>().login(
                      email: _email.text.trim(),
                      password: _password.text,
                    ),
              child: state is LoginLoading
                  ? const SizedBox.square(
                      dimension: 20,
                      child: CircularProgressIndicator(strokeWidth: 2),
                    )
                  : const Text('Login'),
            ),
          ],
        ),
      ),
    ),
  );
}
```

`feature/auth/auth_injection.dart` : per-feature DI module: registers only this feature's classes. It needs `DioClient` and `TokenStorage` from core, which are registered lazily, so call order doesn't matter.

```
import 'package:get_it/get_it.dart';

import 'data/data_source/auth_remote_data_source.dart';
import 'data/repository/auth_repository_impl.dart';
import 'domain/repository/auth_repository.dart';
import 'domain/usecase/login.dart';
import 'presentation/cubit/login_cubit.dart';

/// Per-feature DI module: registers only this feature's classes. It needs
/// DioClient and TokenStorage from core, which are registered lazily in
/// `initDependencies()`, so call order doesn't matter.
void registerAuthFeature(GetIt sl) {
  sl
    ..registerLazySingleton<AuthRemoteDataSource>(
      () => AuthRemoteDataSourceImpl(sl()),
    )
    ..registerLazySingleton<AuthRepository>(
      () => AuthRepositoryImpl(sl(), sl()),
    )
    ..registerLazySingleton(() => Login(sl()))
    ..registerFactory(() => LoginCubit(sl()));
}
```

Mounted in `main.dart` as the `/login` route (`BlocProvider(create: (_) => sl<LoginCubit>(), child: const LoginPage())`); the users page opens it from an AppBar icon, and on success the page pops with a *Logged in* snackbar.

Details worth saying: `if (state is LoginLoading) return;` ignores double-taps; passwords are never logged or stored; form validation (non-empty / contains `@`) can live in a use case (section 16) or in the form.

SECTION 10 — Error handling

```
Dio throws DioException / parsing throws TypeError, FormatException
        │  (DioClient._toException — ONE place)
        ▼
AppException        data layer vocabulary:  ServerException(status) · NetworkException · ParsingException
        │  (guard() in the repository — ONE place)
        ▼
Failure             domain/UI vocabulary:   Unauthorized · Validation · Server · Network · Parsing · Unknown
        │  (Cubit: on Failure catch (f))
        ▼
Error State         UserError(f.message)  →  ErrorView + Retry   /   LoginError → SnackBar
```

Case	Detected as	`AppException`	`Failure`	User sees
400	`badResponse`	`ServerException(400)`	`ServerFailure`	server's `error` / `message` or "Bad request."
401	`badResponse`	`ServerException(401)`	`UnauthorizedFailure`	"Please login again." → refresh flow / go to login
403	`badResponse`	`ServerException(403)`	`UnauthorizedFailure`	"You do not have access."
404	`badResponse`	`ServerException(404)`	`ServerFailure`	"Not found."
422	`badResponse`	`ServerException(422)`	`ValidationFailure`	server's validation message, shown by the form
500+	`badResponse`	`ServerException(5xx)`	`ServerFailure`	"Server error. Please try later."
Timeout	`connection/send/receiveTimeout`	`NetworkException`	`NetworkFailure`	"Request timed out. Please try again."
No internet	`connectionError`	`NetworkException`	`NetworkFailure`	"No internet connection."
Invalid JSON (not parseable)	`unknown` + `FormatException` (Dio wraps it)	`ParsingException`	`ParsingFailure`	"Unexpected data received from server."
Wrong JSON shape (null / wrong type)	`TypeError` from `fromJson`	none (caught in `guard`)	`ParsingFailure`	same
Cancelled	`DioExceptionType.cancel`	`CancelledException`	`CancelledFailure`	nothing: callers ignore it
Anything else	any other throw	none	`UnknownFailure`	"Something went wrong. Please try again."

All of this is in `exceptions.dart`, `failures.dart` and `dio_client.dart` (section 4B), tested in `dio_client_test.dart` and `user_repository_impl_test.dart` (section 17).

Where each conversion happens, and why exactly once:

Conversion	Location	Why there
`DioException` → `AppException`	`DioClient._toException`	Only Dio-aware code touches Dio's types; swap `dio` for `http` and only this file changes
`AppException` / `TypeError` → `Failure`	`guard()` used by every repository	One mapping for the whole app, repositories stay 3 lines
`Failure` → state/message	Cubit	Cubit decides what the screen shows

Rules

1. **Messages are user-safe.** Never show `e.toString()`, stack traces, or URLs. Log those, show friendly text.
2. The Cubit catches `on Failure`, not `catch (e)`, so real bugs still crash into your error reporter instead of showing "Something went wrong" forever.
3. Show errors where they belong: *Query* → `ErrorView` + Retry. *Command* → SnackBar via `BlocListener` / `BlocConsumer`.
4. **Pagination errors keep the list** (`loadMoreFailed`).
5. A sealed `Failure` lets one exhaustive `switch` decide UI per kind (verified):

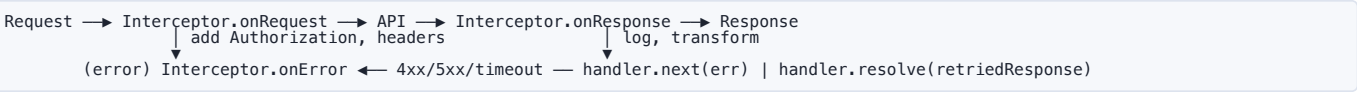
```dart
import '../core/errors/failures.dart';

/// Sealed Failure => the compiler forces you to handle every kind.
/// Add a new Failure subclass and this stops compiling until you handle it.
({String title, bool canRetry}) describe(Failure failure) => switch (failure) {
  UnauthorizedFailure() => (title: 'Session expired. Please login.', canRetry: false),
  NetworkFailure() => (title: 'You are offline.', canRetry: true),
  ValidationFailure() => (title: 'Please check your input.', canRetry: false),
```

```
  ServerFailure() => (title: 'Server problem. Try later.', canRetry: true),
  CancelledFailure() => (title: '', canRetry: false), // ignored: show nothing
  ParsingFailure() || UnknownFailure() => (title: 'Something went wrong.', canRetry: true),
};
```

6. `UnauthorizedFailure` is special: don't just show text: trigger the refresh (section 13) or logout (section 12).
7. **Nothing is swallowed.** `UnknownFailure` / `ParsingFailure` keep the original `cause` + `stackTrace`; the Cubit calls `reportFailure(failure)` (section 4B) so they reach `Bloc.observer.onError`, while the user only sees the safe message.

SECTION 11 — Dio interceptors



Three handlers per interceptor, each **must** call exactly one of `handler.next(...)`, `handler.resolve(...)` or `handler.reject(...)`, otherwise the request hangs forever.

Job	Where	Notes
Common headers (`Accept`, `User-Agent`, app version, locale)	`BaseOptions.headers`, not an interceptor	Static, so no code to run
Authorization token	`onRequest` of `AuthInterceptor` (section 13)	Reads the token from secure storage per request, so token changes apply immediately
Logging	A small interceptor, **debug builds only**	`if (kDebugMode) SafeLogInterceptor()`
Refresh token / retry	`onError` on 401 (section 13)	Use a *separate plain Dio* for the refresh call
Error mapping	❌ not an interceptor in this design	Kept in `DioClient`, simpler to explain and test

Requests pass through interceptors in the order you add them: add `AuthInterceptor` **before** a logger.

Safe logger (verified): method, URL, status only:

```dart
import 'package:dio/dio.dart';
import 'package:flutter/foundation.dart';

/// Logs ONLY method, url, status. Never headers (Authorization), never bodies
/// (passwords, tokens, PII). Register it only in debug builds.
class SafeLogInterceptor extends Interceptor {
  @override
  void onRequest(RequestOptions options, RequestInterceptorHandler handler) {
    debugPrint('-> ${options.method} ${options.uri}');
    handler.next(options);
  }

  @override
  void onResponse(
    Response<dynamic> response,
    ResponseInterceptorHandler handler,
  ) {
    debugPrint('<- ${response.statusCode} ${response.requestOptions.uri}');
    handler.next(response);
  }

  @override
  void onError(DioException err, ErrorInterceptorHandler handler) {
    debugPrint(
      'x  ${err.response?.statusCode} ${err.requestOptions.uri} ${err.type.name}',
    );
    handler.next(err);
  }
}
```

What must NOT be logged in production

Never log	Why
`Authorization` header / access & refresh tokens	Anyone with logs (device, crash reporter, CI) can impersonate the user
Request bodies of `/login`, `/register`, `/password`	Passwords
Response bodies containing tokens or PII (email, phone, address, health, payment)	Privacy / GDPR / compliance
Full bodies at all in release	Size + leak risk + performance
Anything under `release` mode	Wrap in `kDebugMode`; Dio's built-in `LogInterceptor` prints headers by default: use `requestHeader: false` or avoid it

If you must log bodies for debugging, **redact** keys (`password`, `token`, `authorization`) first.

SECTION 12 — Authentication

```
Login ⟶ Access Token (+ Refresh Token) ⟶ Secure Storage ⟶ Interceptor reads it ⟶ "Authorization: Bearer <access>" on every request
```

	Access token	Refresh token
Lifetime	Short (minutes to ~1 h)	Long (days–weeks)
Sent with	**Every** API request	**Only** to the refresh endpoint
If stolen	Limited damage window	Serious: can mint new access tokens
Storage	Secure storage (or memory)	**Secure storage only**
Revocable	Expires by itself	Server can revoke (logout/rotation)

Where it lives: Keychain (iOS) / Keystore (Android) via `flutter_secure_storage`. **Never** `SharedPreferences` or a file: those are plain text.

`core/storage/token_storage.dart` (verified)

```dart
import 'package:flutter/services.dart';
import 'package:flutter_secure_storage/flutter_secure_storage.dart';

/// Keychain (iOS) / Keystore (Android). Never SharedPreferences for tokens.
class TokenStorage {
  const TokenStorage(this._storage);

  final FlutterSecureStorage _storage;

  static const _accessKey = 'access_token';
  static const _refreshKey = 'refresh_token';

  Future<String?> readAccess() => _read(_accessKey);
  Future<String?> readRefresh() => _read(_refreshKey);

  /// Keychain unavailable (e.g. missing entitlement) = "not logged in",
  /// never a crash inside every request.
  Future<String?> _read(String key) async {
    try {
      return await _storage.read(key: key);
    } on PlatformException {
      return null;
    }
  }

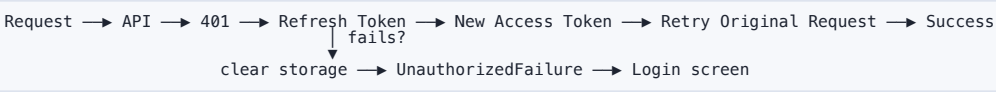
  Future<void> save({required String access, String? refresh}) async {
    await _storage.write(key: _accessKey, value: access);
    if (refresh != null) await _storage.write(key: _refreshKey, value: refresh);
  }

  Future<void> clear() async {
    await _storage.delete(key: _accessKey);
    await _storage.delete(key: _refreshKey);
  }
}
```

The rules

1. **Only the interceptor attaches the token.** Repositories, Cubits and widgets never touch it.
2. The **repository** saves the token after login (a data concern), as in `AuthRepositoryImpl`.
3. **App start:** read the token; present → go Home, absent → Login (a tiny `AuthCubit` /splash decides).
4. **Logout:** `storage.clear()` + reset in-memory state (and any caches) + navigate to Login. Also call the server's logout/revoke endpoint if it exists.
5. **Session expired** (refresh failed): interceptor clears storage and lets the `UnauthorizedFailure` through; one app-level listener (e.g. `BlocListener<AuthCubit>`) navigates to Login. Avoid navigating from inside repositories.
6. 401 means *unauthenticated* (token missing/expired); 403 means *authenticated but not allowed*. Only 401 triggers refresh.

SECTION 13 — Retry & Refresh token



The 20-minute version (say it; code only if asked, ≈12 lines, verified)

```dart
import 'package:dio/dio.dart';

import '../core/storage/token_storage.dart';

/// 12-line interview version: 401 -> refresh -> retry once. No parallel-request
/// handling (mention it verbally: "in production I'd share one refresh Future").
class LiteRefreshInterceptor extends Interceptor {
  LiteRefreshInterceptor(this.storage, this.plainDio);

  final TokenStorage storage;
  final Dio plainDio; // Dio WITHOUT this interceptor

  @override
  Future<void> onError(DioException err, ErrorInterceptorHandler handler) async {
    if (err.response?.statusCode != 401) return handler.next(err);
    try {
      final res = await plainDio.post<Map<String, dynamic>>(
        '/auth/refresh',
        data: {'refresh_token': await storage.readRefresh()},
      );
      final access = res.data!['access_token'] as String;
      await storage.save(access: access);
      final retry = err.requestOptions..headers['Authorization'] = 'Bearer $access';
      handler.resolve(await plainDio.fetch<dynamic>(retry));
    } catch (_) {
      handler.next(err);
    }
  }
}
```

Then say: "*In production I also de-duplicate parallel refreshes.*"

The production version (verified)

```dart
import 'package:dio/dio.dart';

import '../storage/token_storage.dart';

/// 1. onRequest: attach the `Authorization: Bearer` header.
/// 2. onError 401: refresh once, retry the original request with the new token.
class AuthInterceptor extends Interceptor {
  AuthInterceptor({required this.storage, required this.plainDio});

  final TokenStorage storage;

  /// Dio WITHOUT this interceptor (same baseUrl) — used for refresh + retry,
  /// so a failing refresh can never trigger another refresh (infinite loop).
  final Dio plainDio;

  Future<String?>? _refreshing; // all parallel 401s share ONE refresh call

  @override
  Future<void> onRequest(
    RequestOptions options,
    RequestInterceptorHandler handler,
  ) async {
    final token = await storage.readAccess();
    if (token != null) options.headers['Authorization'] = 'Bearer $token';
    handler.next(options);
  }

  @override
  Future<void> onError(
    DioException err,
    ErrorInterceptorHandler handler,
  ) async {
    if (err.response?.statusCode != 401) return handler.next(err);

    final newToken = await (_refreshing ??= _refresh().whenComplete(
      () => _refreshing = null,
    ));
    if (newToken == null) return handler.next(err); // refresh failed -> logout

    try {
```

```
      final retry = err.requestOptions
        ..headers['Authorization'] = 'Bearer $newToken';
      handler.resolve(await plainDio.fetch<dynamic>(retry));
    } on DioException catch (e) {
      handler.next(e);
    }
  }

  Future<String?> _refresh() async {
    final refreshToken = await storage.readRefresh();
    if (refreshToken == null) return null;
    try {
      final response = await plainDio.post<Map<String, dynamic>>(
        '/auth/refresh', // your backend's refresh endpoint
        data: {'refresh_token': refreshToken},
      );
      final access = response.data!['access_token'] as String;
      await storage.save(
        access: access,
        refresh: response.data!['refresh_token'] as String?,
      );
      return access;
    } catch (_) {
      await storage.clear(); // refresh token dead -> user must login again
      return null;
    }
  }
}
```

`createDio()` in `core/di/injection.dart` (section 4B) attaches it when Dio is built, so there is no ordering to get wrong. Reqres has no refresh endpoint; `/auth/refresh` is the template you'd point at your backend, and the fake-HTTP tests in section 17 prove the flow.

The four pitfalls this code avoids (they *will* ask)

Pitfall	Symptom	Fix used above
Refresh call goes through the same interceptor	401 on `/refresh` triggers another refresh → **infinite loop**	`plainDio` (same `baseUrl`, **no** auth interceptor) for refresh *and* the retry
`QueuedInterceptor` + retrying on the **same** Dio	**Deadlock**: the retry waits in the queue behind the handler that is waiting for the retry	Plain `Interceptor` + `plainDio` for retry
5 screens fire 5 requests, all get 401	5 refresh calls; with refresh-token rotation, 4 of them **invalidate each other**	`_refreshing ??=` shares **one** in-flight refresh `Future`
Retry forever	Endless loop on a truly invalid session	Retry **once** (the retry uses `plainDio`, so it can't re-enter the interceptor); failed refresh → `storage.clear()`

Retry for flaky networks (different from 401 refresh)

Retry **idempotent reads only** (GET), **only** network failures (not 4xx), with backoff. Never auto-retry POST. Verified helper:

```
import '../core/errors/exceptions.dart';

/// Retry ONLY idempotent calls (GET). Never blindly retry POST (double charge!).
/// Retries network problems, not 4xx (the same request will fail the same way).
Future<T> retry<T>(Future<T> Function() action, {int maxAttempts = 3}) async {
  for (var attempt = 1; ; attempt++) {
    try {
      return await action();
    } on NetworkException {
      if (attempt >= maxAttempts) rethrow;
      await Future<void>.delayed(Duration(milliseconds: 300 * attempt)); // backoff
    }
  }
}

// In the data source:
//   final json = await retry(() => _client.get(ApiConstants.users, query: {...}));
```

SECTION 14 — Cubit vs BLoC

	Cubit	BLoC
Input	**Methods**: `cubit.loadMore()`	**Events**: `bloc.add(UsersNextPageRequested())`
Mental model	"Call a function, state changes"	"Something *happened*; map it to state"
Boilerplate	Less (no event classes)	More (event sealed class + `on<E>` handlers)
Traceability	You see state transitions	You see **event** → **state** transitions (`onTransition` , great for logging/analytics/replay)
Concurrency control	Manual guards (`if (isLoadingMore) return`)	**Transformers**: `droppable()` , `restartable()` , `sequential()` , `debounce` (via `bloc_concurrency` / `stream_transform`)
Live-coding speed	**Faster**	Slower (~+25% typing)
Explain-ability	Simplest	Needs a sentence on events

When to choose Cubit

Request/response screens: lists, details, login, forms, settings. One action ≈ one method. *Examples:* user list with pagination, login, profile, toggling a favorite.

When to choose BLoC

- **Many overlapping triggers** that must be coordinated (search typing + filters + pagination + pull-to-refresh) → `restartable()` for search, `droppable()` for scroll.
- You need an **event log** (analytics, undo/redo, debugging "what sequence led here?").
- **Streams as input**: websocket messages, location updates, debounced text events.
- The interviewer says "use Bloc". *Examples:* chat (messages in + typing + presence events), search-as-you-type with filters, cart with undo, checkout wizard.

Interview line: "*I'll start with Cubit for speed and clarity; if concurrency between triggers becomes the problem, that's exactly what Bloc's event transformers solve, and the state, use case, and UI barely change.*"

Same feature as a BLoC (verified)

Same `UserState` , same `GetUsers` ; only the input changes and the manual guard becomes a transformer:

```dart
import 'package:bloc_concurrency/bloc_concurrency.dart';
import 'package:equatable/equatable.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import '../core/errors/failures.dart';
import '../feature/user/domain/usecase/get_users.dart';
import '../feature/user/presentation/cubit/user_state.dart';

// ---------- events: WHAT HAPPENED (not what to do)
sealed class UserEvent extends Equatable {
  const UserEvent();

  @override
  List<Object?> get props => [];
}

class UsersStarted extends UserEvent {
  const UsersStarted();
}

class UsersNextPageRequested extends UserEvent {
  const UsersNextPageRequested();
}

// ---------- same states, same use case as the Cubit version
class UserBloc extends Bloc<UserEvent, UserState> {
  UserBloc(this._getUsers) : super(const UserInitial()) {
    on<UsersStarted>(_onStarted, transformer: restartable());
    // droppable(): ignore taps/scroll events while one is already running.
    // This replaces the manual `if (isLoadingMore) return;` guard.
    on<UsersNextPageRequested>(_onNextPage, transformer: droppable());
  }

  final GetUsers _getUsers;

  Future<void> _onStarted(UsersStarted event, Emitter<UserState> emit) async {
    emit(const UserLoading());
    try {
      final result = await _getUsers(page: 1);
```

```
      emit(
        UserLoaded(users: result.users, currentPage: result.page, totalPages: result.totalPages),
      );
    } on Failure catch (failure) {
      emit(UserError(failure.message));
    }
  }

  Future<void> _onNextPage(UsersNextPageRequested event, Emitter<UserState> emit) async {
    final current = state;
    if (current is! UserLoaded || current.hasReachedMax) return;

    emit(current.copyWith(isLoadingMore: true, loadMoreFailed: false));
    try {
      final result = await _getUsers(page: current.currentPage + 1);
      emit(
        current.copyWith(
          users: [...current.users, ...result.users],
          currentPage: result.page,
          totalPages: result.totalPages,
          isLoadingMore: false,
        ),
      );
    } on Failure {
      emit(current.copyWith(isLoadingMore: false, loadMoreFailed: true));
    }
  }
}
// UI:  context.read<UserBloc>().add(const UsersNextPageRequested());
```

Provide with `BlocProvider(create: (_) => UserBloc(GetUsers(repository))..add(const UsersStarted()))`; `BlocBuilder<UserBloc, UserState>` is unchanged.

SECTION 15 — Model vs Entity

```
JSON ──► UserModel ──toEntity()──► User
 data ▒▒ data ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒ domain
```

Why API DTOs must not leak into the domain/UI:

1. **The backend is not yours.** It renames `first_name` → `given_name`, adds fields, makes one nullable. With a mapper, one file changes; without it, every widget, Cubit and test changes.
2. **Different shapes serve different needs.** Model: `first_name`, `avatar`, `per_page`, `meta`. Entity: `fullName`, maybe a computed `isAdmin`.
3. **The domain must stay pure Dart** so it is trivially testable and portable (no `fromJson`, no `json_annotation`).
4. **Versioning**: API v1 and v2 can have different models mapping to the **same** entity.

	`UserModel extends User`	`UserModel` → `toEntity()` → `User`
Speed in a 20-min interview	**Fastest** (`super.id`, no mapper)	+6–8 lines per model (≈ 30 s)
Domain purity	Entity inherits from nothing, but models *are* entities, so the shape coupling is permanent	✅ Clean: entity has no idea JSON exists
Extra API fields (`perPage`, `meta`)	Forced into the entity or lost	Stay in the model
Backend renames a field	Ripples through the app	Mapper only
Wrapper/envelope types	Tempts you to put them in `data/` and import upward (what happened in the audit)	Natural: wrapper model → wrapper entity
Verdict	Acceptable *only if* you say "shortcut for time" **and** the entity still lives in `domain/` with no JSON code	**Default. Use it in the interview and production.**

Fast-but-honest version, if truly out of time:

```dart
class UserModel extends User {                    // shortcut for time;
  const UserModel({required super.id, required super.email /* … */});
  factory UserModel.fromJson(Map<String, dynamic> j) => UserModel(id: j['id'] as int, email: j['email'] as String /* … */);
}                                                 // production: toEntity()
```

SECTION 16 — UseCase

Use case	Wraps	Earns its place?
`GetUsers(page)`	`repository.getUsers`	Thin. Keep as the layering seam.
`GetUserDetails(id)`	`repository.getUser`	Thin.
`Login(email, password)`	`authRepository.login`	Becomes **useful** the day you add validation, analytics, or "save token + fetch profile".
`CreateUser` , `UpdateUser` , `DeleteUser`	repository writes	Thin; useful with rules ("can't delete yourself").
`PlaceOrder`	`cartRepo` + `stockRepo` + `paymentRepo`	**Very** useful: orchestrates several repositories and rules.

Useful when

- It **owns a business rule** (validation, permissions, pricing, ordering of steps).
- It **combines multiple repositories** (`GetUserWithPosts`).
- You want one **named, mockable seam** per business action (clear Cubit tests).
- Several screens/Cubits reuse the same action.

Over-engineering when

- Every repository method gets a pass-through use case on a tiny app (**25 files for CRUD**).
- A "use case" that just renames the repository method and adds nothing.
- A generic `UseCase<Type, Params>` base class + `NoParams` + `Either` on a 20-min task.

Interview stance: "*For a simple read I'd be fine calling the repository from the Cubit. Here I included a use case because it's the seam where rules live, and it keeps the Cubit ignorant of data sources. It's one small file.*" Include it; interviewers expect the layer, and you show judgment by naming the trade-off.

A use case that **earns its place** (verified): validation is a business rule, not UI code and not data code.

```dart
import '../core/errors/failures.dart';
import '../feature/auth/domain/entity/auth_session.dart';
import '../feature/auth/domain/repository/auth_repository.dart';

/// A use case that earns its place: it owns a BUSINESS RULE (validation),
/// not just a pass-through.
class LoginWithValidation {
  const LoginWithValidation(this._repository);

  final AuthRepository _repository;

  Future<AuthSession> call({required String email, required String password}) {
    if (!email.contains('@')) throw const ValidationFailure('Enter a valid email.');
    if (password.length < 6) {
      throw const ValidationFailure('Password must be at least 6 characters.');
    }
    return _repository.login(email: email, password: password);
  }
}
```

The Cubit already handles it: `ValidationFailure` is a `Failure` , so `on Failure catch` shows the message.

SECTION 17 — Testing

Layer	Test	Mock	Verifies
Widget	`UserPage`	Fake `UserRepository` (the interface makes this 10 lines)	Loader → list; error → Retry → list; pagination appends
Cubit	`bloc_test`	`GetUsers` (mocktail)	Exact state sequences: success, failure, `loadMore` append, last page, page-2 failure
UseCase	plain test	`UserRepository`	Rules (validation) / delegation
Repository	plain test	`UserRemoteDataSource`	Model→Entity mapping; each Exception → right Failure
DataSource	plain test	`DioClient`	Correct path/query; JSON parsed into model
DioClient	plain test	`Dio`	Status/timeout/invalid JSON/cancel → right `AppException`; server message extracted; all 5 verbs
Interceptor	plain test	**fake HTTP adapter** (scripted responses) + mocked `TokenStorage`	Bearer header; 401 → refresh → retry; parallel 401s share ONE refresh; failed refresh clears storage; no loop
Token storage	plain test	`FlutterSecureStorage`	Reads/writes the right keys; unavailable keychain = not logged in
DI graph	plain test	none	Whole graph resolves; Cubits are factories; Dio has baseUrl, timeouts, auth interceptor
Safe logger	plain test	fake adapter + captured `debugPrint`	Logs method/url/status only: never tokens, passwords, bodies

What **not** to test: Dio itself, `fromJson` of trivial models one-by-one, private methods. Test behaviour at the boundaries.

Run: `flutter test`. (All of the following are from the verified suite.)

Success + Failure + Pagination: the Cubit

```dart
import 'dart:async';

import 'package:bloc_test/bloc_test.dart';
import 'package:demoapi/core/errors/failures.dart';
import 'package:demoapi/feature/user/domain/entity/paged_users.dart';
import 'package:demoapi/feature/user/domain/entity/user.dart';
import 'package:demoapi/feature/user/domain/usecase/get_users.dart';
import 'package:demoapi/feature/user/presentation/cubit/user_cubit.dart';
import 'package:demoapi/feature/user/presentation/cubit/user_state.dart';
import 'package:mocktail/mocktail.dart';
import 'package:flutter_test/flutter_test.dart';

class MockGetUsers extends Mock implements GetUsers {}

void main() {
  late MockGetUsers getUsers;

  const user1 = User(
    id: 1,
    email: 'a@a.com',
    firstName: 'A',
    lastName: 'One',
    avatar: '',
  );
  const user2 = User(
    id: 2,
    email: 'b@b.com',
    firstName: 'B',
    lastName: 'Two',
    avatar: '',
  );
  const page1 = PagedUsers(users: [user1], page: 1, totalPages: 2);
  const page2 = PagedUsers(users: [user2], page: 2, totalPages: 2);

  setUp(() => getUsers = MockGetUsers());

  group('fetchUsers', () {
    blocTest<UserCubit, UserState>(
      'emits [Loading, Loaded] on success',
      build: () {
        when(() => getUsers(page: 1)).thenAnswer((_) async => page1);
        return UserCubit(getUsers);
      },
      act: (cubit) => cubit.fetchUsers(),
      expect: () => const [
        UserLoading(),
```

```
        UserLoaded(users: [user1], currentPage: 1, totalPages: 2),
      ],
    );

    blocTest<UserCubit, UserState>(
      'emits [Loading, Error] on failure',
      build: () {
        when(() => getUsers(page: 1))
            .thenThrow(const NetworkFailure('No internet connection.'));
        return UserCubit(getUsers);
      },
      act: (cubit) => cubit.fetchUsers(),
      expect: () => const [UserLoading(), UserError('No internet connection.')],
    );
  });

  group('loadMore', () {
    blocTest<UserCubit, UserState>(
      'appends page 2 to page 1',
      build: () {
        when(() => getUsers(page: 2)).thenAnswer((_) async => page2);
        return UserCubit(getUsers);
      },
      seed: () =>
          const UserLoaded(users: [user1], currentPage: 1, totalPages: 2),
      act: (cubit) => cubit.loadMore(),
      expect: () => const [
        UserLoaded(
          users: [user1],
          currentPage: 1,
          totalPages: 2,
          isLoadingMore: true,
        ),
        UserLoaded(users: [user1, user2], currentPage: 2, totalPages: 2),
      ],
    );

    blocTest<UserCubit, UserState>(
      'does nothing when last page reached',
      build: () => UserCubit(getUsers),
      seed: () =>
          const UserLoaded(users: [user1], currentPage: 2, totalPages: 2),
      act: (cubit) => cubit.loadMore(),
      expect: () => const <UserState>[],
      verify: (_) => verifyNever(() => getUsers(page: any(named: 'page'))),
    );

    blocTest<UserCubit, UserState>(
      'keeps the list and flags loadMoreFailed when page 2 fails',
      build: () {
        when(() => getUsers(page: 2))
            .thenThrow(const ServerFailure('Server error.'));
        return UserCubit(getUsers);
      },
      seed: () =>
          const UserLoaded(users: [user1], currentPage: 1, totalPages: 2),
      act: (cubit) => cubit.loadMore(),
      expect: () => const [
        UserLoaded(
          users: [user1],
          currentPage: 1,
          totalPages: 2,
          isLoadingMore: true,
        ),
        UserLoaded(
          users: [user1],
          currentPage: 1,
          totalPages: 2,
          loadMoreFailed: true,
        ),
      ],
    );
  });

  group('hardening', () {
    blocTest<UserCubit, UserState>(
      'a cancelled request is ignored: no error UI',
      build: () {
        when(() => getUsers(page: 1)).thenThrow(const CancelledFailure());
        return UserCubit(getUsers);
      },
      act: (cubit) => cubit.fetchUsers(),
      expect: () => const [UserLoading()],
    );

    blocTest<UserCubit, UserState>(
```

```
    'unexpected failures reach Bloc.observer.onError (logging hook)',
    build: () {
      when(() => getUsers(page: 1)).thenThrow(
        UnknownFailure(
          cause: StateError('boom'),
          stackTrace: StackTrace.current,
        ),
      );
      return UserCubit(getUsers);
    },
    act: (cubit) => cubit.fetchUsers(),
    expect: () => const [
      UserLoading(),
      UserError('Something went wrong. Please try again.'),
    ],
    errors: () => [isA<StateError>()],
  );

  blocTest<UserCubit, UserState>(
    'expected failures (offline) are NOT reported as errors',
    build: () {
      when(() => getUsers(page: 1))
          .thenThrow(const NetworkFailure('No internet connection.'));
      return UserCubit(getUsers);
    },
    act: (cubit) => cubit.fetchUsers(),
    expect: () => const [UserLoading(), UserError('No internet connection.')],
    errors: () => <Matcher>[],
  );

  test('a stale response from an earlier fetchUsers() is dropped', () async {
    final slow = Completer<PagedUsers>();
    when(() => getUsers(page: 1)).thenAnswer((_) => slow.future);
    final cubit = UserCubit(getUsers);
    final firstCall = cubit.fetchUsers();

    when(() => getUsers(page: 1)).thenAnswer((_) async => page1);
    await cubit.fetchUsers(); // the user hit Retry; this one wins
    slow.complete(const PagedUsers(users: [user2], page: 1, totalPages: 1));
    await firstCall;

    expect(
      cubit.state,
      const UserLoaded(users: [user1], currentPage: 1, totalPages: 2),
    );
    await cubit.close();
  });

  test(
    'loadMore result is discarded when the list was reloaded meanwhile',
    () async {
      when(() => getUsers(page: 1)).thenAnswer((_) async => page1);
      final cubit = UserCubit(getUsers);
      await cubit.fetchUsers();

      final slowPage2 = Completer<PagedUsers>();
      when(() => getUsers(page: 2)).thenAnswer((_) => slowPage2.future);
      final loading = cubit.loadMore();

      await cubit.fetchUsers(); // refresh while page 2 is in flight
      slowPage2.complete(page2);
      await loading;

      expect(
        cubit.state,
        const UserLoaded(users: [user1], currentPage: 1, totalPages: 2),
        reason: 'page 2 of the OLD list must not be appended to the new one',
      );
      await cubit.close();
    },
  );

  blocTest<UserCubit, UserState>(
    'closing the cubit mid-request does not emit (no StateError)',
    build: () {
      when(() => getUsers(page: 1)).thenAnswer((_) async {
        await Future<void>.delayed(const Duration(milliseconds: 10));
        return page1;
      });
      return UserCubit(getUsers);
    },
    act: (cubit) {
      final pending = cubit.fetchUsers();
      cubit.close();
      return pending;
    },
```

```
      expect: () => const [UserLoading()],
    );
  });
}
```

Note: `seed:` starts the Cubit in a given state (page 1 loaded) so the pagination case is a 4-line test. States compare with `==` **because they are** `Equatable`: that is the practical payoff.

Repository: exceptions → failures, model → entity

```
import 'package:demoapi/core/errors/exceptions.dart';
import 'package:demoapi/core/errors/failures.dart';
import 'package:demoapi/feature/user/data/data_source/user_remote_data_source.dart';
import 'package:demoapi/feature/user/data/model/user_model.dart';
import 'package:demoapi/feature/user/data/model/users_response_model.dart';
import 'package:demoapi/feature/user/data/repository/user_repository_impl.dart';
import 'package:mocktail/mocktail.dart';
import 'package:flutter_test/flutter_test.dart';

class MockRemote extends Mock implements UserRemoteDataSource {}

void main() {
  late MockRemote remote;
  late UserRepositoryImpl repository;

  setUp(() {
    remote = MockRemote();
    repository = UserRepositoryImpl(remote);
  });

  test('maps model -> entity on success', () async {
    when(() => remote.getUsers(page: 1)).thenAnswer(
      (_) async => const UsersResponseModel(
        page: 1,
        perPage: 6,
        total: 12,
        totalPages: 2,
        data: [
          UserModel(
            id: 1,
            email: 'a@a.com',
            firstName: 'A',
            lastName: 'One',
            avatar: '',
          ),
        ],
      ),
    );

    final result = await repository.getUsers(page: 1);

    expect(result.users.single.fullName, 'A One');
    expect(result.hasNext, isTrue);
  });

  test('ServerException(404) -> ServerFailure', () {
    when(() => remote.getUsers(page: 1))
        .thenThrow(const ServerException('Not found.', statusCode: 404));

    expect(repository.getUsers(page: 1), throwsA(isA<ServerFailure>()));
  });

  test('ServerException(401) -> UnauthorizedFailure', () {
    when(
      () => remote.getUsers(page: 1),
    ).thenThrow(const ServerException('Please login again.', statusCode: 401));

    expect(repository.getUsers(page: 1), throwsA(isA<UnauthorizedFailure>()));
  });

  test('ServerException(422) -> ValidationFailure', () {
    when(() => remote.getUsers(page: 1))
        .thenThrow(const ServerException('Email is invalid.', statusCode: 422));

    expect(repository.getUsers(page: 1), throwsA(isA<ValidationFailure>()));
  });

  test('NetworkException -> NetworkFailure', () {
    when(() => remote.getUsers(page: 1))
        .thenThrow(const NetworkException('No internet connection.'));

    expect(repository.getUsers(page: 1), throwsA(isA<NetworkFailure>()));
  });
```

```
  test('bad JSON shape (TypeError) -> ParsingFailure', () {
    when(() => remote.getUsers(page: 1))
        .thenAnswer((_) async => UsersResponseModel.fromJson({'page': 'oops'}));

    expect(repository.getUsers(page: 1), throwsA(isA<ParsingFailure>()));
  });

  test('CancelledException -> CancelledFailure', () {
    when(() => remote.getUsers(page: 1))
        .thenThrow(const CancelledException('Request cancelled.'));

    expect(repository.getUsers(page: 1), throwsA(isA<CancelledFailure>()));
  });

  test('ParsingFailure keeps the technical cause for logging', () async {
    when(() => remote.getUsers(page: 1))
        .thenAnswer((_) async => UsersResponseModel.fromJson({'page': 'oops'}));

    await expectLater(
      repository.getUsers(page: 1),
      throwsA(
        isA<ParsingFailure>()
            .having((f) => f.cause, 'cause', isA<TypeError>())
            .having((f) => f.stackTrace, 'stackTrace', isNotNull)
            .having(
              (f) => f.message,
              'user-safe message',
              isNot(contains('type')),
            ),
      ),
    );
  });

  test('unknown errors become UnknownFailure and keep cause + stack', () async {
    when(() => remote.getUsers(page: 1)).thenThrow(StateError('boom'));

    await expectLater(
      repository.getUsers(page: 1),
      throwsA(
        isA<UnknownFailure>()
            .having((f) => f.cause, 'cause', isA<StateError>())
            .having((f) => f.stackTrace, 'stackTrace', isNotNull),
      ),
    );
  });
}
```

Data source: path, query, parsing

```
import 'package:demoapi/core/network/dio_client.dart';
import 'package:demoapi/feature/user/data/data_source/user_remote_data_source_impl.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:mocktail/mocktail.dart';

class MockDioClient extends Mock implements DioClient {}

void main() {
  late MockDioClient client;
  late UserRemoteDataSourceImpl dataSource;

  setUp(() {
    client = MockDioClient();
    dataSource = UserRemoteDataSourceImpl(client);
  });

  test('GET /users?page=2 and parses the JSON', () async {
    when(() => client.get('/users', query: any(named: 'query'))).thenAnswer(
      (_) async => {
        'page': 2,
        'per_page': 6,
        'total': 12,
        'total_pages': 2,
        'data': [
          {
            'id': 7,
            'email': 'm@reqres.in',
            'first_name': 'Michael',
            'last_name': 'Lawson',
            'avatar': 'x.jpg',
          },
        ],
      },
    );
```

```
    final result = await dataSource.getUsers(page: 2);

    expect(result.page, 2);
    expect(result.data.single.firstName, 'Michael');
    verify(() => client.get('/users', query: {'page': 2})).called(1);
  });
}
```

DioClient: HTTP errors → exceptions

```
import 'package:demoapi/core/errors/exceptions.dart';
import 'package:demoapi/core/network/dio_client.dart';
import 'package:dio/dio.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:mocktail/mocktail.dart';

class MockDio extends Mock implements Dio {}

void main() {
  late MockDio dio;
  late DioClient client;

  setUp(() {
    dio = MockDio();
    client = DioClient(dio);
  });

  final options = RequestOptions(path: '/x');
  Response<dynamic> ok(Object? data) =>
      Response(requestOptions: options, data: data);

  DioException badResponse(int code, [Object? body]) => DioException(
    requestOptions: options,
    type: DioExceptionType.badResponse,
    response: Response(requestOptions: options, statusCode: code, data: body),
  );

  void stubGet(Object Function() answer) {
    when(
      () => dio.get<dynamic>(
        '/x',
        queryParameters: any(named: 'queryParameters'),
        cancelToken: any(named: 'cancelToken'),
      ),
    ).thenAnswer((_) async {
      final a = answer();
      if (a is Exception) throw a;
      return ok(a);
    });
  }

  group('success', () {
    test('GET returns response.data', () async {
      stubGet(() => {'ok': true});
      expect(await client.get('/x'), {'ok': true});
    });

    test('POST / PUT / PATCH / DELETE use the right Dio method', () async {
      when(
        () => dio.post<dynamic>(
          '/x',
          data: any(named: 'data'),
          cancelToken: any(named: 'cancelToken'),
        ),
      ).thenAnswer((_) async => ok('post'));
      when(
        () => dio.put<dynamic>(
          '/x',
          data: any(named: 'data'),
          cancelToken: any(named: 'cancelToken'),
        ),
      ).thenAnswer((_) async => ok('put'));
      when(
        () => dio.patch<dynamic>(
          '/x',
          data: any(named: 'data'),
          cancelToken: any(named: 'cancelToken'),
        ),
      ).thenAnswer((_) async => ok('patch'));
      when(
        () => dio.delete<dynamic>('/x', cancelToken: any(named: 'cancelToken')),
      ).thenAnswer((_) async => ok(''));

      expect(await client.post('/x', data: {'a': 1}), 'post');
```

```
    expect(await client.put('/x', data: {'a': 1}), 'put');
    expect(await client.patch('/x', data: {'a': 1}), 'patch');
    expect(await client.delete('/x'), '');
    verify(() => dio.post<dynamic>('/x', data: {'a': 1})).called(1);
  });

  test('forwards the CancelToken to Dio', () async {
    final token = CancelToken();
    when(
      () => dio.get<dynamic>(
        '/x',
        queryParameters: any(named: 'queryParameters'),
        cancelToken: token,
      ),
    ).thenAnswer((_) async => ok(1));

    await client.get('/x', cancelToken: token);

    verify(
      () => dio.get<dynamic>('/x', queryParameters: null, cancelToken: token),
    ).called(1);
  });
});

group('errors', () {
  test('400 {"error": ...} -> ServerException with that message', () {
    stubGet(() => badResponse(400, {'error': 'Missing password'}));

    expect(
      client.get('/x'),
      throwsA(
        isA<ServerException>()
            .having((e) => e.message, 'message', 'Missing password')
            .having((e) => e.statusCode, 'statusCode', 400),
      ),
    );
  });

  test('{"message": ...} is understood too', () {
    stubGet(() => badResponse(422, {'message': 'Email is invalid'}));

    expect(
      client.get('/x'),
      throwsA(
        isA<ServerException>().having(
          (e) => e.message,
          'message',
          'Email is invalid',
        ),
      ),
    );
  });

  test('no body: falls back to a default message per status', () {
    stubGet(() => badResponse(404));

    expect(
      client.get('/x'),
      throwsA(
        isA<ServerException>().having((e) => e.message, 'm', 'Not found.'),
      ),
    );
  });

  test('5xx -> "Server error" message', () {
    stubGet(() => badResponse(503));

    expect(
      client.get('/x'),
      throwsA(
        isA<ServerException>().having(
          (e) => e.message,
          'm',
          contains('Server error'),
        ),
      ),
    );
  });

  test('timeout -> NetworkException', () {
    stubGet(
      () => DioException(
        requestOptions: options,
        type: DioExceptionType.receiveTimeout,
      ),
    );
```

```
      expect(client.get('/x'), throwsA(isA<NetworkException>()));
    });

    test('connection error -> NetworkException "No internet"', () {
      stubGet(
        () => DioException(
          requestOptions: options,
          type: DioExceptionType.connectionError,
        ),
      );
      expect(
        client.get('/x'),
        throwsA(
          isA<NetworkException>().having(
            (e) => e.message,
            'm',
            'No internet connection.',
          ),
        ),
      );
    });

    test('cancelled request -> CancelledException (not a network error)', () {
      stubGet(
        () => DioException(
          requestOptions: options,
          type: DioExceptionType.cancel,
        ),
      );
      expect(client.get('/x'), throwsA(isA<CancelledException>()));
    });

    test('invalid JSON body -> ParsingException (not a network error)', () {
      stubGet(
        () => DioException(
          requestOptions: options,
          type: DioExceptionType.unknown,
          error: const FormatException('Unexpected character'),
        ),
      );
      expect(client.get('/x'), throwsA(isA<ParsingException>()));
    });
  });
}
```

Widget test with a fake repository

```
import 'package:demoapi/core/errors/failures.dart';
import 'package:demoapi/feature/user/domain/entity/paged_users.dart';
import 'package:demoapi/feature/user/domain/entity/user.dart';
import 'package:demoapi/feature/user/domain/repository/user_repository.dart';
import 'package:demoapi/feature/user/domain/usecase/get_users.dart';
import 'package:demoapi/feature/user/presentation/cubit/user_cubit.dart';
import 'package:demoapi/feature/user/presentation/pages/user_page.dart';
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';
import 'package:flutter_test/flutter_test.dart';

/// Possible only because the UI depends on the UserRepository INTERFACE.
class FakeUserRepository implements UserRepository {
  FakeUserRepository({this.failFirstCall = false, this.failPage2Once = false});

  bool failFirstCall;
  bool failPage2Once;

  @override
  Future<PagedUsers> getUsers({required int page}) async {
    if (failFirstCall) {
      failFirstCall = false;
      throw const NetworkFailure('No internet connection.');
    }
    if (page == 2 && failPage2Once) {
      failPage2Once = false;
      throw const NetworkFailure('No internet connection.');
    }
    return PagedUsers(
      page: page,
      totalPages: 2,
      users: List.generate(
        6,
        (i) => User(
          id: (page - 1) * 6 + i,
          email: 'u${(page - 1) * 6 + i}@mail.com',
          firstName: 'First${(page - 1) * 6 + i}',
```

```
          lastName: 'Last',
          avatar: '',
        ),
      ),
    );
  }
}

Widget _app(FakeUserRepository repo) => MaterialApp(
  home: BlocProvider(
    create: (_) => UserCubit(GetUsers(repo))..fetchUsers(),
    child: const UserPage(),
  ),
);

void main() {
  testWidgets('shows loader, users, then auto-loads page 2', (tester) async {
    await tester.pumpWidget(_app(FakeUserRepository()));
    expect(find.byType(CircularProgressIndicator), findsOneWidget);

    await tester.pumpAndSettle();
    expect(find.text('First0 Last'), findsOneWidget);

    // ListView is lazy: scroll to the last row to prove page 2 was appended.
    await tester.scrollUntilVisible(find.text('First11 Last'), 200);
    expect(find.text('First11 Last'), findsOneWidget);
    expect(find.byType(CircularProgressIndicator), findsNothing); // max reached
  });

  testWidgets('error view with Retry recovers', (tester) async {
    await tester.pumpWidget(_app(FakeUserRepository(failFirstCall: true)));
    await tester.pumpAndSettle();

    expect(find.text('No internet connection.'), findsOneWidget);

    await tester.tap(find.text('Retry'));
    await tester.pumpAndSettle();

    expect(find.text('First0 Last'), findsOneWidget);
  });

  testWidgets('page 2 fails: list stays, footer offers Retry, retry loads it', (
    tester,
  ) async {
    await tester.pumpWidget(_app(FakeUserRepository(failPage2Once: true)));
    await tester.pumpAndSettle();

    // page 1 is still on screen and the footer shows the retry button
    expect(find.text('First0 Last'), findsOneWidget);
    await tester.scrollUntilVisible(
      find.text('Failed to load. Tap to retry'),
      200,
    );

    await tester.tap(find.text('Failed to load. Tap to retry'));
    await tester.pumpAndSettle();

    await tester.scrollUntilVisible(find.text('First11 Last'), 200);
    expect(find.text('First11 Last'), findsOneWidget);
  });
}
```

Gotcha: `ListView` is **lazy**: rows off-screen aren't built, so `find.text('First11 Last')` fails until you `scrollUntilVisible`.

Login Cubit (Command style)

```
import 'package:bloc_test/bloc_test.dart';
import 'package:demoapi/core/errors/failures.dart';
import 'package:demoapi/feature/auth/domain/entity/auth_session.dart';
import 'package:demoapi/feature/auth/domain/usecase/login.dart';
import 'package:demoapi/feature/auth/presentation/cubit/login_cubit.dart';
import 'package:demoapi/feature/auth/presentation/cubit/login_state.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:mocktail/mocktail.dart';

class MockLogin extends Mock implements Login {}

void main() {
  late MockLogin login;
  setUp(() => login = MockLogin());

  blocTest<LoginCubit, LoginState>(
    'emits [Loading, Success] when credentials are valid',
    build: () {
```

```
      when(() => login(email: 'a@a.com', password: 'secret1'))
          .thenAnswer((_) async => const AuthSession(token: 't'));
      return LoginCubit(login);
    },
    act: (c) => c.login(email: 'a@a.com', password: 'secret1'),
    expect: () => const [LoginLoading(), LoginSuccess()],
  );

  blocTest<LoginCubit, LoginState>(
    'emits [Loading, Error] when the server rejects the login',
    build: () {
      when(
        () => login(
          email: any(named: 'email'),
          password: any(named: 'password'),
        ),
      ).thenThrow(const ServerFailure('Missing password', statusCode: 400));
      return LoginCubit(login);
    },
    act: (c) => c.login(email: 'a@a.com', password: ''),
    expect: () => const [LoginLoading(), LoginError('Missing password')],
  );
}
```

Auth interceptor against a fake HTTP layer

`FakeAdapter` replaces Dio's network layer: the test scripts every response, so the refresh flow is tested without a server.

```
import 'dart:convert';
import 'dart:typed_data';

import 'package:dio/dio.dart';

/// Scripted HTTP layer for Dio: no network, the test decides every response.
class FakeAdapter implements HttpClientAdapter {
  FakeAdapter(this._handler);

  final ResponseBody Function(RequestOptions) _handler;
  final requests = <RequestOptions>[];

  @override
  Future<ResponseBody> fetch(
    RequestOptions options,
    Stream<Uint8List>? requestStream,
    Future<void>? cancelFuture,
  ) async {
    requests.add(options);
    return _handler(options);
  }

  @override
  void close({bool force = false}) {}
}

ResponseBody json(Map<String, dynamic> body, int status) =>
    ResponseBody.fromString(
      jsonEncode(body),
      status,
      headers: {
        Headers.contentTypeHeader: [Headers.jsonContentType],
      },
    );
```

```
import 'package:demoapi/core/network/auth_interceptor.dart';
import 'package:demoapi/core/storage/token_storage.dart';
import 'package:dio/dio.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:mocktail/mocktail.dart';

import '../helpers/fake_adapter.dart';

class MockTokenStorage extends Mock implements TokenStorage {}

void main() {
  late MockTokenStorage storage;
  late String access;
  String? refreshToken;
  late int refreshCalls;
  late bool refreshWorks;

  Dio buildDio({int? alwaysStatus}) {
    final adapter = FakeAdapter((o) {
      if (o.path == '/auth/refresh') {
        refreshCalls++;
```

```
      return refreshWorks
          ? json({'access_token': 'new'}, 200)
          : json({'error': 'refresh expired'}, 401);
    }
    if (alwaysStatus != null) {
      return json({'status': alwaysStatus}, alwaysStatus);
    }
    // a "server" that only accepts the NEW token
    return o.headers['Authorization'] == 'Bearer new'
        ? json({'ok': true}, 200)
        : json({'error': 'expired'}, 401);
  });
  final plain = Dio(BaseOptions(baseUrl: 'http://api.test'))
    ..httpClientAdapter = adapter;
  return Dio(BaseOptions(baseUrl: 'http://api.test'))
    ..httpClientAdapter = adapter
    ..interceptors.add(AuthInterceptor(storage: storage, plainDio: plain));
}

setUp(() {
  storage = MockTokenStorage();
  access = 'old';
  refreshToken = 'refresh';
  refreshCalls = 0;
  refreshWorks = true;
  when(() => storage.readAccess()).thenAnswer((_) async => access);
  when(() => storage.readRefresh()).thenAnswer((_) async => refreshToken);
  when(
    () => storage.save(
      access: any(named: 'access'),
      refresh: any(named: 'refresh'),
    ),
  ).thenAnswer((i) async => access = i.namedArguments[#access] as String);
  when(() => storage.clear()).thenAnswer((_) async {});
});

test('attaches "Authorization: Bearer <token>" to every request', () async {
  final dio = buildDio(alwaysStatus: 200);

  await dio.get<dynamic>('/users');

  final adapter = dio.httpClientAdapter as FakeAdapter;
  expect(adapter.requests.single.headers['Authorization'], 'Bearer old');
});

test('no token stored -> no Authorization header', () async {
  when(() => storage.readAccess()).thenAnswer((_) async => null);
  final dio = buildDio(alwaysStatus: 200);

  await dio.get<dynamic>('/users');

  final adapter = dio.httpClientAdapter as FakeAdapter;
  expect(
    adapter.requests.single.headers.containsKey('Authorization'),
    isFalse,
  );
});

test(
  '401 -> refresh -> retries the original request with the new token',
  () async {
    final dio = buildDio();

    final response = await dio.get<dynamic>('/users');

    expect(response.data, {'ok': true});
    expect(refreshCalls, 1);
    verify(() => storage.save(access: 'new', refresh: null)).called(1);
  },
);

test('parallel 401s share ONE refresh call', () async {
  final dio = buildDio();

  final results = await Future.wait([
    dio.get<dynamic>('/a'),
    dio.get<dynamic>('/b'),
    dio.get<dynamic>('/c'),
  ]);

  expect(results.map((r) => r.statusCode), everyElement(200));
  expect(refreshCalls, 1);
});

test('refresh fails -> storage cleared, original 401 is surfaced', () async {
  refreshWorks = false;
```

```
    final dio = buildDio();

    await expectLater(
      dio.get<dynamic>('/users'),
      throwsA(
        isA<DioException>().having(
          (e) => e.response?.statusCode,
          'status',
          401,
        ),
      ),
    );

    expect(refreshCalls, 1);
    verify(() => storage.clear()).called(1);
  });

  test('no refresh token -> does not call the refresh endpoint', () async {
    refreshToken = null;
    final dio = buildDio();

    await expectLater(dio.get<dynamic>('/users'), throwsA(isA<DioException>()));

    expect(refreshCalls, 0);
  });

  test('non-401 errors pass through untouched (no refresh)', () async {
    final dio = buildDio(alwaysStatus: 500);

    await expectLater(
      dio.get<dynamic>('/users'),
      throwsA(
        isA<DioException>().having(
          (e) => e.response?.statusCode,
          'status',
          500,
        ),
      ),
    );

    expect(refreshCalls, 0);
  });

  test(
    'retry with the NEW token still 401 -> error surfaced, no refresh loop',
    () async {
      final adapter = FakeAdapter((o) {
        if (o.path == '/auth/refresh') {
          refreshCalls++;
          return json({'access_token': 'new'}, 200);
        }
        return json({
          'error': 'still unauthorized',
        }, 401); // even for the new token
      });
      final plain = Dio(BaseOptions(baseUrl: 'http://api.test'))
        ..httpClientAdapter = adapter;
      final dio = Dio(BaseOptions(baseUrl: 'http://api.test'))
        ..httpClientAdapter = adapter
        ..interceptors.add(AuthInterceptor(storage: storage, plainDio: plain));

      await expectLater(
        dio.get<dynamic>('/users'),
        throwsA(
          isA<DioException>().having(
            (e) => e.response?.statusCode,
            'status',
            401,
          ),
        ),
      );

      expect(refreshCalls, 1, reason: 'one refresh, not a loop');
    },
  );
}
```

Token storage, DI graph, safe logger

```
import 'package:demoapi/core/storage/token_storage.dart';
import 'package:flutter/services.dart';
import 'package:flutter_secure_storage/flutter_secure_storage.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:mocktail/mocktail.dart';
```

```
class MockSecureStorage extends Mock implements FlutterSecureStorage {}

void main() {
  late MockSecureStorage secure;
  late TokenStorage storage;

  setUp(() {
    secure = MockSecureStorage();
    storage = TokenStorage(secure);
    when(
      () => secure.write(
        key: any(named: 'key'),
        value: any(named: 'value'),
      ),
    ).thenAnswer((_) async {});
    when(() => secure.delete(key: any(named: 'key'))).thenAnswer((_) async {});
  });

  test('reads access and refresh tokens from their own keys', () async {
    when(() => secure.read(key: 'access_token')).thenAnswer((_) async => 'a');
    when(() => secure.read(key: 'refresh_token')).thenAnswer((_) async => 'r');

    expect(await storage.readAccess(), 'a');
    expect(await storage.readRefresh(), 'r');
  });

  test('keychain unavailable (PlatformException) = not logged in', () async {
    when(() => secure.read(key: any(named: 'key')))
        .thenThrow(PlatformException(code: '-34018'));

    expect(await storage.readAccess(), isNull);
    expect(await storage.readRefresh(), isNull);
  });

  test('save writes access, and refresh only when given', () async {
    await storage.save(access: 'a');
    verify(() => secure.write(key: 'access_token', value: 'a')).called(1);
    verifyNever(
      () => secure.write(
        key: 'refresh_token',
        value: any(named: 'value'),
      ),
    );

    await storage.save(access: 'a2', refresh: 'r2');
    verify(() => secure.write(key: 'refresh_token', value: 'r2')).called(1);
  });

  test('clear removes both tokens', () async {
    await storage.clear();

    verify(() => secure.delete(key: 'access_token')).called(1);
    verify(() => secure.delete(key: 'refresh_token')).called(1);
  });
}
```

```
import 'package:demoapi/core/di/injection.dart';
import 'package:demoapi/core/network/auth_interceptor.dart';
import 'package:demoapi/core/storage/token_storage.dart';
import 'package:demoapi/feature/auth/presentation/cubit/login_cubit.dart';
import 'package:demoapi/feature/user/presentation/cubit/user_cubit.dart';
import 'package:dio/dio.dart';
import 'package:flutter_secure_storage/flutter_secure_storage.dart';
import 'package:flutter_test/flutter_test.dart';

void main() {
  setUp(() async {
    await sl.reset();
    initDependencies();
  });
  tearDown(() => sl.reset());

  test('the whole graph resolves (catches wiring mistakes at test time)', () {
    expect(sl<UserCubit>(), isA<UserCubit>());
    expect(sl<LoginCubit>(), isA<LoginCubit>());
  });

  test('Cubits are factories: every request gets a NEW instance', () {
    expect(identical(sl<UserCubit>(), sl<UserCubit>()), isFalse);
  });

  test(
    'Dio is a singleton with baseUrl, timeouts and the auth interceptor',
    () {
```

```
      final dio = sl<Dio>();

      expect(identical(dio, sl<Dio>()), isTrue);
      expect(dio.options.baseUrl, 'https://reqres.in/api');
      expect(dio.options.connectTimeout, isNotNull);
      expect(dio.interceptors.whereType<AuthInterceptor>(), hasLength(1));
    },
  );

  test('createDio does not need call-order tricks', () {
    final dio = createDio(const TokenStorage(FlutterSecureStorage()));

    expect(dio.interceptors.whereType<AuthInterceptor>(), hasLength(1));
  });
}
```

```
import 'package:demoapi/core/network/safe_log_interceptor.dart';
import 'package:dio/dio.dart';
import 'package:flutter/foundation.dart';
import 'package:flutter_test/flutter_test.dart';

import '../helpers/fake_adapter.dart';

void main() {
  late List<String> logs;
  late DebugPrintCallback originalDebugPrint;

  setUp(() {
    logs = [];
    originalDebugPrint = debugPrint;
    debugPrint = (message, {wrapWidth}) => logs.add(message ?? '');
  });
  tearDown(() => debugPrint = originalDebugPrint);

  Dio dioReturning(int status) => Dio(BaseOptions(baseUrl: 'http://api.test'))
    ..httpClientAdapter = FakeAdapter(
      (_) => json({'password': 'hunter2', 'token': 'SECRET'}, status),
    )
    ..interceptors.add(SafeLogInterceptor());

  test('logs method, url and status only', () async {
    await dioReturning(200).post<dynamic>(
      '/login',
      data: {'password': 'hunter2'},
      options: Options(headers: {'Authorization': 'Bearer SECRET'}),
    );

    final all = logs.join('\n');
    expect(all, contains('-> POST http://api.test/login'));
    expect(all, contains('<- 200 http://api.test/login'));
  });

  test('NEVER logs tokens, passwords or response bodies', () async {
    await dioReturning(200).post<dynamic>(
      '/login',
      data: {'password': 'hunter2'},
      options: Options(headers: {'Authorization': 'Bearer SECRET'}),
    );

    final all = logs.join('\n');
    expect(all, isNot(contains('SECRET')));
    expect(all, isNot(contains('hunter2')));
    expect(all, isNot(contains('Authorization')));
  });

  test('errors log status and type, still without the body', () async {
    await expectLater(
      dioReturning(500).get<dynamic>('/users'),
      throwsA(isA<DioException>()),
    );

    final all = logs.join('\n');
    expect(all, contains('x  500 http://api.test/users badResponse'));
    expect(all, isNot(contains('SECRET')));
  });
}
```

Login page, auth repository

```
import 'dart:async';

import 'package:bloc_test/bloc_test.dart';
import 'package:demoapi/feature/auth/presentation/cubit/login_cubit.dart';
import 'package:demoapi/feature/auth/presentation/cubit/login_state.dart';
```

```dart
import 'package:demoapi/feature/auth/presentation/pages/login_page.dart';
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:mocktail/mocktail.dart';

class MockLoginCubit extends MockCubit<LoginState> implements LoginCubit {}

void main() {
  late MockLoginCubit cubit;

  setUp(() => cubit = MockLoginCubit());

  Widget app({Widget? home}) => MaterialApp(
    home:
        home ??
        BlocProvider<LoginCubit>.value(value: cubit, child: const LoginPage()),
  );

  testWidgets('tapping Login sends the typed email and password', (
    tester,
  ) async {
    when(() => cubit.state).thenReturn(const LoginInitial());
    when(
      () => cubit.login(
        email: any(named: 'email'),
        password: any(named: 'password'),
      ),
    ).thenAnswer((_) async {});

    await tester.pumpWidget(app());
    await tester.enterText(
      find.widgetWithText(TextField, 'Email'),
      ' eve@reqres.in ',
    );
    await tester.enterText(
      find.widgetWithText(TextField, 'Password'),
      'cityslicka',
    );
    await tester.tap(find.widgetWithText(FilledButton, 'Login'));

    verify(() => cubit.login(email: 'eve@reqres.in', password: 'cityslicka'))
        .called(1);
  });

  testWidgets('loading disables the button and shows a spinner', (
    tester,
  ) async {
    when(() => cubit.state).thenReturn(const LoginLoading());

    await tester.pumpWidget(app());

    expect(find.byType(CircularProgressIndicator), findsOneWidget);
    final button = tester.widget<FilledButton>(find.byType(FilledButton));
    expect(button.onPressed, isNull);
  });

  testWidgets('error shows the server message in a snackbar', (tester) async {
    whenListen(
      cubit,
      Stream.fromIterable(const [
        LoginLoading(),
        LoginError('Missing password'),
      ]),
      initialState: const LoginInitial(),
    );

    await tester.pumpWidget(app());
    await tester.pump();
    await tester.pump();

    expect(find.text('Missing password'), findsOneWidget);
  });

  testWidgets('success pops back and confirms with a snackbar', (tester) async {
    final controller = StreamController<LoginState>();
    addTearDown(controller.close);
    whenListen(cubit, controller.stream, initialState: const LoginInitial());

    await tester.pumpWidget(
      MaterialApp(
        home: Builder(
          builder: (context) => Scaffold(
            body: Center(
              child: TextButton(
                onPressed: () => Navigator.of(context).push(
```

```
                MaterialPageRoute<void>(
                  builder: (_) => BlocProvider<LoginCubit>.value(
                    value: cubit,
                    child: const LoginPage(),
                  ),
                ),
              ),
            ),
            child: const Text('open login'),
          ),
        ),
      ),
    ),
  );
  await tester.tap(find.text('open login'));
  await tester.pumpAndSettle();
  expect(find.byType(LoginPage), findsOneWidget);

  controller.add(const LoginSuccess());
  await tester.pumpAndSettle();

  expect(find.byType(LoginPage), findsNothing);
  expect(find.text('Logged in'), findsOneWidget);
 });
}
```

```
import 'package:demoapi/core/errors/exceptions.dart';
import 'package:demoapi/core/errors/failures.dart';
import 'package:demoapi/core/storage/token_storage.dart';
import 'package:demoapi/feature/auth/data/data_source/auth_remote_data_source.dart';
import 'package:demoapi/feature/auth/data/model/login_request_model.dart';
import 'package:demoapi/feature/auth/data/model/login_response_model.dart';
import 'package:demoapi/feature/auth/data/repository/auth_repository_impl.dart';
import 'package:demoapi/feature/auth/domain/entity/auth_session.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:mocktail/mocktail.dart';

class MockRemote extends Mock implements AuthRemoteDataSource {}

class MockTokenStorage extends Mock implements TokenStorage {}

void main() {
  late MockRemote remote;
  late MockTokenStorage storage;
  late AuthRepositoryImpl repository;

  setUpAll(
    () =>
        registerFallbackValue(const LoginRequestModel(email: '', password: '')),
  );

  setUp(() {
    remote = MockRemote();
    storage = MockTokenStorage();
    repository = AuthRepositoryImpl(remote, storage);
    when(
      () => storage.save(
        access: any(named: 'access'),
        refresh: any(named: 'refresh'),
      ),
    ).thenAnswer((_) async {});
  });

  test(
    'success: sends the request model, stores the token, returns entity',
    () async {
      when(() => remote.login(any()))
          .thenAnswer((_) async => const LoginResponseModel(token: 'abc'));

      final session = await repository.login(
        email: 'a@a.com',
        password: 'secret1',
      );

      expect(session, const AuthSession(token: 'abc'));
      final sent =
          verify(() => remote.login(captureAny())).captured.single
              as LoginRequestModel;
      expect(sent.toJson(), {'email': 'a@a.com', 'password': 'secret1'});
      verify(() => storage.save(access: 'abc', refresh: null)).called(1);
    },
  );

  test('400 -> ServerFailure and NO token is stored', () async {
    when(() => remote.login(any()))
```

```
        .thenThrow(const ServerException('Missing password', statusCode: 400));

    await expectLater(
      repository.login(email: 'a@a.com', password: ''),
      throwsA(
        isA<ServerFailure>().having(
          (f) => f.message,
          'message',
          'Missing password',
        ),
      ),
    );
    verifyNever(
      () => storage.save(
        access: any(named: 'access'),
        refresh: any(named: 'refresh'),
      ),
    );
  });

  test('offline -> NetworkFailure', () {
    when(() => remote.login(any()))
        .thenThrow(const NetworkException('No internet connection.'));

    expect(
      repository.login(email: 'a@a.com', password: 'x'),
      throwsA(isA<NetworkFailure>()),
    );
  });
}
```

What to say: "Because every layer depends on an interface, each test mocks exactly one neighbour. No test touches the network."

SECTION 18 — Interview Cheat Sheet (one page)

Architecture formula

```
UI → Cubit → UseCase → Repository(interface) → RepositoryImpl → DataSource → Dio → API
presentation ─► domain ◄─ data              (imports point INWARD)
```

Response formula

```
API → JSON → Model → toEntity() → Entity → Cubit → State → UI
```

Error formula

```
DioException → AppException (DioClient) → Failure (guard in Repository) → Cubit → Error State → Retry
```

SOLID formula

```
S = One responsibility        (Cubit = state, Repo = data source choice, Model = JSON)
O = Extend without modifying  (new endpoint = new files + 1 DI line)
L = Substitutable abstraction (Fake/Impl behind the same interface)
I = Small interfaces          (1–3 methods per repository/data source)
D = Depend on abstractions    (Cubit→UseCase→Repository INTERFACE; Impl wired in DI)
```

Debug formula: when the API integration fails, check bottom → top

```
API        → hit it with curl/Postman first (is the server returning what you think?)
Dio        → baseUrl/path/query/headers correct? (enable the safe logger) timeout? internet permission/cleartext on Android?
DataSource → response.data runtimeType? does fromJson match the REAL JSON (snake_case, nulls, int vs String)?
Repository → is guard() mapping what you expect? did it throw a Failure?
UseCase    → called with the right arguments?
Cubit      → did it emit? is `on Failure` swallowing a TypeError you should see? isClosed?
UI         → is the right BlocProvider above this widget? does the `switch` handle the state?
```

(Frequent culprits: missing `INTERNET` permission in `AndroidManifest.xml` · `http://` blocked by iOS ATS / Android cleartext · `localhost` instead of `10.0.2.2` on the Android emulator · BlocProvider below the widget that reads it · `as int` on a value that is a `String`.)

13-step live-coding order

```
1 JSON → 2 Entity → 3 Model(+toEntity) → 4 Repo interface → 5 DataSource → 6 RepoImpl
→ 7 UseCase → 8 Cubit → 9 States → 10 UI → 11 DI/main → 12 RUN → 13 pagination/search
```

Gotcha checklist

- [ ] Interface in **domain**; no `import '…/data/…'` in domain
- [ ] Entities have **no** `fromJson`; models have `toEntity()`
- [ ] Cubit is a **factory**; closed over `isClosed` after `await`
- [ ] `on Failure catch`, user-safe messages, Retry button
- [ ] `loadMore` guarded (state type, `isLoadingMore`, last page)
- [ ] Footer/loader not a full-screen replace on page 2
- [ ] Debounce search; ignore stale responses
- [ ] Never log tokens/bodies in release; token in **secure** storage
- [ ] POST isn't auto-retried

File budget

	Files	Lines
LITE feature (MVP w/o pagination)	14	≈ 270
LITE + pagination	14	≈ 320
FULL (errors, DioClient, widgets, GetIt)	24	≈ 790

SECTION 19 — Verbal explanation scripts

20-second answer

> "It's Clean Architecture with feature-first folders. The UI talks to a Cubit; the Cubit calls a use case; the use case depends on a repository **interface** in the domain layer; the data layer implements it with Dio and maps JSON models to domain entities. Dependencies point inward, so the domain knows nothing about Dio or Flutter."

60-second answer

> "I'm following Clean Architecture with three layers per feature. **Domain** holds entities, the repository interface and use cases: pure Dart, no Flutter, no Dio. **Data** has the JSON models, a remote data source that owns the endpoint and parsing, and a repository implementation that maps models to entities and converts exceptions into failures. **Presentation** has a Cubit with sealed states and the widgets. Dependencies point inward: presentation and data both depend on domain, never the other way. That's why the repository interface lives in domain and the implementation in data. The only place that knows the concrete classes is the composition root, so I can swap REST for a cache or a fake in tests without touching the Cubit or UI. Errors flow `DioException → AppException → Failure → error state`, so the UI shows a friendly message and a Retry button. Pagination appends pages and ignores duplicate or last-page calls."

2-minute detailed answer

> "Let me walk it top to bottom. The **UI** is a stateless page with a `BlocBuilder` that `switch`es over a sealed `UserState`, so the compiler forces me to handle loading, loaded and error. It never fetches; it forwards intent: `fetchUsers`, `loadMore`. The **Cubit** holds the screen state. It calls a **use case**, `GetUsers`, which expresses one business action and is the seam where rules would live. The Cubit catches only my typed `Failure`, never a raw exception, and emits an error state with a user-safe message. For pagination it keeps `currentPage`, `totalPages` and `isLoadingMore`, appends new pages, and guards against double calls, loading-in-progress and the last page. If page 2 fails, the list stays and the footer offers a retry. The use case depends on the **repository interface**, which I put in the domain layer on purpose: the domain defines what it needs, and the data layer adapts to it, so import arrows point inward. The **repository implementation** calls the remote data source, converts the `UsersResponseModel` into a `PagedUsers` entity via `toEntity()`, and wraps everything in `guard()` which converts exceptions into failures, so Dio types never leave the data layer. The **data source** knows the endpoint and the JSON: strict casts so a wrong shape fails loudly instead of rendering blank rows. HTTP goes through a `DioClient` wrapper with all five verbs and a single `DioException → AppException` mapping covering timeouts, no internet, 4xx/5xx with the server's message, and invalid JSON. **Dio** is configured once, with base URL, timeouts and interceptors: auth token attached in an interceptor, refresh-on-401 with one shared refresh call, and debug-only logging without headers or bodies. Everything is wired in one composition root: manual DI for one feature, GetIt as it grows, with Cubits as factories because `BlocProvider` closes them. Because each layer depends on an interface, tests mock exactly one neighbour: bloc_test for the Cubit, a mocked data source for the repository, and a fake repository for a widget test. SOLID falls out of this: single responsibility per class, open for new endpoints without modification, and dependency inversion at the repository boundary."

Handling "why did you skip X?"

> "I prioritized a correct, layered, running screen first. X is a hardening step; here's exactly where I'd add it and what it would look like." *(Then point to the file.)*

SECTION 20 — Common interview questions (34, beginner → advanced)

Beginner

1. What is Dio? A powerful Dart HTTP client: interceptors, `BaseOptions`, timeouts, cancellation, form data, multipart, typed errors (`DioException`). I use it over `http` for interceptors (auth/refresh/logging) and consistent error types.

2. What is a REST API? An HTTP API organised around **resources** (`/users/7`) with verbs: GET read, POST create, PUT replace, PATCH partial update, DELETE remove; stateless, JSON bodies, meaningful status codes.

3. What is JSON? A text format for structured data (objects, arrays, strings, numbers, booleans, null). In Dart it decodes to `Map<String, dynamic>` / `List`; I turn it into typed models with `fromJson`.

4. What is the Repository pattern? An abstraction that hides *where* data comes from (REST, cache, DB). The domain asks the repository for entities; the implementation chooses sources, maps models and converts errors.

5. What is a Cubit? A lightweight state holder from `flutter_bloc`: methods call `emit(newState)`; the UI rebuilds from the state stream. A Bloc minus events.

6. What is BLoC? Business Logic Component: events in, states out, via streams. It adds traceability and event transformers (`droppable`, `restartable`) at the cost of boilerplate.

Intermediate

7. Why Clean Architecture? Testability, replaceable infrastructure (swap Dio/DB without touching business logic), parallel team work per layer, and a place for every concern. Cost: more files; not worth it for a one-screen throwaway.

8. Why a UseCase? One named business action, a home for rules and orchestration across repositories, and a mockable seam for Cubit tests. For pure pass-through reads it's optional; I include it for consistency.

9. Why a Repository interface? Dependency inversion: domain defines the contract, data implements it. Enables fakes in tests and swapping implementations without touching callers.

10. Model vs Entity? Model = API shape (JSON-aware, in data). Entity = business shape (pure, in domain). `toEntity()` is the boundary so backend changes don't ripple.

11. Where should API calls happen? Only in the **data layer's remote data source** (through `DioClient`). Never in widgets or Cubits.

12. How do you handle errors? `DioException → AppException` in `DioClient`, `AppException/TypeError → Failure` in a repository `guard()`, Cubit catches `on Failure` and emits an error state with a friendly message and Retry. Typed failures let me treat 401, offline and 422 differently.

13. How do you implement pagination? State holds list + `currentPage`, `totalPages`, `isLoadingMore`. `loadMore()` guards against wrong state / in-flight / last page, fetches `currentPage+1`, **appends**. Footer shows a spinner or Retry. Initial load is a separate full-screen state.

14. Why sealed classes for state? Exhaustive `switch`: add a state and every UI switch fails to compile until handled. No `default:` hiding bugs.

15. Why `Equatable` on states? Value equality → no rebuilds for identical emissions and `bloc_test` can compare sequences with `==`.

16. Why is the Cubit a factory in DI? `BlocProvider(create:)` closes it on dispose; a singleton would be closed after first use. Stateless things (Dio, repository, use case) are singletons.

17. GetIt vs manual DI? Manual composition root for one feature (simplest); GetIt when features multiply, for lazy singletons and test overrides.

18. Cubit vs Bloc? Cubit for request/response screens (faster, simpler). Bloc for event-heavy flows needing transformers or an event log.

19. What does `if (isClosed) return;` do? After `await`, the user may have left the screen and the Cubit closed; emitting then throws `StateError`. The guard makes it a no-op.

20. `BlocBuilder` vs `BlocListener` vs `BlocConsumer`? Builder = draw UI from state. Listener = one-off side effects (snackbar, navigation). Consumer = both. Never navigate from `builder`.

21. Where do base URL, timeouts and headers go? `BaseOptions` on the Dio created once at the composition root; paths as constants; auth header in an interceptor.

Advanced

22. How do you refresh an expired token? Interceptor `onError`: on 401, call refresh with a **separate Dio** (no auth interceptor), store the new tokens, retry the original request once with the new header. Share one in-flight refresh `Future` across parallel 401s. If refresh fails → clear storage, surface `UnauthorizedFailure`, go to login.

23. How do you retry failed requests? Only **idempotent** calls (GET) and only for **network** failures, with exponential backoff and a max attempt count. Never auto-retry POST (duplicates). A small `retry()` helper around the data-source call or a retry interceptor.

24. How do you prevent duplicate pagination calls? Guard in the Cubit (`isLoadingMore`, last page, state type). In Bloc: `droppable()` transformer. UI can fire as often as it likes.

25. How do you cancel API requests? Dio `CancelToken`: keep one per search/screen, `cancel()` on new input or `close()`. Dio throws `DioException(type: cancel)`, which I map to a "cancelled" outcome that the UI ignores. In Bloc, `restartable()` cancels the previous handler.

26. How do you handle concurrent requests? Independent calls: `Future.wait` (or Dart 3 records `(await (a, b).wait)`) for parallel loading; combine in a use case. Dependent calls: sequential. Guard against races on one resource: ignore stale responses (compare request id/query), `droppable` / `restartable`, de-dupe in-flight futures.

27. How do you cache API responses? A repository **decorator**: `CachedUserRepository(UserRepositoryImpl)` with TTL (in-memory map) so UI/use case don't change (verified code below). For persistence use a DB/ `hive` / `drift`; for HTTP semantics honour `ETag` / `Cache-Control` (`dio_cache_interceptor`). Invalidate on mutations. Strategy names: cache-first, network-first, **stale-while-revalidate**.

28. How do you implement offline-first? The **local DB is the single source of truth**; UI observes the DB (stream). The repository syncs: fetch remote → upsert DB → UI updates. Writes go to the DB + an **outbox queue** retried when online (idempotency keys, conflict policy: last-write-wins/server-wins/merge). Connectivity is a hint, not truth: always handle failures.

29. How do you test a Cubit? `bloc_test`: mock the use case, `build` → `act` → `expect` state sequence; `seed` for starting states; cover success, failure, pagination, last page and page-2 failure.

30. How do you test a Repository? Mock the data source; assert model→entity mapping and each exception → failure mapping (404, 401, 422, network, parsing).

31. How do you handle API versioning? Version in the base URL (`/v2`) or header; keep **models per version** mapping to the **same entity**, so the domain doesn't change; migrate data sources one at a time; feature-flag the switch.

32. How do you keep large JSON parsing off the UI thread? `compute()` / `Isolate.run()` for payloads in the hundreds of KB (e.g. `Isolate.run(() => UsersResponseModel.fromJson(json))`); not needed for small lists, so measure first.

33. Cursor vs page-number pagination? Page-number/offset is simple but unstable under inserts/deletes; **cursor** tokens are stable and scale. With cursors the state stores `nextCursor` instead of `currentPage`.

34. How do you secure the app's network layer? HTTPS only; tokens in secure storage; never log secrets; short-lived access tokens + rotating refresh tokens; certificate pinning for high-risk apps (with a rotation plan); obfuscate release builds; validate/sanitize server data via strict models.

Cancel + cache snippets referenced above (verified):

```dart
import 'package:dio/dio.dart';

import '../feature/user/domain/entity/paged_users.dart';
import '../feature/user/domain/repository/user_repository.dart';

// ---------- 1. Cancel in-flight requests (e.g. user typed a new search term)
//   In DioClient.get(...) accept `CancelToken? cancelToken` and pass it to Dio.
//   In the Cubit:
class Cancellable {
  CancelToken? _token;

  CancelToken next() {
    _token?.cancel('superseded');
    return _token = CancelToken();
  }
}

// ---------- 2. In-memory cache with TTL, as a Repository DECORATOR.
//   UI / Cubit / UseCase don't change. DI wires CachedUserRepository(UserRepositoryImpl(...)).
class CachedUserRepository implements UserRepository {
  CachedUserRepository(this._inner, {this.ttl = const Duration(minutes: 5)});

  final UserRepository _inner;
  final Duration ttl;
  final _cache = <int, ({PagedUsers value, DateTime at})>{};

  @override
  Future<PagedUsers> getUsers({required int page}) async {
    final hit = _cache[page];
    if (hit != null && DateTime.now().difference(hit.at) < ttl) return hit.value;

    final fresh = await _inner.getUsers(page: page);
    _cache[page] = (value: fresh, at: DateTime.now());
    return fresh;
  }
}
```

SECTION 21 — The Final Live-Coding Template (memorize this)

When the interviewer says **"Integrate this API."**, think:

```
 1. Understand JSON        → paste the response into a comment; note envelope, nullable fields, types
 2. Create Entity          → what the SCREEN needs (domain, plain Dart)
 3. Create Model           → mirror the JSON + toEntity()
 4. Create Repository IF   → abstract, returns entities, in domain
 5. Create DataSource      → endpoint + parse + (LITE) map DioException → Failure
 6. Create RepositoryImpl  → remote → toEntity()
 7. Create UseCase         → one-line call()
 8. Create Cubit           → fetch (+ loadMore)
 9. Create States          → sealed: Initial / Loading / Loaded / Error
10. Create UI              → switch on state: loader · error+retry · list
11. Connect DI             → manual in main.dart (Dio → DataSource → Repo → UseCase → Cubit)
12. Test API               → RUN. If it fails: debug bottom → top (section 18)
13. Add pagination/search  → only if asked / time remains
```

The shortest implementation sequence (LITE, ~270 lines to a running screen)

#	File	~Lines	Minute
1	`core/constants/api_constants.dart`	4	1
2	`domain/entity/user.dart`	15	3
3	`domain/entity/paged_users.dart`	9	3
4	`data/model/user_model.dart` (+ `toEntity`)	28	5
5	`data/model/users_response_model.dart`	19	6
6	`domain/repository/user_repository.dart`	5	6
7	`core/errors/failure.dart`	5	6
8	`data/data_source/user_remote_data_source.dart`	31	8
9	`data/repository/user_repository_impl.dart`	13	9
10	`domain/usecase/get_users.dart`	10	9
11	`presentation/cubit/user_state.dart`	41	11
12	`presentation/cubit/user_cubit.dart` (add `loadMore` later)	50	12
13	`presentation/pages/user_page.dart`	55	14
14	`main.dart` (manual DI)	36	15
▶	**RUN**		**15**
15	`loadMore()` + footer	(in 12/13)	18
16	Explain		20

(If you only have time for one layer to be perfect, make it **the boundary**: interface in domain, `toEntity()`, and an error state with Retry.)

Minute-by-minute self-check (glance at the clock)

- **min 5**: entity + model + interface done? If not → skip `UserResponse` generics, hard-code the concrete types.
- **min 9**: data layer done? If not → drop the data-source interface.
- **min 13**: Cubit + states done? If not → call repository from Cubit, say "use case next".
- **min 15**: app running? If not → debug bottom→top *now*, stop adding features.

The 15-second close

> "That's UI → Cubit → UseCase → Repository interface → Impl → DataSource → Dio, with entities in domain and models in data. I'd harden it with typed failures, a DioClient, an auth interceptor with refresh, and tests, and I can add any of them next."

➡️ Next: reproduce **LITE** from memory on a blank project twice, against `https://reqres.in/api/users?page=1`; time it. Then once with search added.